SequoiaDB Information Center

Contents

SequoiaDB Database Description

SequoiaDB database is a kind of new corporation distributed non-relation database. It helps corporation to reduce IT cost, and provides a steady, dependable, efficient and flexible underlying platform.

Advantage

- Through non-structure storage and distributed processing, SequoiaDB provides near-linear scalability without caring about underlying storage limit.
- SequoiaDB provides high-useability accurate to sharding level, prevents the system from going down caused by problems of server, machine room or human false, and keeps data availibale on line in 24x7 hours.
- SequoiaDB provides complete corporation functionalities and enables users to conveniently manage high-concurrent tasks and big data analysis programs.
- Hasing strengthened non-relation data module, SequoiaDB helps corporations to rapidly develop and deploy applications in order to achieve the high-flexiblity of applications.
- SequoiaDB guarantees data eventual consistency, which eradicates data loss.
- In SequoiaDB, on-line applications and big data analysis programs share the same background database at the same time. In SequoiaDB, data analysis programs and on-line application in the same system are invisible to each other because of read/write splitting.

Features

With corporation requirements being more and more complex and diverse, and business being rapidly expanded and producing vast data, IT departments need to provide their users with more and more information. Meanwhile under the current background of economy, IT department needs to not only offer efficient service but also reduce the cost of device and program maintenance.

SequoiaDB database provides massive cluster data platform based on PC server, enables IT departments to provide steady, dependable and efficient data service, and greatly reduce the cost of development, device and program maintenance of applications in IT departments.

Through deploying and using SequoiaDB database, users can gain:

Horizontal Scalability

Traditional relation database cannot acheive horizontal scalability, but SequoiaDB can perfectly solve this problem. Through vertical splitting of data and the application of non-relation data model, SequoiaDB greatly break through the bottleneck of massive data exchange within shardings in relation database. In this way, it acheive horizontal scalability.

Permanent High-useability

SequoiaDB stores multiple real-time copies of every piece of data, in order to prevent the system from going down caused by problems of server, machine room or human false, and keeps data availibale on line in 24x7 hours.

Complete Corporation Support

SequoiaDB provides corporation with user-friendly and complete management, maintenance and monitoring user interface, and 24x7-hour telephone and present technical support.

Strengthened Non-relation Data Module

SequoiaDB uses the structure called JSON, flexibly simplifies the data maintenance in relation database. In SequoiaDB, data is stored in type of object according to the requirement of applications, which greatly reduce the cost of developmetn and maintenance of application.

Data Eventual Consistency

SequoiaDB guarantees data eventual consistency in the massive distributed infrastructure, which meets users' requirement of real-time manipulation and data consistency.

Combination of On-line Applications and Big Data Analysis

In SequoiaDB, data analysis programs and on-line application in the same system are invisible to each other because of read/write splitting. Big data analysis is acheive through Hadoop technology.

SequoiaDB is the first corporation file-based non-relation database in the world. It is aimed at serving users with a distributed data platform with high-scability, high-useability, high-quality and easy to maintain. SequoiaDB meets users' needs of big data analysis and low cost.

Data Model

SequoiaDB use the data model called JSON rather than traditional relation data module.

The full name of JSON is *Javascript Object Notation*. It is a kind of lightweight data-exchange format. It is easy for human to read and write JSON. It is also easy for machine to generate and parse JSON.

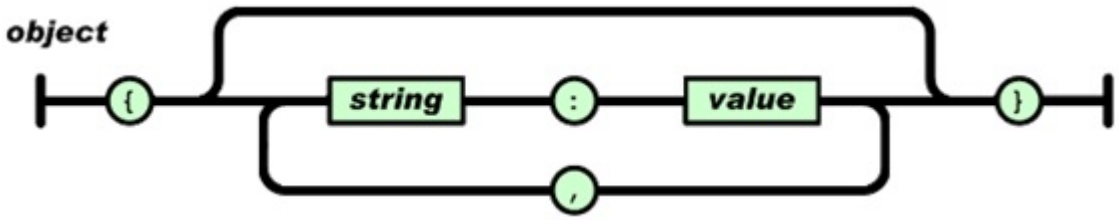
It is based on a subset of *JavaScript Programming Language, Standard ECMA-262 3rd Edition – December 1999*. It is in type of text. JSON supports nesting structure and array.

The generation of JSON is based on two kinds of structure:

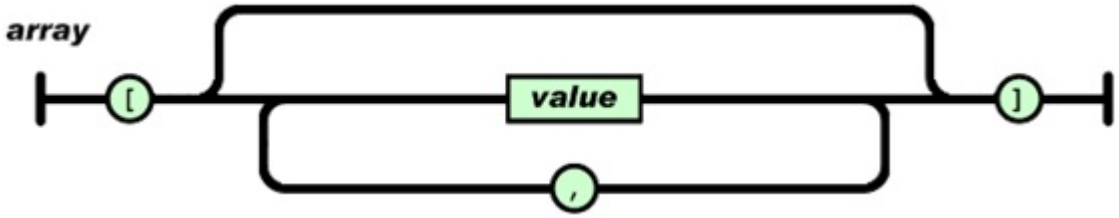
- Key-value pair collection. In the structure of key-value pair collection, aevery data element contains a name and a value. The value in it can be in the type of figure, string and other common types, or nested JSON object and array.
- Array. Every element in a array doesn't contain element name.The value in it can be in the type of figure, string and other common types, or nested JSON object and array.

The format of JSON may be:

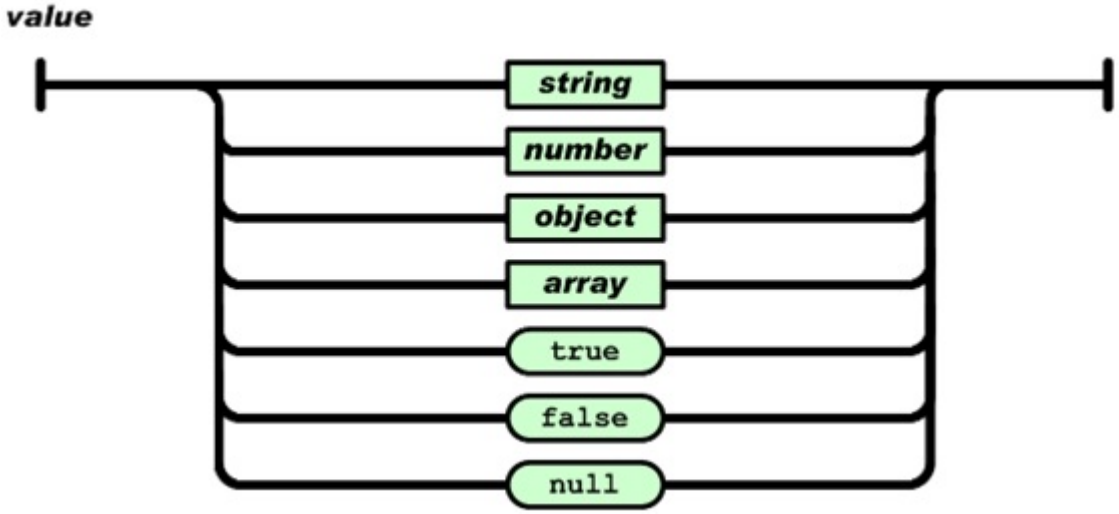
- The object is a unordered "key-value pair" collection. It begins with "{" (left brace), and ends with "}" (right brace). Every element name is followed with a ":" (colon). Elements are seperated with "," (comma).



- Array is a ordered value collection. It begins with a "[" (left bracket), and ends with "]" (right bracket). Values are seperate with "." (comma).

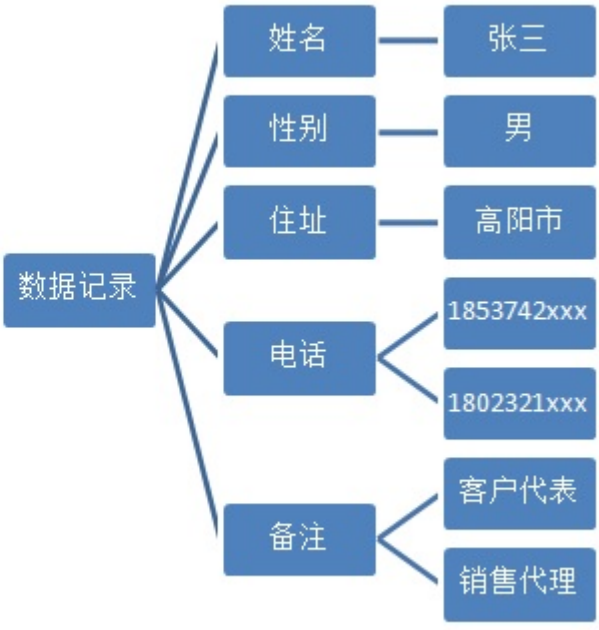


- Value is in type of string, figure, true, false, null, objdect, array, and special data structures in SequoiaDB (such as date, time .etc). It is surrounded with double quotations.



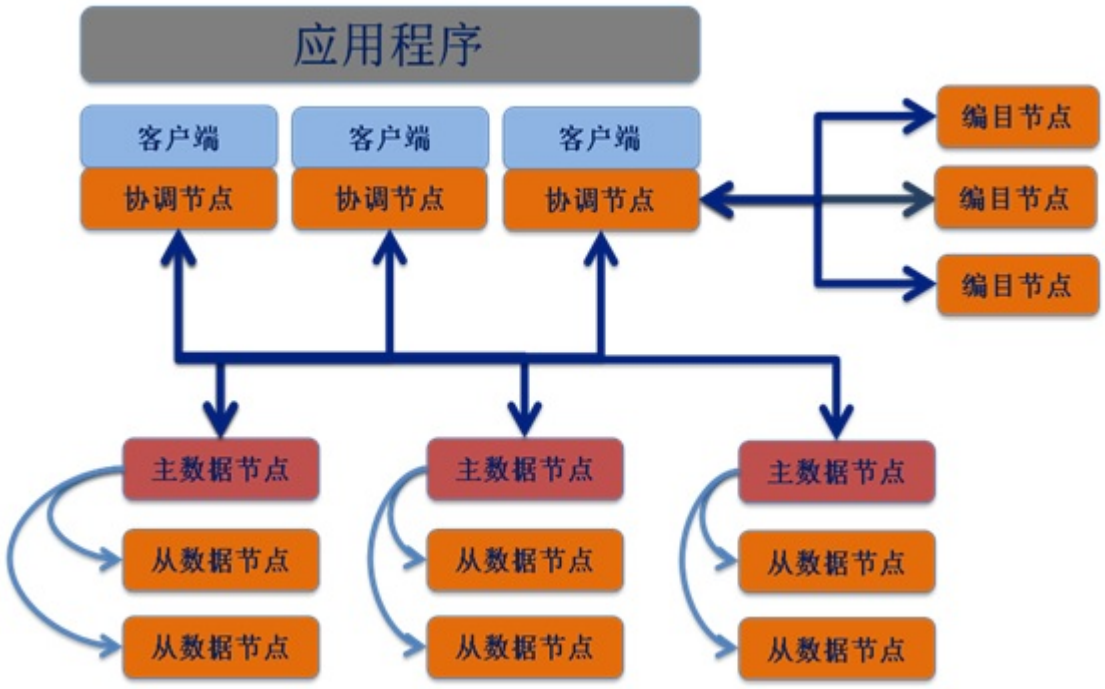
A typical nesting structure is as follow:

```
{
    "姓名": "张三",
    "性别": "男",
    "住址": "高阳市",
    "电话": [
        1853742xxx,
        1802321xxx
    ],
    "备注": [
        "客户代表",
        "销售经理"
    ]
}
```



Infrastructure

SequoiaDB applies distributed structure. The picture below describes a general summary of SequoiaDB system architecture.



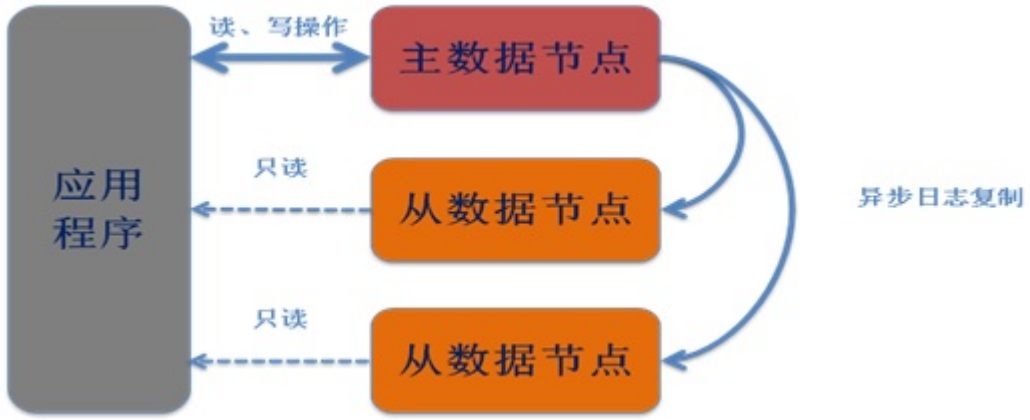
In a client terminal (or an application terminal), local or/and remote applications are linked to SequoiaDB client libary. Local or/and remote applications communicate with catalog node under TCP/IP protocol.

Catalog node doesn't store any user data. It is only a node that receive requests and distribute them to target data nodes.

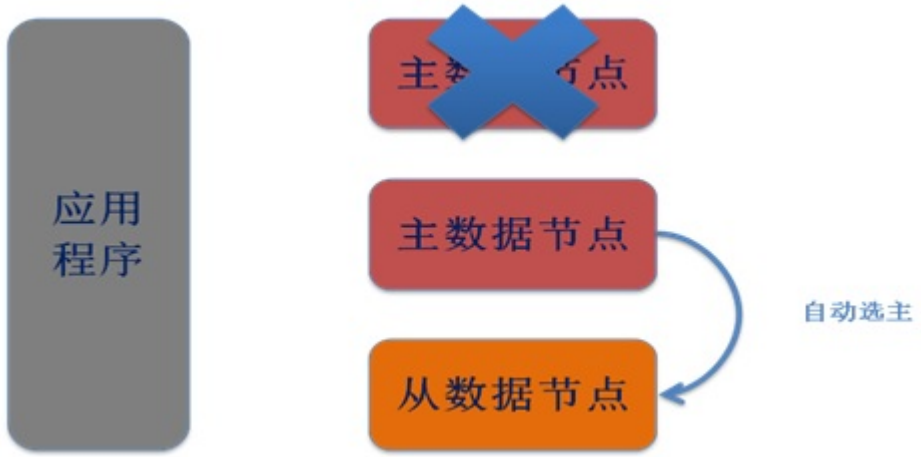
Catalog nodes store system metadata information. Coord nodes get the location of data on data nodes by communicating with catalog nodes. One or more catalog nodes can constitute replset cluster.

Data nodes are used to store users' data information, One or more data nodes can constitute a replset. In a replset, data in all data node is eventually consistent. Data replset is also called shard. Different shards store different data.

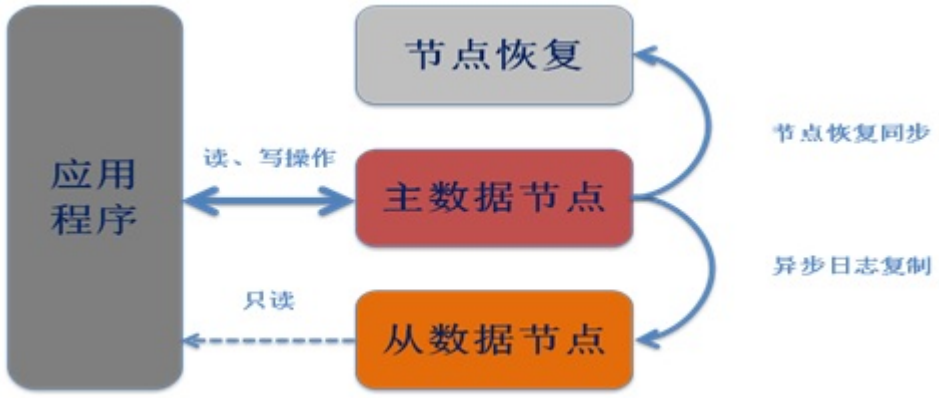
Every shard contains one or more data nodes. When there are several nodes in it, asynchronous replication is fulfilled. In a shard, there are master node and several slave nodes. Main nodes allows read and write operations. Slave nodes allows read operations.



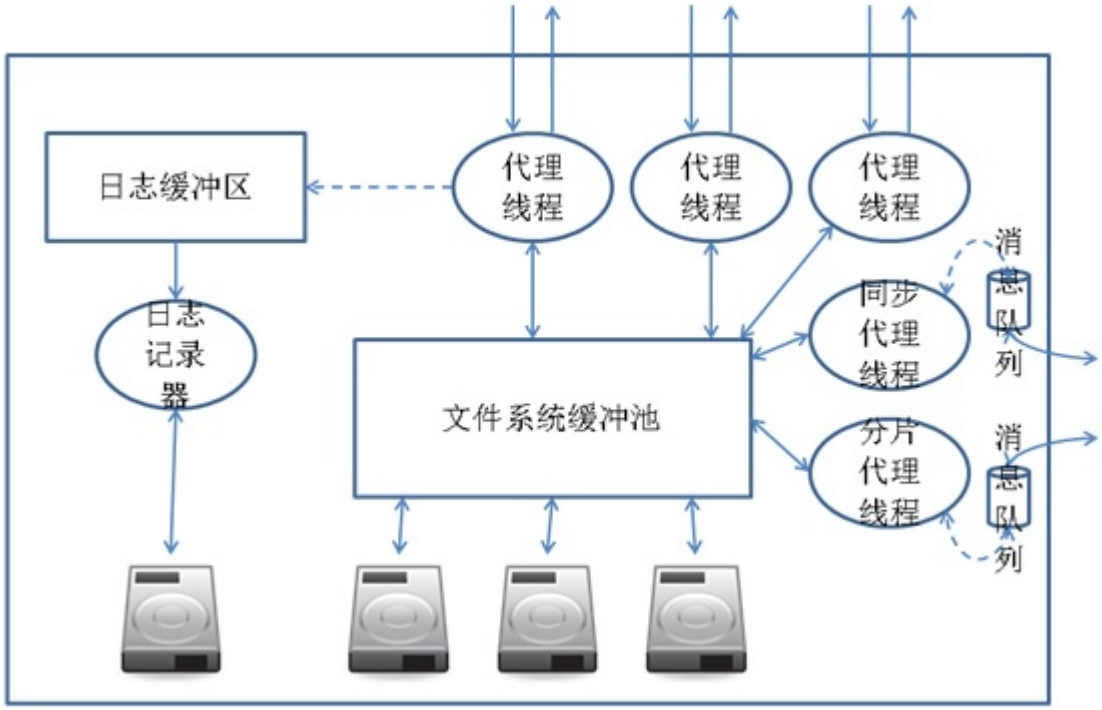
If slave nodes are off-line, master node can work well as usual. When master node goes down, slave nodes will automatically vote and elect a new master node to cope with write requests.



When broken-down nodes recover or new nodes join a shard, replication will be fulfilled between data nodes in order to guarantee the consistency of master node and slave nodes.



The architechture of a single node is as follow:



In data nodes, activities are controlled by EDU. Every node is a process in operating system. Every EDU is a thread in data nodes. Agent thread copes with user request from outside. Synchoronous agent thread copes with synchoronized transaction within shards in cluster. Shard agent thread copes with synchoronized transaction between shards in cluster.

All write operations are recorded in log cache. Log recorder will asynchronously write them into disk.

User data is written into file system cache pool by agent thread. Then operating system will asynchronously write them into underlying disk.

Database Concept

Database concepts related content.

Database

Database objects inlucde document, collection, collection space and index.

Document

Concept

In SequoiaDB, document is in the format of JSON, also named as record. In database, JSON data is stored in the format of BSON after transformation. Generally, a document is consisted of one or more fields. Every field is consisted of 2 parts: key and value.This is a document containing 2 fields:

{ "name" : "Zhangsan", "sex" : "male" }

Note:

BSON documents may have more than one field with the same name;however,most SequoiaDB interfaces represent SequoiaDB

with a structure(e.g a hash table) that does not support duplicate field names,if you need to manipulate documents that have more

than one fiels with the same name,see your driver's documentation for more information.

Some documents created by internal SequoiaDB processes may have duplicate fields,but no SequoiaDB process will ever add

duplicate keys to an existing user document.

Field type

The key of a field is in the format of string, but value can be in the format of figure, string, nested JSON project, nestd array.etc. SequoiaDB support these types:

Type	Definition	Sample
Integer	Integer，range from -2147483648 to 2147483647	{ "key" : 123 }
Long	Integer，range from -9223372036854775808 to 9223372036854775807. If a value is not suitible to an Integer, it is automatically transform into Long by SequoiaDB.	{ "key" : 3000000000 }
Float	Float, range from 1.7E-308 to 1.7E+308	{ "key" : 123.456 }or{ "key" : 123e+50 }
String	String within double quotations	{ "key" : "value" }
Object ID（OID）	12-byte object ID	{ "key" : { "$oid" : "123abcd00ef12358902300ef" } }
Bool	true or false	{ "key" : true }or{ "key" : false }
Date	In the format of YYYY-MM-DD	{ "key" : { "$date" : "2012-01-01" } }
Timeline	In the format of YYYY-MM-DD-HH.mm.ss.ffffff	{ "key" : { "$timestamp" : "2012-01-01-13.14.26.124233" } }
Bindata	Base64 binary data	{ "key" : { "$binary" : "aGVsbG8gd29ybGQ=", "$type" : 1 } }

Type	Definition	Sample
Regex	Regular expression	{ "key" : { "$regex" : "^Zhang", "$options" : "i" } }
Object	Nested JSON document object	{ "key" : { "subobj" : "value" } }
Array	Nested array object	{ "key" : ["abc", 0, "def"] }
NULL	null	{ "key" : null }

Field order

Fields are unordered. The order of fields may be changed in the process of data manipulation.

Within a nested object, fields are invoked in the format of "object.field". For example,

```
{ "name" : "zhangsan", "address" : { "street" : "shuilan street", "city" : "xx", "block" :
        "yy" } }
```

The field of "city" is invoked as "address.city".

Others

- The max size of each document is 16MB
- Document should contains "_id" field. If users do not offer it, syetem will automatically generate a field in the type of object ID.
- "_id" is unique in a collection
- Field name in document should not begin with the character "$".
- Field name in document should not contain ".".

Array

Concept

Arrays in SequoiaDB, in the format of JSON object in documents, are generally named records.

Format

If there are multiple values in an array, users can store data in the format of data structure. Array begins with "[" and ends with "]". It contains 0 or multiple values.

```
{ Field Name : [ <Value1>,, <Value2>, <Value3> ... ] }
```

Sample

Array can contain different data types as values. The index of elements start with 0.For example:

```
{ "key" : [ "hello", "world" ] }
```

The index of "hello" is 0. The subscript value of "world" is 1. The values in an array are in order. The order of values is unchangable when operation is implemented in an array.

Element in array is invoked as "field name.index". For exmaple, in the sample above, "world" is invoked as "key.1".

Object ID

Concept

Object ID is a 12-bit BSON object. It contains:

- 4-byte timeline measured in seconds
- 3-byte system (physical machine) identity
- 2-byte process ID
- 3-byte sequence number that starts with a random number

4-byte timeline	3-byte system identity	2-byte process ID	3-byte sequence number

The object ID can identify each process of each system in cluster environment with 16777216 different values, so it is regarded as a globally unique value in cluster environment.

In SequoiaDB, each file in a collection stores at least an "id" field , which is unique within the collection.

Format

The format of an object ID is:

```
{ "$oid" : "<24-byte hexademical string>" }
```

Sample

Object ID is desplayed as:

```
{ "key" : { "$oid" : "5156c192f970aed30c020000" } }
```

Date

Concept

In SequoiaDB, date is in the format of YYYY-MM-DD. It is transformed into a 4-byte integer when it is stored in the database.

Form

The format of date is：

```
{ "$date" : "<YYYY-MM-DD>" }
```

Sample

For example:

```
{ "createTime" : { "$date" : "2012-05-12" } }
```

Timeline

Concept

In SequoiaDB, timeline is in the format of YYYY-MM-DD-HH.mm.ss.ffffff. It is transformed into an 8-byte integer when it is stored in the database.

Format

The format of timeline is：

```
{ "timestamp" : "YYYY-MM-DD-HH.mm.ss.ffffff" }
```

Sample

For example：

```
{ "createTime" : { "$timestamp" : "2012-05-12-13.15.21.241523" } }
```

Bindata

Concept

In SequoiaDB, data is read in the format of JSON object. So users should encode bindata with Base64, and send it to database in the format of string.

Format

The format of bindat is as follow

```
{ "$binary" : "<data>", "$type" : "<type>" }
```

In this format, "data" should be encoded with Base64. "Type" is a decimal value between 0 and 255. Users can choose type values in this range to identify types in applications.

Base64 is an universal format of transforming data, which is mainly transforming bindata into byte stream in the format of ASCII string. Generally, data becomes longer after transformation.

In order to save storage space, in SequioaDB, Base64 data is transformed into original data before being stored in database. When users request to read data, it is transformed and provided in the format of Base64 again.

Sample

For example, string "hello world" is encoded with Base64 into "aGVsbG8gd29ybGQ=".

JSON data containing "hello world" bindata, with type of "1" is:

```
{ "key" : { "$binary" : "aGVsbG8gd29ybGQ=", "$type" : 1 } }
```

Regex

Concept

SequoiaDB can search for user data with regex.

Format

The format of input is :

```
{ "$regex" : "regex", "$options" : "optoins" }
```

Regular expression is a regex string. Options can be:

Options	Description
i	Case-insensitive.
m	Allow mutiple match; when the parameter is set, "^" and "&" respectively matches characters before and after a line break.
x	Ignore blank characters matched by regex. When blank character is needed, it should be prefixed with "₩", the escape character.
s	Allow to match line break with ".".

When using optoins, users can choose several options at a time.

Sample

Users can match string that is case-insensitive and begins with the character "W" with:

```
{ "key" : { "$regex" : "^W", "$options" : "i" } }
```

Please refer to Perl Regex Manual for more about rules of regex.

Collection

Concept

Collection a logical object where documents are stored. Every document belongs to one and only one collection.

Collection consists of "<collection space name>.<collection name>". The length of a collection name is at most 127 bytes, encoded with UTF-8. A collection contains zero or more documents. (The max amount of documents depends on the size of collection space.)

In sharding enviroment, apart from name, every collection contains three attributes.

field name	description
Sharding Key	Specified sharding key. In collections, the field specified by sharding key is used as sharding information of document. Every document is put in its corresponding sharding.
ReplSize	It specifies the default replicaiton size in the collection. If its value is less than 1, write request will return when one copy is written successfully. If its value is more than 1, write request will return when the amount of successfully written copies euqals at least the value of ReplSize.
Compressed	Specified the value of "Compressed" means whether stored data in compressed form, it has two values:true and false, false is in default.

Collection Space

Concept

Collection Space is physical object where collections store. Every collection belongs to onre and only one collection space.

The length of collection space name is at most 127 bytes, encoded with UTF-8. A collection space contains at most 4096 collections. Every data node contains at most 4096 collection spaces.

Every collection space has a corresponding file on the data node. The format of file name is <collection space name>.1”。

Page

A file is departed in to several fixed-size pages by space collection. When creating a collection space, users can specify the size of a page in it, which is unchangeable.

In every sharding node, a collection space can visit at most 16777216 different pages. According to pages of different sizes, the max size of a single sharding is:

Size of page (byte)	Collection space max capacity (GB)
4096	64
8192	128
16384	256
32768	512
65536	1024

Block

A block is consisted of one or more pages. Every collection in collection spaces is consisted of zero or more blocks. The size of block is changeable according to the length of data. Every document is stored in only one block.

Blocks in a collection space are stored as follow:

In this chart, there are 3 collection in a collection space, represented by 3 colors. Data of every collection is stored in its corresponding page. Data block is consisted of one or more sequential data pages. A document cannot be divided and stored in different data blocks.

Database server

Concept

Database server provides software services for the purpose of managing information securely and efficiently.

A database server is a computer that has installed SequoiaDB database engine. SequoiaDB engine is the basic unit of access to data. In distributed architecture, every database is regarded as a node. These nodes share nothing.

In a computer, every SequoiaDB engine has a database path. All the collection spaces of the database is located in the directory.

Database path contains one or more collection spaces. In a data node or a coord node, there should be at least one SYSTEMP system temporary collection space. In a catalog node, there should be at least one SYSTEMP system temporary collection space and one SYSCAT system catalog collection space.

A database engine contains 4096 collection spaces at most.

Index

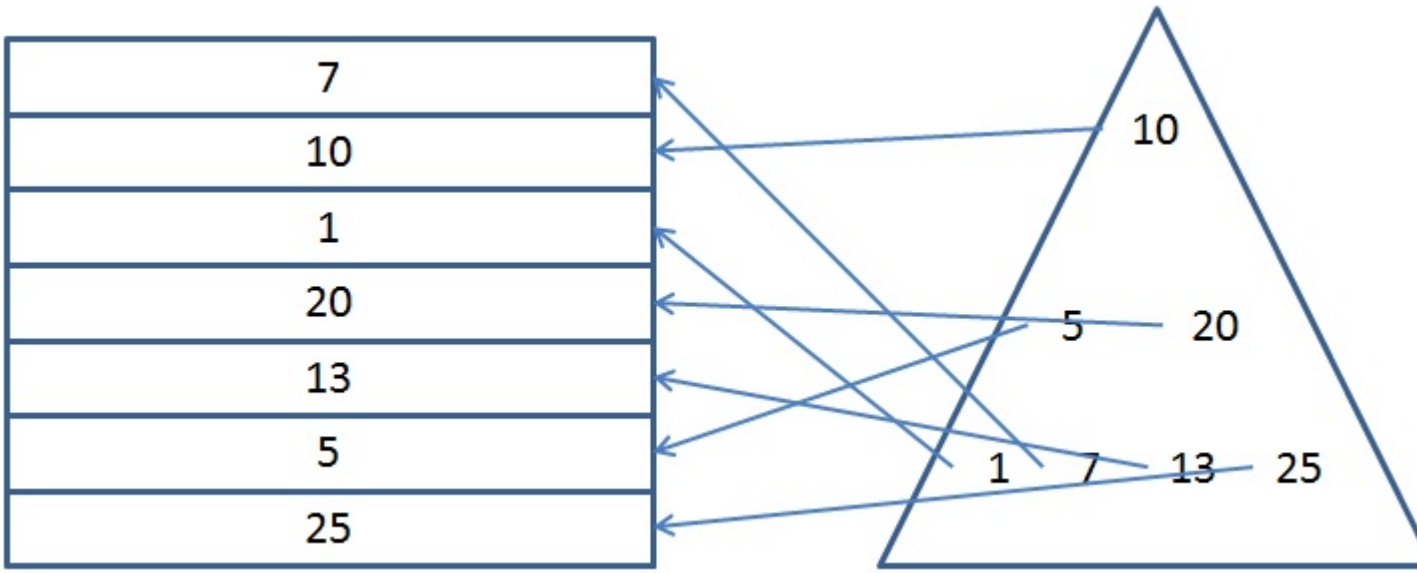
Concept

In SequoiaDB database, indexes are a kind of special objects. Instead of being used to store user data, indexes, a kind of special metadata, optimize speed and performance in finding relevant documents for a search query.

Every index is put in a colleciotn. A collection contains 64 indexes at most.

Indexing is regarded as an approach to sort data based on one or more fields, and rapidly get results which match users' query statements from it as soon as queries are submitted. In SequoiaDB, indexes are stored in structures called B-trees.

A typical index structure is as follow：



In this figure, the data is on the left, the indexes are on the right in the triangle. Indexes are sorted in the structure of trees in ascending order. every document will have a unique index value.

Through tree tranversal, the position of specific data can be rapidly found.

SequoiaDB can create indexes to any type of data. An index contains:

Attribute	Description
name	Index name. Index name is unique in a collection.
key	Index key, as a JSON object, contains one or multiple indexes field and direction field. If direction field is 1 or -1, the order is ascending or descending.
unique	"Unique", a optonal parameter, shows that whether an index is unique. The default value of it is "false". When its value is "true", the index is unique. As for unique index, there should not be repetitive records in a collection.
enforced	"Enforced" shows that whether an index is enforced to be unique. The default value is "false". If its value is "true", with "unique" set "true", there is no more than one null index key.

In SequoiaDB, every collection contains an enforced unique index named "$id". The index contains a index key of "_id".

An index named "$shard" is automatically is generated when sharding collection is created. The index key of it is the sharding key specified by users.

Note:

In sharding collection, all unique indexes should contain all the fields specified by collection sharding key.

In sharding collection, "$id" index merely ensure the uniqueness of records within a single node. If users plan to create a globally unique field, they have to create an extra unique field, which contains all the fileds specified by collection sharding key.

Format

The format of index contains two fields: "name" and "key". The type of "name" is string. The type of "key" is JSON object.

```
{ "name" : "<index name>", "key" : "{ "<index field1>" : <direction> [, "<index field2>" : <direction> ...] },
 [ "unique" : <true|false> ], [ "enforced" : <true|false> ] }
```

The object of "key" contains at least one field. The field name of it is the field name that user search for. Its value is "1" or "-1". "1" represents ascending sequence of data. "-1" represents descending sequence of data.

Sample

- Non-unique index, index name: "employee_id_key", index field is forward "employee_id".

```
{ "name" : "employee_id_key", "key" : { "employee_id" : 1 } }
```

- Unique index, index name is "record_id_index", index field is forward "product_key" and reverse "record_key".

```
{ "name" : "record_id_index", "key" : { "product_key" : 1, "record_key" : -1 }, "unique" : true }
```

  In Delete on page 94this index, there are no two records with the same "product_key" and the same "record_key". (But if either of them is the same, it is still regarded as an unique index).

- Enforced unique index. The index name is "test index". The index field is ascending "test case name".

```
{ "name" : "test index", "key" : { "test case name" : 1 }, "unique" : true, "enforced" : true }
```

  In an enforced unique index, all records should follow the rule of unique index. There should not be more than one record that contains a null value of "test case name" field.

Transaction

Transaction is consisted of a series of actions by logical unit of work.In the same session(or connection),only allow exist one transaction in the same time.For another word,when the users create a transaction in a session,before the end of the transaction,the users can't create new transaction.

Transation performed as a complete unit of work.The operations in transaction execute either all succeed or all fail.In SequoiaDB, operations of transaction can only be insert、modify and delete data,others will not be included in transaction.It is saied that non-transactional operations will not perform rollback when execute transaction rollback action.If a collection or collectionspace have data relate to transaction operation,then the collection or collectionspace is not allowed to drop.

In default,transaction function is close,if users want to open it,need to configure param in the configuration file of nodes:transaction=TRUE;when create node,add json type of param:{transaction:YES}.

Eventually Consistent

In SequoiaDB,to improve data reliability and data read/write Splitting,we adopt "eventually consistent strategy" among ReplicaSets.Maybe,the data is not lastest in a moment,but ultimately the same.

Glossary

W：number of write-replicas

R：number of read-replicas

N：number of replicas

In SequoiaDB,the value of "R" is default set to "1" ,and not configurable.

By default,The master node in data ReplicaSets will return immediately after processing a writen request,that is W=1.Data syn will be done asny in the background (Sys-log) and achieved eventually consistent。At this moment,the data for extenal read request may not be the lastest.In the case of less demanding data consistency,this way will provide a optimal write-performance.

As the time of creating collection,you can set the value of "W" by "ReplSize" param.

- By default,W=1.
- If the value of ReplSize is equal "0",then the number of W will be changed based on the number of nodes in current ReplicaSets.That is,there have three nodes in a ReplicaSet at beginning,then W is equal 3;if adding a node,then W change to "4".

- if specify the number of W by manually,It can not exceed the number of nodes in current ReplicaSet .

Increaing the value of W can effectively improve the consistency and reliability of data.When the value of W is equal to the number of nodes in a ReplicaSet and write-request is done seccessfully,then the data for read-request are always the lastest.But this will reduce the write-performance.For attention,although,the value of W can be set to equal to the number of nodes,it not mean that the data in SequoiaDB hava strong consistency.When a Relica write failed(such as the disk is full),it may exist one or more data versions in a RelicaSet,this time,you may read the lastest data,and may also the older.When the failed replica get right,it will syn the lastest data from the master node,and achieve eventually consistent.

Read/Write Splitting

Write-request

All of write-requests will only be sent to the master node,no master,no write-request.

Read-request

It will based on the session(or the connection)randomly select one node in a replicaset for read-request(external transparent).If the last query operation(including query,fetch) returned success,then the next will not re-elect node,otherwise,the next query operation have to select node.If there have no available node,return failure.In one query operation,will not re-elect.

Cluster

SequoiaDB cluster is a mode designed to improve the efficience of data request by achieving parallel computing multiplied database servers

Using SequoiaDB cluster，a user can get:

High-qualified Data Visit

With parallel computing and executing distributed tasks on more than one data server, the system saves resource consumption on every nodes, gets higher throughput and improves the quality of data visit.

24x7 High Usability

Data replication in replsets guarantees data userability, so if any server doesn't works accidently, data can be visited as well.

Horizon dilation ability of database

Database can contain more data by adding more replsets. Database can cope with more requests by adding more nodes in a replset. The more database nodes, the higher global throughput.

Mode

Concept

Mode means the approach of starting SequoiaDB service. Generally, it is independent mode or cluster mode.

Independent Mode

Independent mode is the briefest mode to start SequoiaDB. It merely needs to start a data node of independent mode before having data service.

Under independent mode, SequoiaDB Database is a independent process which doesn't need to communicate with other clients.All the data is stored in data nodes.

In a database started under independent mode, it is not allowed to split space or replicate data. So it is not recommended to use independent mode if high data security is required.

In a database start under independent mode, there is no catalogue information.

It is recommended to use independent mode in development environment in order to reduce requirement of hard resource.

Cluster Mode

Cluster mode is the standard mode to start SequoiaDB, which require 3 nodes.

Under cluser mode, SequoiaDB needs 3 kinds of roles:

- data node
- catalogue node
- coordinating node

The minimum configuration under cluster mode contains at least one of each kind of roles, which is essential to build a complete cluster mode.

Under cluster mode,client and application can connect to coordinating node directly. Other kind of nodes are invisible to them.

s

Applications care nothing about the location of data in data nodes. Coord node can analyze request that it receives and automatically send it to relevant data nodes.

Under cluster mode, data set is unvisable to each other.In order to achieve data consistence, the data replication among nodes within a data set is asynchronous.

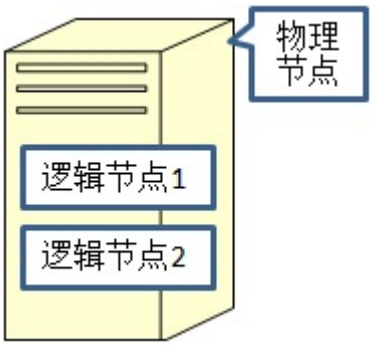
Node

Concept

In cluster mode of SequoiaDB, there are 2 kinds of nodes: physical nodes and logical nodes

Logical Node	Logical node is an independent service, representing the most basic database service logical unit. In Windows OS and Unix OS, a logical node is a sequoiaDB process. There can be mutiple logical nodes running on a compute.
Physical Node	Physical node is a server which runs an OS, representing the most basic database service physical unit. In virtual environment, a physical node is a vortual machine.

Multiple logical nodes can run on a physical nodes. Gnerally, each logical node on the same physical node listens to different ports, receives request from their corresponding ports, and returns results.



Catalog Node

Concept

Catalog node is a kind of logical node, which stores the metadata of a database system instead of data of other users.

Catalog node includes 3 space collections:

- SYSCAT system catalog contains 4 system collections:

collection name	descripton
SYSCOLLECTIONS	store user collection information in the cluster
SYSCOLLECTIONSPACES	store user collection space information in the cluster
SYSNODES	store information of logic nodes and replsets in the cluster
SYSTASKS	store information of background-runnning tasks in the cluster

- SYSTEMP system temporary collection space is used to create at most 4096 temporary collections in order to sequence data when users visit the database.
- SYSAUTH system validation collection space contains a user colleciton, which stores information of all the users in current system

collection name	descripton
SYSUSRS	stores all the user information in the cluster

Except for catalog nodes, all the other nodes in the cluster don't store any global metadata information in the disk memory. When users want to visit data on other nodes, the system will find collection information from local cache. If it is not found, the system will search for it in catalog nodes.

Catalog nodes contacts with other nodes mainly through catalog service port(catalogname parameter).

SYSCOLLECTIONS Collection

Collection Space

SYSCAT

Concept

SYSTASKS collection contains information of all user collections. Information of a user collection is saved in a corresponding file.

Every file contains the following fields:

Field Name	Type	Descrption
Name	String	Full name of a collection. The form is <collection space>.<collection name>.

Field Name	Type	Descrption
Version	Integer	Version number of the collection, starts with 1. It increases by 1 after any metadata alteration on the collection.
ShardingKey	Object	ShardingKey is in sharding collections. It contains one or more fields. The field name is its sharding field name. The value is 1 or -1, which represents forward or reserve sequencing.
CataInfo	Array	Information of logical nodes of a collection. In a single sharding collection, the arrary contains only one element, which represents the replset of the collection. In a multiple sharding collection, the array contains one or more elements, which represent the replset which covers all the ranges of values in the collection. Every range of values includes two value: LowBound and UpBound, which respectively represent floor and ceiling. The interval is left-close and right-open.

Sample

A typical single sharding collection is as follow:

```
{ "Name" : "test.foo", "Version" : 1, "CataInfo" : [ { "GroupID" : 1000 } ] }
```

A typical multiple sharding collection is as follow:

```
{ "Name" : "foo.test",
  "Version" : 1,
  "ShardingKey" : { "Field1" : 1, "Field2" : -1 },
  "CataInfo" : [
     { "GroupID" : 1000,
       "LowBound" : { "" : MinKey, "" : MaxKey },
       "UpBound" : { "" : MaxKey, "" : MinKey } } ]
  }
```

SYSCOLLECTIONSPACES Collection

Collection Space

SYSCAT

Concept

SYSCOLLECTIONSPACES collection contains information of all user collection spaces. Information of a user collection space is saved in a corresponding file.

Every file contains the following fields:

Field Name	Type	Descrption
Name	String	Collection space name.
Collection	Array	All the collection names of the collection space. Each collection is a JSON object, which contains the field "Name" and the name of the corresponding collection.
Group	Array	The replset ID of the collection space
PageSize	Integer	The size of a data page in the collection space

Sample

A typical collection space, stored in a replset, that contains one collection is as follow:

```
{ "Collection" : [ { "Name" : "foo" } ],
  "Group" : [ { "GroupID" : 1000 } ],
  "Name" : "test",
  "PageSize" : 4096 }
```

SYSNODES Collection

Collection Space

SYSCAT

Concept

SYSNODES collection contains information of all the nodes in the cluster and of replset. Information of a replset is saved in a corresponding file.

Every file contains fields as follow:

Field Name	Type	Describtion
GroupName	String	Replset name.
GroupID	Integer	Replset ID. It is unique in the cluster.
PrimaryNode	Integer	The ID of the master node in a replset.
Role	Integer	The role of a replset. It can be: • 0: data node • 2: cataloge node
Status	Integer	• 1: activate replset • 0: inactive replset • not exists: inactive replset
Version	Integer	Version number. It starts with 1. It increases by 1 after any execution on the replset.
Group	Array	It contains information of nodes in a replset,as it is shown as follow:

If there are more than one node in a replset, each node is store in Group as an object. Each object contains:

Field Nmae	Type	Description
HostName	String	The hostname which a node belongs to, should absolutely matches the output of the instruction, "hostname", that runs on the operating system in which the node exists.
dbpath	String	Database path, is the absolute path of a physical node that the node belongs to.
NodeID	Integer	Node ID,is unique in the cluster.
Service	Array	Server name. Every logical node corresponds to 4 services. Every service contains its type and server name (it may be port number of server name in service files). The types are as follow: • 0：direct service, corresponds to database argument svcname

Field Nmae	Type	Description
		• 1 : replicate service, corresponds to database argument replname • 2 : replset service, corresponds to database argument shardname • 3 : catalog service, corresponds to database argument catalogname

Note:

The group name of a catalog replset should be "SYSCatalogGroup". The ID of a catalog replset should be 1.

Data replset ID starts from 1000.

Data node ID starts from 1000.

Sample

A typical catalog replset that contains only one node is:

```
{ "Group" : [
              { "NodeID" : 2,
                "HostName" : "vmsvr1-rhel-x64",
                "Service" : [
                   { "Type" : 3, "Name" : "60003" },
                   { "Type" : 1, "Name" : "60001" },
                   { "Type" : 2, "Name" : "60002" },
                   { "Type" : 0, "Name" : "60000" } ],
                "dbpath" : "/home/sequoiadb/sequoiadb/catalog"
              } ],
              "GroupID" : 1,
              "GroupName" : "SYSCatalogGroup",
              "PrimaryNode" : 2,
              "Role" : 2,
              "Version" : 1 } }
```

A typical data replset that contains only one node is:

```
{ "Group" : [
              { "dbpath" : "/home/sequoiadb/sequoiadb/data3",
                "HostName" : "vmsvr1-rhel-x64",
                "Service" : [
                   { "Type" : 0, "Name" : "53000" },
                   { "Type" : 1, "Name" : "53001" },
                   { "Type" : 2, "Name" : "53002" },
                   { "Type" : 3, "Name" : "53003" } ],
                "NodeID" : 1001 } ],
                "GroupID" : 1001,
                "GroupName" : "foo1",
                "PrimaryNode" : 1001,
                "Role" : 0,
                "Status" : 1,
                "Version" : 1 } }
```

SYSTASKS Collection

Collection Space

SYSCAT

Concept

SYSTASKS collection contains information of all the running background tasks. Information of a task is saved in a corresponding file.

Every file contains the following fields:

Field Name	Type	Descrption
JobType	Integer	Task type, respectively represents:

Field Name	Type	Descrption
		• 0: data split
Status	Integer	Task status, repectively represents • 0: preparing • 1: running • 2: paused • 3: finished
CollectionSpace	String	Collection space same
Collection	String	Colletion name

Data Split

Regarding data split, every file contains the following fields:

Field Name	String	Description
SourceName	String	Source replset name
TargetName	String	Target replset name
SourceID	Integer	Source replset ID
TargetID	Integer	Target replset ID
SplitValue	Object	Data split key

SYSUSRS Collection

Collection Space

SYSAUTH

Concept

SYSUSRS collection contains information of all registered users in the cluster. Information of a user is saved in a corresponding file.

Every file includes：

Name	Type	Description
User	String	User name
Password	String	MD5 hash value of password

Note:

If the collection is empty, it won't provide any connection with identification validation .

Coord Node

Concept

Coord node is a kind of logical node, which stores information except user data information.

Coord node is the coordinator of data requests, which merely send requests to relative data nodes rather than takes part in data execution.

Generally, the process of executing coord node is as follow:

- get a request
- analyze the request
- find cooresponding information of the request in local cache

- if not found, find it in catalogue nodes
- send the request to corresponding data node
- get result from data node
- collect results and send them to client

Coord node contacts with other nodes mainly through catalog service port(shardname parameter).

Data Node

Concept

Data node is a kind of logical node, which stores user data information.

Data node doesn't contain special catalogue information collection, so before the first visit to a collection, it is essential to get metadata information from catalog information collection.

Under independent mode, a data node ,as an independent service provider, is able to communicate with application and client without visiting any catalogue information collection.

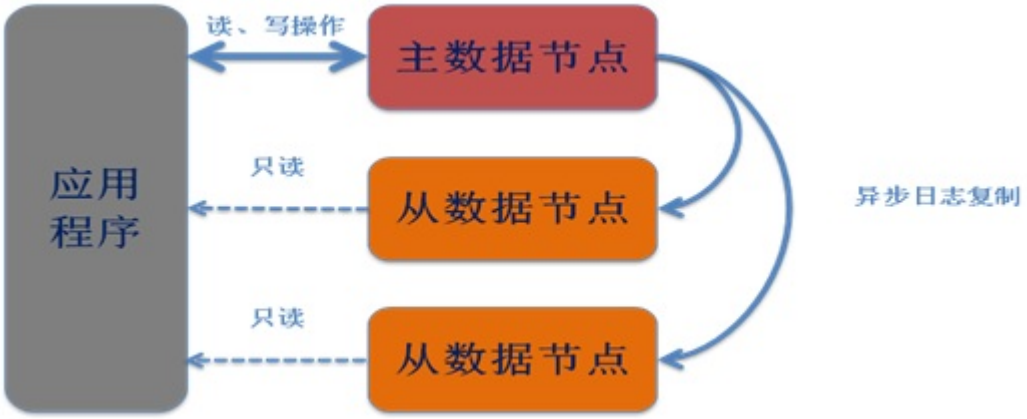
Repleset

Concept

Replset is also named as replication group. A replset may contain one or more data nodes (or catalog node).The data replicattion between nodes is asynchronous log file replication,which ensures data consistence.

All the nodes in a replset contacts with each other through replication service port (replname parament), and regularly sent heartbeat package to each other to validate their states.

The structure of replset is as follow:



Each node in a replset has two statuses:

Master node	Master node is also called main node. Master node can be written and read. All the written fata is recorded in log file aynchronously. Log information in log files is written into nodes asynchronously.
Slave node	Slave node is also called slave node. Slave node can only be read. All the data written into master node is asynchronously written into slave node. So the data in main node and inferior may be inconsistent, but replication can guarantee data consistence.

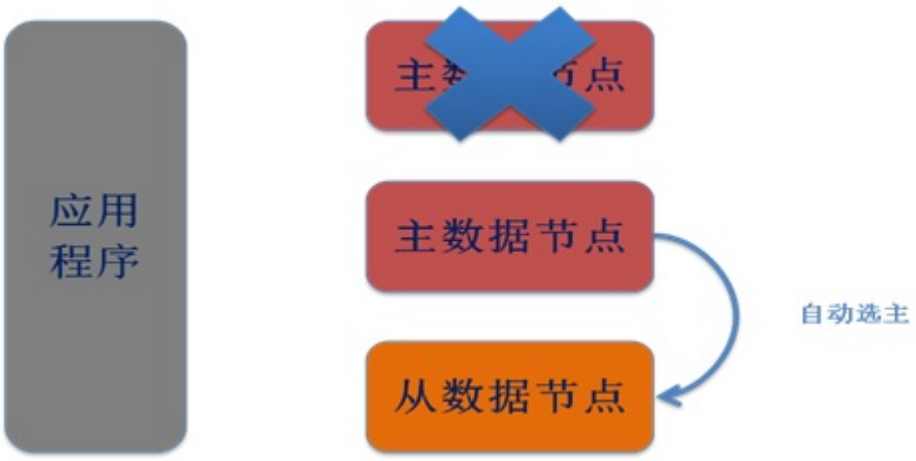
Replset achieves high usability through data replication, read/write splitting and vote.

Vote

Concept

Vote guarantees that there is a master node in a replset at any time. When the master node in a replset goes down, other nodes will automatically vote for another master node among them. In this way, reading and writing can be executed as usual when the master node goes down.

The core of vote is to supervise the statuses of nodes.Each node in a replset regularly sends its status to other nodes. So when the master node goes down, all the slave nodes will vote, and the node that currently matches the former master node will be elected to be the new main node.



The precondition of a successful vote is that more than half of nodes take part in the vote, or the vote will be canceled to avoid the problem of double-activation (two main nodes exist at the same time).

If there are less than half of nodes in the replset, the current master node will automatically becomes an slave node. Meanwhile, all the user connections to the current node will be disconnected.

When a new node joins a replset, or a broken-down node joins a replset again,full sync will be fulfilled.

Replicate

Concept

Data replication is synchronously executed between two nodes in a replset.

Any insertion, deletion or alteration in data nodes or catalog nodes will be recored in a log file. SequoiaDB puts the log file in a log cache, and then asynchronously stores it in disk memory.

Every data replication is executed between nodes:

Source Node	It is a kind of node containing new data. Master is not always the source node in the process of replication.Slave node can also be a source node.
Target Node	It is a node to copy data for a request.

In the process of replication between 2 nodes, the target node chooses the closest node to it, then sends a replicaition request to it. After the source node gets the request,it will package all the log records after the synchronous timeline provided by the source node and send them to the target node.

The target node copes with all the manipulation in log files after revieving data package.

There are 2 statuses in the replication between nodes：

- PEER：When the log files requested by a target node is still stored in the log cache of source node, the status of the two nodes is PEER.
- Remote Catchup: When the log files requested by the target node is not stored in the log cache of source node and is still stored in the log file of source node, the status of the 2 nodes is "Remote Catchup".

If the log files requested by the target node is not stored in the log file of source node, the target node will enter the status of full sync

Regarding the status of PEER, the system can directly read data from RAM in response to sync request. So when target node choose source node, it attempts to choose the closest node according to current log files. In this way, it can save time by reading data from RAM in most cases.

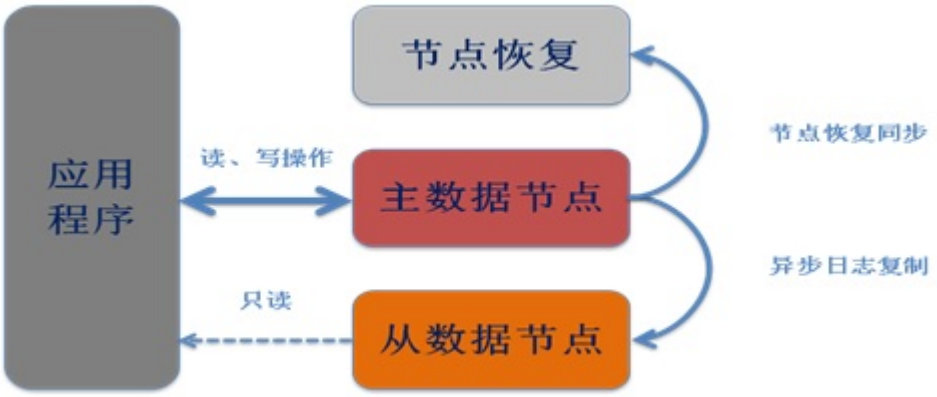
Full Sync

Concept

When a new node joins a replset or a broken-down node returns to a replset, it is essential to implement full sync to guarantee the consistence of data between the new node and other nodes.

In the process of full sync, two nodes take part in it:

Source node	Source node contains valid data. Master node is not always source node in full sync. Any slave node synchoronous with the master node can be a source node in full sync as well.
Target node	Target node is a new node that joins a replset or a broken-down node that returns to a replset. In full sync, original data on target node is discarded.



In the process of full sync, a target node will regularly request data from a source node. The source node packs data into big data block and sends it to the target node. When the target node receives all the data in the block, it will requset new data block from the source node.

In order to guarantee accessible writing on source node, if any data page sent to the target node is alerted again, it will be updated to the target node. In this way, new data is not discarded in this process.

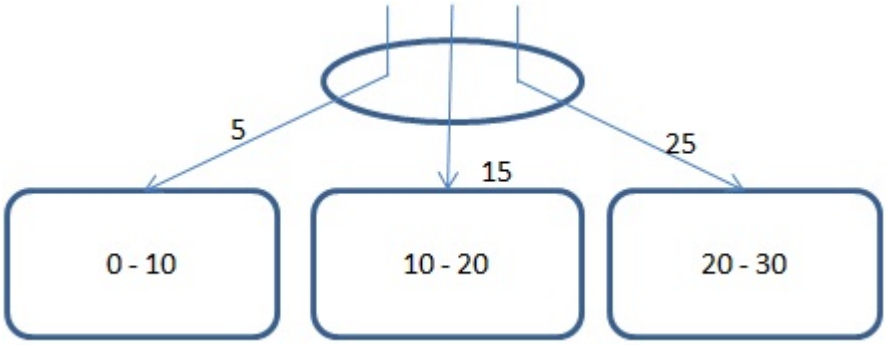
Sharding

In SequoiaDB cluster environment, users always put data in different logical node and physical node in order to acheive pallarel computing.

Since data in all the nodes in a sharding are the same absolutely, every sharding group is named as "Shard".

Every shard shares nothing.

The way that SequoiaDB uses data segment determines the sharding to insert data. The field that determine the sharding is named "sharding key". Sharding is based on the definition of collection. Every collection key contains one or more fields.



In this chart, 3 square areas represent 3 data nodes. The ellipse represents coord node. Every data node defines the range of data in it . For example, node 1 stores data between 0 and 10 including 0.

When users insert a record of data, coord node firstly finds out the sharding which the sharding key of the data belongs to. If there is no sharding key, it's defined in the type of Undefined. (Data in Undefined type can also be compared to that in common data types.)

After the sharding is found, coord node will send the request that it receives to it.

Sharding Key

Concept

Sharding key difines the sharding rules of data in every collection. Every colleciton corresponds a sharding key. A sharding key contains one or more fields.

In catalog node, every collection has its own sharding range. In sharding range, every range segment corresponds to a sharding group, which shows that certain data segment is in the sharding group.

Note:

When creating a collection, index key of it is specified. After collection is created, index key is unalterable.

In sharded collection, after a record is inserted into database, the sharding key is not allowed to be updated.

Format

The format of sharding key is similar to index key, which is a JSON object. Every field in JSON object corresponds to a field in sharding key. The value is "1" or "-1", respectively representing forward order or reverse order.

```
{ <field 1> : <1|-1> [, <field 2> : <1|-1> ...] }
```

Sample

A sharding key containing 2 fields, forward sequence and reverse sequence is as follow:

```
{ Field1 : 1, Field2 : -1 }
```

Sharded collection

Concept

Sharded collection is a collection that has defined a sharding key. Sharded collection can split data in collection to more than one data sharding group according to the fields specified by sharding key.

When collection is generated, users can specify sharding key. Sharded collection is generated in a random data sharding group. User can split a collection to several data sharding groups by themselves.

Sharding Interval

Every interval in sharded collection is named sharding interval.

When a sharded collection is generated, the sharding group of it contains all the ranges, which include range from MinKey to MaxKey for all fields.

Every range is left-closed and right-open. That's to say, data in it is greater than or equal to the lower boundary and lesser than the upper boundary. For example:

```
{ LowBound: { "": 10 }, UpBound: { "": 20 } }
```

In this example, the lower boundary is 10 and the high boundary is 20. So all the data in this range for all the sharding field is greater than or equal to 10 and lesser than 20.

Note: The definition of all ranges within a collection does not contain field name. The fields in it are consistent with those defined in sharding key in terms of type and number.

When a sharding key contains several fields, the matching principle is to match first field ahead. If it is located in boundary, it will match the next field. For example,

```
{ LowBound: { "": 10, "": 5 }, UpBound: { "": 20, "": 1 } }
```

In this shardin range, if the sharding key users input is located between 10 and 20, it is judged to belong to this range.If it is lesser than 10 or greater than 20, it doesnt belong to this range. If it is 10 or 20, it will compare the second field with the principle of left-closed-right-open.

Rule

Please refer to the definition of SYSCOLLECTIONS for more about the the definition principle of sharded collection.

Sample

A typical sharded range which exists in two sharding group is as follow:

```
[
      { "GroupID" : 1000,
        "LowBound" : { "" : MinKey, "" : MaxKey },
        "UpBound" : { "" : 10, "" : 5 }
      },
      { "GroupID" : 1001,
        "LowBound" : { "" : 10, "" : 5 },
        "UpBound" : { "" : MaxKey, "" : MinKey }
      }
]
```

The replset ID of the first range in this sample is 1000. The sample contains a sharding key with two fields:

- lower boundary : { "" : MinKey, "" : MaxKey }
- upper boundary : { "" : 10, "" : 5 }

The replset ID of the second range in this sample is 1001. The sample contains a sharding key with two fields:

- lower boundary : { "" : 10, "" : 5 }
- upper boundary : { "" : MaxKey, "" : MinKey }

Sharding Index

Concept

There is a default index named "$shard" in every sharded collection. This is sharding index.

Only sharded collections have sharding index.

Sharding index exists in every sharding group in a sharded collection. The order of field definations and the order of fields are the same as that of sharding key.

Note:

Any unique index defined by user should contain all the fields in sharding index. But the order of fields of them may not be the same.

In sharded collection, the field of "_id" is unique in the sharding, but not globally unique.

Sample

A typical sharding index is as follow:

```
{
        "IndexDef" : {
           "name" : "$shard",
           "_id" : { "$oid" : "515954bfa88873112fa6bd3a" },
           "key" : { "Field1" : 1, "Field2" : -1 },
           "v" : 0,
           "unique" : false,
           "dropDups" : false,
           "enforced" : false
        },
        "IndexFlag" : "Normal"
}
```

Background Task

Concept

Background task is a kind of task that never block front session. It won't stop when a front session pauses.

All the background tasks is recorded in the collection of SYSCAT.SYSTASKS in catalog node. Different types of background tasks may contain different fields.

All the background tasks contain fields as follow:

Field Name	Type	Describtion
JobType	Integer	Task type, representd: • 0: data split
Status	Integer	Task status, represents: • 0 : prepare • 1 : run • 2 : pause • 3 : finish
CollectionSpace	String	colelction space name
Collection	String	collection name

Background task type includes:

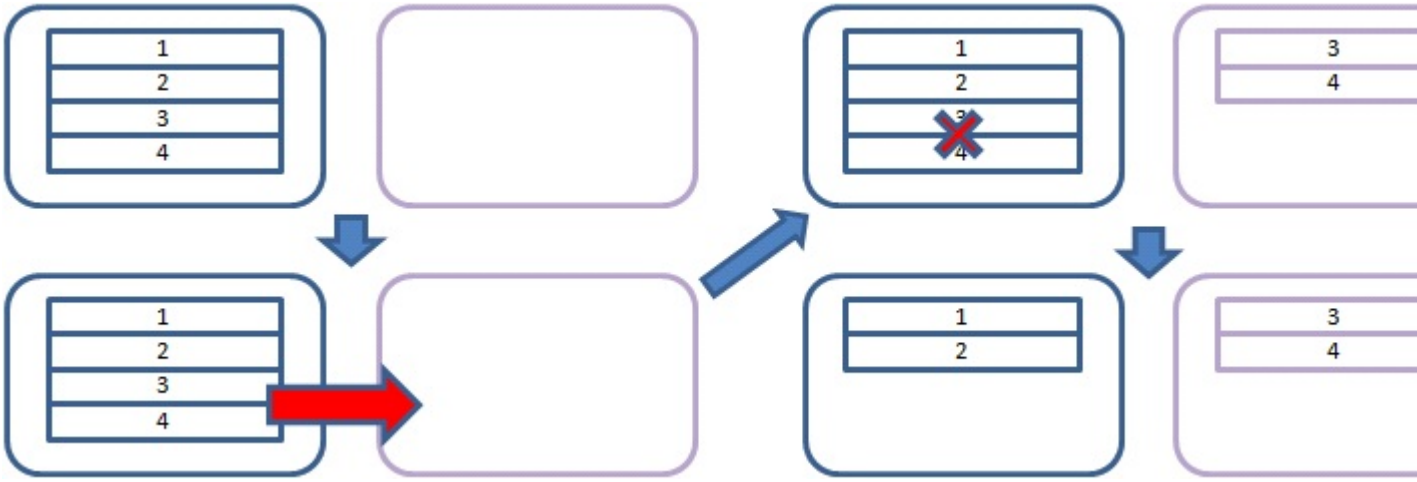
- data split

Split

Concept

A sharded collection is created in a random replset. If users want to horizontally expand the collection and allocate it to more than one replset, data should be splited.

Split is an approach to transfer online data from one replset to another replset. In the process of data transference, results of queries may not be consistent, but SequoiaDB can guarantee the consistence of data in disks.



In this chart, the upper left part is the initial status of a system. 4 records are stored in the left node. The split is defined to begin with 3, so data 3 and 4 will be splited from the left node, and transferred to the right node (the lower left part of the chart).

The upper right part is the 3rd status, two replsets store the same part of data at the same time. At this moment, data is temporarily inconsistent. But the final status is transformed into the 4th one in the lower right part. Data which is successfully transferred is deleted. Data is consistent finally.

Two replsets communicate with each other in the process of data split:

- source replset: data is originally stored in this replset
- target replset: data is transferred to this replset

Note: It is not allowed to split data among several replsets within a colleciton at the same time. If it is essential to split a collection into several replsets, it should be executed one by one.

Background Task

Data splie is executed by a background task。

This kind of background task contains several special fields:

Field Name	Type	Description
SourceName	String	Source replset name
TargetName	String	Target replset name
SourceID	Integer	Source replset ID
TargetID	Integer	Target replset ID
SplitValue	Object	Data split key

Split is executed through several stages:

Prepare Phase	In preparing phase, no task record is inserted into SYSCAT.SYSTASKS of catalog node. The system checks the validity of request in catalog node, and requests the record which meets the specific split condition from source node.
Ready Phase	In ready phase, coord node gets sharding key corresponding to split condition from source node, and sends it to catalog node. Catalog node inserts background execution records into the collection called SYSCAT.SYSTASKS.
Running Phase	In running phase, catalog node sends split request to target node. Target node generates a background task, requests data from source node, and sends its status to catalog node. When target node successfully generate a background task, catalog node can immediately continue to work. So catalog node won't be blocked by target node.
Clean-up Phase	In clean-up phase, target node has got all data it needs from source node, so it sends clean-up request to catalog node, then source node clean up data that has transferred.
Finish Phase	After source node clean up data that has transferred, it informs catalog node. Catalog deletes the task in the collection SYSCAT.SYSTASKS.

Install Overview

The basic steps to install Sequoiadb are as follow:

Plan Database Deploy

System Requirement of SequoiaDB

Install Media

SequoiaDB server installation

System configuration and start

Overview on Database Deployment

SequoiaDB is a completely distributed system architecture. It supports all kinds flexible deployment. In order to improve hardware and software performance, before installing system, we need to plan the deployment and the network connection. SequoiaDB supports 2 approaches of deployment:

- Standalone mode

  Start one business process that manipulate data. This mode supports high performance. It is easier to deploy. But the disadvantage is that it doesn't support distributed deployment or high-accessibility. It fits situation that has low amount of data and low total IOPS throughout, and requires low latency in single manipulation.

- Cluster Mode

  In this mode, the system can be deployed on multiple physical machines. It supports at most 300 physical machines.In cluster mode, catalog node, data node, coord node and Web management node (optional) should be deployed. As many as 65535 logical nodes can be deployed on one physical machine.

  Note: Dependent mode can be shifted to cluster mode. In this process, the system will pause business processes which cost least than10 minutes.

Users can plan their deloyment according to data capacity, performance, reliability, cost. Here are some typical deploment approaches for reference.

Actually, the approach of deployment can be very flexible. Users can combine different kind of deployment to meet their specific needs.

Easiest Deployment

High Availibility Deployment

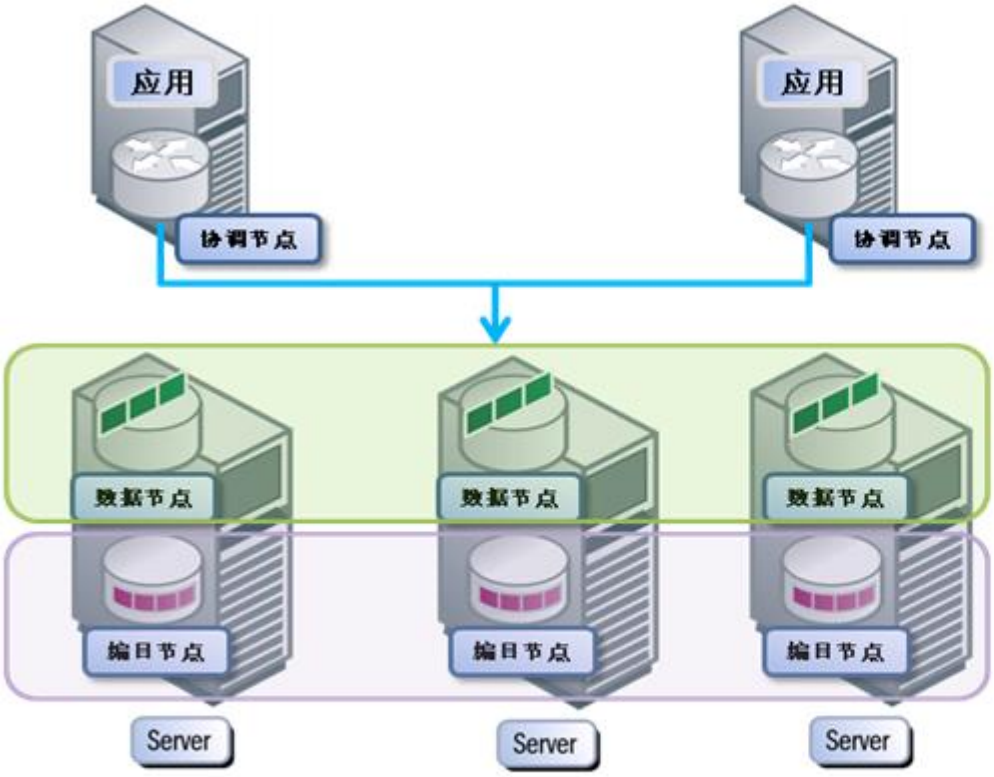
High Performance Deployment

Easiest deployment

Easiest deploy is proper to undemanding appliccations that has low requirement to database, small amount of data and low throughput, requiring low reliability. In this mode, SequoiaDB just start one database server process. Application can be put on the same server as database, or deployed in another server.

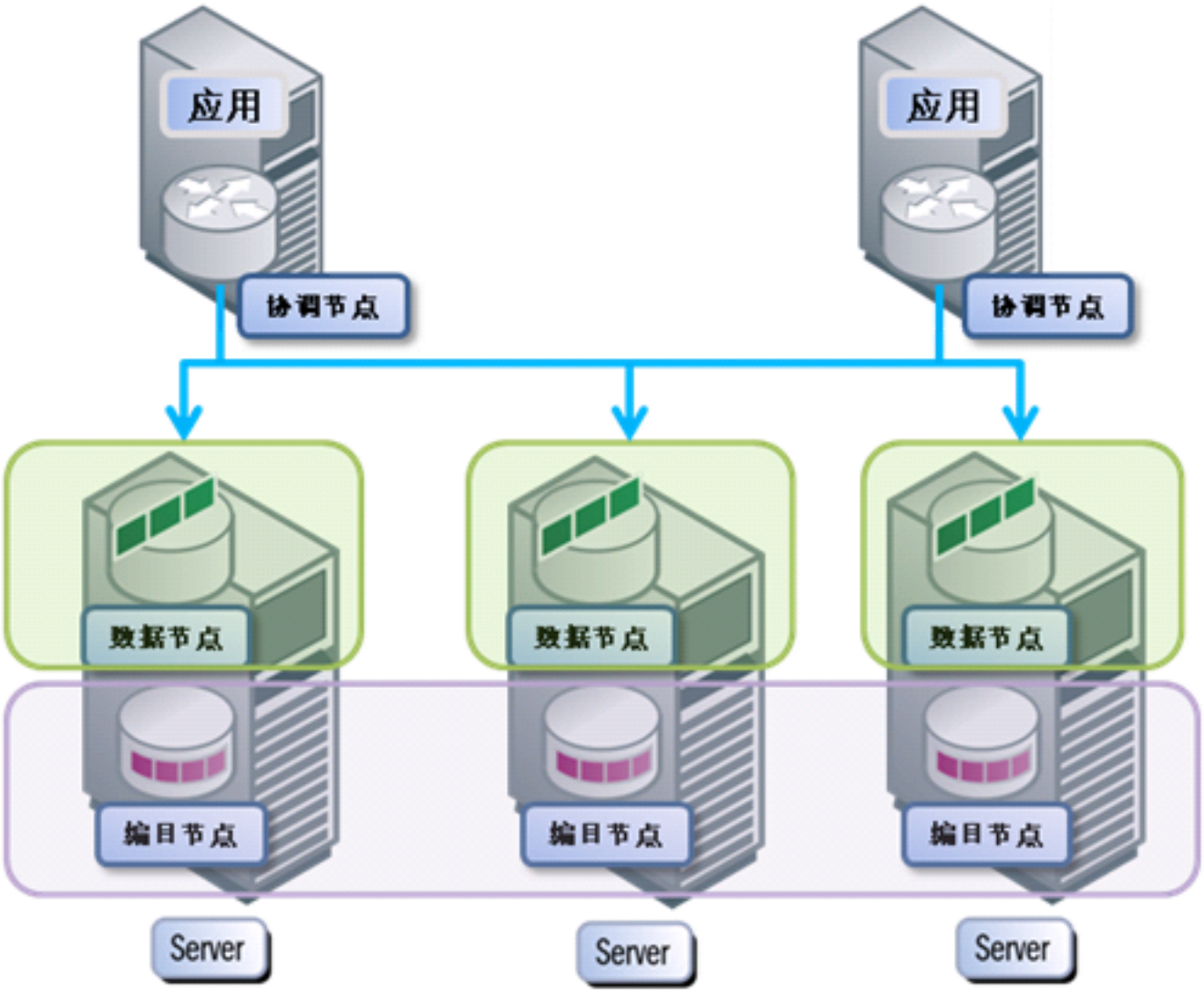
High-availability Deployment

High-availability deployment fits applications that require high reliability, but low amount of data and low total throughput. In this mode, there are catalog nodes and data nodes in three physical machines.A replset is consisted of 3 data nodes. Every copy cluster is consisted of 3 catalog nodes. Coord nodes and applications should be deployed on different servers or the same database server. The advantage of this mode is high reliability. If any server breaks down, all data will be able to be read or written.But the capacity of data is the same with that of a server, and hardware cost is higher.



High-performance Deployment

High-performance deployment fits applications that require high total throughput and high performance, low hardware cost but low reliability. In this mode, there are coord nodes and data nodes in three physical machines.A replset is consisted of 3 coord nodes. Every data node makes up one replset (It contains just one copy). Catalog nodes and applications should be deployed on different servers or the same database server. This mode can make full use of all memory in servers. The total memory capacity is the sum of memeory capacities of 3 servers. But the reliability is low. If any server breaks down, some data will fail to be read or written.



System Requirement of SequoiaDB Installation

Before installing SequoiaDB, please ensure that the system you choose can meet the requirement of OS, hardwarem, communication, disk and memory. The command "sdbpreck" is used to check the precondition of system to guarantee security.

Hardware Requirement

Supported OS

Software Requirement

Hardware Requirement

Requirement	Requirement	Suggestion
CPU	Support the following processor • x86(Intel Pentium、Intel Xeon 和 AMD) 32位 Intel	Suggest to use X64(64-bit AMD64 和 Intel EM64T processor)

Requirement	Requirement	Suggestion
	和 AMD processor • X64(64-bit AMD64 和 Intel EM64T processor)	
Disk	At least 10GB	More than 100GB
Memory	More than 128MB	More than 2GB
Network card	At least 1 network card	At least 1 network card

Supported Operating System

Type of system	List of system
Linux	• Red Hat Enterprise Linux (RHEL) 5; • Red Hat Enterprise Linux (RHEL) 6; • SUSE Linux Enterprise Server (SLES) 10 Service Pack 1 • SUSE Linux Enterprise Server (SLES)10 Service Pack 2 • SUSE Linux Enterprise Server (SLES) 11 Service Pack 1 • Ubuntu 8.04 • Ubuntu 7.04
Windows	• Windows Server 2003 DataCenter Edition(32-bit or 64-bit) • Windows Server 2003 Enterprise Edition(32-bit or 64-bit) • Windows Server 2003 Standard Edition(32-bit or 64-bit) • Windows Server 2008 DataCenter Edition(32-bit or 64-bit) • Windows Server 2008 Enterprise Edition(32-bit or 64-bit) • Windows Server 2008 Standard Edition(32-bit or 64-bit)
Power PC	• Windows Server 2003 DataCenter Edition(32bir or 64bit) • Windows Server 2003 Enterprise Edition(32bir or 64bit)

Software Requirement

Linux System requirement

- System configuration requirement

Configuration Item	Configuration method	Verification method
Set hostname	• Log in the system in the role of root, open "/etc/hosts" with the command "vi /etc/hosts" • Change hostname as follow: Change "127.0.0.1 localhost " into "127.0.0.1 localhost sdbserver1". (Here, "sdbserver1" is the hostname) • Save and exit	Check with "ping sdbserver1".
Enable server nodes to visit each other through hostname	• Log in the system in the role of root, open "/etc/hosts" with the command "vi /etc/hosts" • Alter "/etc/hosts", set the mapping between hostname of server node and IP in this file. 192.168.20.200 sdbserver1 192.168.20.201 sdbserver2 192.168.20.202 sdbserver3	Check with "ping sdbserver2"

Configuration Item	Configuration method	Verification method
	• 3 .Save and exit	

Note: Each database server should be configured

Recommended configuration in Linux

- Adjusting ulimit

  modify the configuration file "/etc/security/limits.conf ":

```
#<domain>      <type>    <item>     <value>
*               soft      core       0
*               soft      data       unlimited
*               soft      fsize      unlimited
*               soft      rss        unlimited
*               soft      as         unlimited
```

  Params Description：

  core：the "core" file is generated as the time of database failure to troubleshooting.,you had better close the system for advice.

  data：The size of data memory that database processes allow to allocate.

  fsize：The size of file that database processes allow to address.

  rss：The max size of resident set that database processes allowed.

  as：The max amount of virtual memory addressing space that database processes allowed.

  Note:

  1.Each database server should be configured

  2. It need to relogin after changing to be effective.。

- Adjusting kernel params

  1. Using the following command to output the current configuration of vm,and be placed on file

```
cat /proc/sys/vm/swappiness
cat /proc/sys/vm/dirty_ratio
cat /proc/sys/vm/dirty_background_ratio
cat /proc/sys/vm/dirty_expire_centisecs
cat /proc/sys/vm/vfs_cache_pressure
cat /proc/sys/vm/min_free_kbytes
```

  2.Adding the following params to the file of "/etc/sysctl.conf" to adjust kernel params

```
vm.swappiness = 0
vm.dirty_ratio = 100
vm.dirty_background_ratio = 10
vm.dirty_expire_centisecs = 50000
vm.vfs_cache_pressure = 200
vm.min_free_kbytes =  <5% of physical memory size,unit is KB>
```

  Note: When the available physical memory size of database isn't enough 8GB,doesn't need to set "vm.swappiness = 0".

  3. Executing the following command to be effective

```
/sbin/sysctl -p
```

  Note: Each database server should be configured

- Database directory structure

  The users had better put the directory of data,index,and log in different physical disks,to reduce the competition between sequential I/O and random I/O.

Install media

The table below is a look-up table of SequoiaDB product package and OS. Please choose proper package according to the type of your server OS.

	Linux 64-bit	Windows 32-bit	Windows 64-bit
Product package			
1.0.0 4/16/2013	sequoiadb_1.0.0_linux_x86_64_installer.run download	sequoiadb_1.0.0_windows_i586_installer.run download	sequoiadb_1.0.0_windows_x86_64_installer.run download
Driver(C/C++/Java/C#/PHP)			
1.0.0 4/16/2013	sequoiadb_driver_1.0.0_linux_x86_64.tar.gz download	sequoiadb_driver_1.0.0_window_i586.tar.gz download	sequoiadb_driver_1.0.0_windows_x86_64.zip download
Kit			
apache and php	apache2-linux-x64-installer.tar.gz download	apache2-windows-i586-installer.tar.gz download	apache2-windows-x86_64-installer.zip download

Installation of SequoiaDB Server

Availible system of installation :

Installation	Linux	Windows
Wizard Installation	Yes	Yes

Wizard Installation

Wizard installation is a GUI installation program available for Linux and Windows. It is easy to install through the GUI program. It is used to show settings and install SequoiaDB server.

In Linux, X Server is essential to show GUI. It there is no X Server in Linux, users can choose to install through text interface.

Installation of SequoiaDB in Linux

Installing Linux version on SequoiaDB Server

Preparation before installation

- Ensure that the system support the requirement of installation, content and disk;
- Log in on the role of root to install SequoiaDB database service;
- Check the mapping between SequoiaDB package and OS;
- If you choose to install through GUI mode, please make sure that X server service is running.
- Check whether the server has set hostname, and it can generate network connection with other servers (For example, ssh hostname);

Note: SequoiaDB installation wizard doesn't support non-English characters.

Installation steps

Explaination

(1)Here we take sequoiadb-1.0.0-linux-x86_64-installer.run as the sample installation package

(2)The steps are introduced through command lines. In GUI installation, users can install server program accoding to image wizard prompts.

If there are more than one server, users should repeatedly install server program on them one by one according to the following steps ;

- Refer System configuration requirement to configure Host Name and change the system kernel params
- Run installation program

```
./sequoiadb-1.0.0-linux-x86_64-installer.run --mode text
```

- The system prompt you to choose wizard language.

```
Language Selection
Please select the installation language
[1] English - English
[2] Simplified Chinese - 简体中文
Please choose an option [1] :2
```

- Input 1 to choose English, then the system prompt you to input installation catalog

```
-------------------------------------------------------------------------------
Created with an evaluation version of BitRock InstallBuilder

Welcome to the SequoiaDB Server Setup Wizard.

-------------------------------------------------------------------------------
Please specify the directory where SequoiaDB Server will be installed.

Installation Directory [/opt/sequoiadb]:
```

- After inputting the installation path(installed in */opt/sequoiadb* in default) .Then you are prompted to enter a user name which used to run *sequoiadb service*.

```
Confiture user information
Please insert the desired username and password
User Name [sdbadmin]:
```

- Pressing *enter* after inputting the user name(creating *sdbadmin* user in default).Then system prompts you to enter password and confirmed password.

```
Password [********] :
Re-enter [********] :
```

- After inputing password twice (The default password is sdbadmin), the system will prompt you to set service port.

```
-------------------------------------------------------------------------------
SDBCM Port

SequoiaDB Cluster Manager port

Port [50010]:

-------------------------------------------------------------------------------
```

Note: Service ports of all servers should be the same.

- Input port (The default port is 50010). Then affirm to install the program:

```
-------------------------------------------------------------------------------
Setup is now ready to begin installing SequoiaDB Server on your computer.

Do you want to continue? [Y/n]:
```

- Input "Y" and press return key. Then the system begin to install. After about 1 minute, installation is finiched;

```
Installing SequoiaDB Server to your computer,please wait for a moment.Installing
0% ______________ 50% ______________100%
########################################
-------------------------------------------------------------------------------
SequoiaDB Server Program has been installed to your computer.
```

Start SequoiaDB web service (optional)

By starting the background Web service of SequoiaDB,can also do operetions to database,and convenient for management.

Windows environment

Steps:

- run the command "cmmd.exe"
- enter to the installed directory of SequoiaDB, for example

```
cd C:\sequoiadb\tools\server\php\bin
```

- Execute the file "php.exe"

```
php.exe #S <server name:port> -t <doc directory>
```

server name:specify the service IP address,such as "192.168.10.10"

port:specify the service port,such as "8080"

doc director:specify the root directory that the server pointed,such as "/opt/sequoiadb/www/"

Finally enter "http://192.168.10.10:8080" in the browser,then you can access to the database management page.

Finally enter "http://192.168.10.10:8080" in the browser,then you can access to the database management page.

Linux environment

Steps：

- enter the directory:/opt/sequoiadb/tools/server/php/bin
- execute the following command:

```
/opt/sequoiadb/tools/server/php/bin/php #S <server name:port> -t <doc director>
```

server name:specify the service IP address,such as "192.168.10.10"

port:specify the service port,such as "8080"

doc director:specify the root directory that the server pointed,such as "/opt/sequoiadb/www/"

Finally enter "http://192.168.10.10:8080" in the browser,then you can access to the database management page.

System configuration and startup

Catalog:

Configuration and startup of standalone mode

Explanation:

(1) This section will take the easiest deployment as an example to introduce configuration and startup steps.

(2)The following manipulation supposes that SequoiaDB is installed under the catalog "/opt/sequoiadb".

(3)All of "sdb" service processes run as "sdbadmin", please ensure all of the database files have the "sdbadmin" read and write permissions

- switch to "sdbadmin"

```
su sdbadmin
```

- Enter installation catalog (The following content supposes that the default installation path is "/opt/sequoiadb")

```
cd/opt/sequoiadb
```

- Create a catalog to store configuration file

```
mkdir #p /opt/sequoiadb/conf/local/50000
```

(Here 50000 is the port of database service)

- Copy the sample configuraiton of standalone mode

```
cp /opt/sequoiadb/conf/samples/sdb.conf.standalone /opt/sequoiadb/conf/local/50000/sdb.conf
```

- Open the configuration file

```
vi  /opt/sequoiadb/conf/local/50000/sdb.conf
```

- Alter nessesary configuration parameters

```
# database path
```

```
dbpath=/opt/sequoiadb/database/standalone
```

The path of database. It is configurated according to requirement, and ensure that the path exists (Please do not manually create it!)

- Type in ":wq" to save and exit.
- Create the path to store data file

```
mkdir #p /opt/sequoiadb/database/standalone
```

This path is the same as that in previous step

- Start database process

```
/opt/sequoiadb/bin/sdbstart #c /opt/sequoiadb/conf/local/50000
```

- Finish configuration and startup of database

Configuration and startup of cluster mode

Instruction：

(1)This section will introduce the steps of configuration and startup by taking highly accessible deployment as an example.

(2)The follpwing steps is based on the fact that SequoiaDB is installed under the catalog "/opt/sequoiadb".

(3)All of "sdb" service processes run as "sdbadmin", please ensure all of the database files have the "sdbadmin" read and write permission

- Step 1: Check the configuration service status of SequoiaDB.

  Check the configuration service status of SequoiaDB in each database server: service sdbcm status

  If the system prompts "sdbcm is running", we can know that service is running, or please run the following command to reconfigurate service program.

  service sdbcm start

- Step 2:start one coord node

  1.switch to "sdbadmin"

```
su sdbadmin
```

  2. On any database server (the following steps are manipulated on this server), create the coord node configuration catalog.

```
mkdir -p /opt/sequoiadb/conf/local/50000
```

Here "50000" is the service port of coord node. It can be set according to requirement.

3. Copy sample configuration file of coord node

```
cp /opt/sequoiadb/conf/samples/sdb.conf.coord  /opt/sequoiadb/conf/local/50000/sdb.conf
```

4. Alter configuration file

```
vi /opt/sequoiadb/conf/local/50000/sdb.conf
```

Alter content

```
# database path
dbpath=/opt/sequoiadb/database/coord
```

This is the path of database. It can be changed according ot requirement. Please ensure that the path exists (Please create manually if it does't exsit.).

5. Type in ":wq" to save and exit.

6. Create path that stores data file

```
 mkdir -p /opt/sequoiadb/database/coord
```

The path is the path in previous step

7. Start coord node process

```
 /opt/sequoiadb/bin/sdbstart -c /opt/sequoiadb/conf/local/50000/
```

- Step 3: Configurate and start catalog nodes with statements

1. Start SequoiaDB Shell console on the physical machine of coord node.

```
/opt/sequoiadb/bin/sdb
```

2. Connect to coord node

Input in shell:

```
> var db = new Sdb("localhost",50000)
```

Here "50000" is the service port of coord node.

3. Create a catalog node group

```
>db.createCataRG("sdbserver1", 30000, "/opt/sequoiadb/database/cata/30000")
```

Here "sdbserver1" is the hostname of the 1st server;

Here "30000" is the service port of catalog node.

" /opt/sequoiadb/database/cata/30000" is the path of data file of catalog node

4. Wait for 5 second, and add 2 more catalog nodes

```
>var catarg = db.getRG("SYSCatalogGroup");
>var node1 = catarg.createNode("sdbserver2", 30000,"/opt/sequoiadb/database/cata/30000")
>var node2 = catarg.createNode("sdbserver3", 30000,"/opt/sequoiadb/database/cata/30000")
```

5. Start catalog node group

```
>node1.start()
>node2.start()
```

Note:

When creating a node, the first parameter should be "hostname", rather than IP of the host.

- Step 4: Configurate and start data nodes with statements

1. Create data node group

```
>var datarg = db.createRG("datagroup1")
```

2. Add data node

```
>datarg.createNode("sdbserver1", 51000, "/opt/sequoiadb/database/data/51000")
>datarg.createNode("sdbserver2", 51000,"/opt/sequoiadb/database/data/51000")
>datarg.createNode("sdbserver3", 51000, "/opt/sequoiadb/database/data/51000")
```

Note:

When creating a node, the first parameter should be "hostname", rather than IP of the host.

3. Start data node group

```
>datarg.start()
```

4. Exit SequoiaDB shell console

```
 >quit
```

- Step 5: Start coord node on other 2 servers

1. Create configuration catalog of coord node

```
mkdir -p /opt/sequoiadb/conf/local/50000      Here "50000" is the service port of coord node. It can be set
according to requirement.
```

2. Copy sample configuration file of coord node

```
cp /opt/sequoiadb/conf/samples/sdb.conf.coord  /opt/sequoiadb/conf/local/50000/sdb.conf
```

3. Alter configuration file

```
vi /opt/sequoiadb/conf/local/50000/sdb.conf
Alter content
# database path
dbpath=/opt/sequoiadb/database/coord  This is the path of database. It can be changed according ot requirement.
 Please ensure that the path exists (Please create manually if it does't exsit.).

the following line
# catalog addr (hostname1:servicename1,hostname2:servicename2,...)
#catalogaddr=
modify to
# catalog addr (hostname1:servicename1,hostname2:servicename2,...)
catalogaddr=sdbserver1:30003,sdbserver2:30003,sdbserver3:30003  This id the address and port of Catalog service.
```

4. Type in ":wq" to save and exit

5. Create path of data file

```
mkdir -p /opt/sequoiadb/database/coord  This path is the one in previous step.
```

6. Start coord node process

```
/opt/sequoiadb/bin/sdbstart -c /opt/sequoiadb/conf/local/50000/
```

End

Uninstall

- Stop SequoiaDB configuration service program

  Do the following in every database server:

  1. Log in database server in the identity of SDB administrator
  2. Execute "service sdbcm stop"
- Stop SequoiaDB database service program

  Do the following in every database server:

  1. Log in database server in the identity of SDB administrator
  2. Execute "/opt/sequoiadb/bin/sdbstop"
- Uninstall SequoiaDB software

  Do the following in every database server:

1. Log in database server in the identity of SDB administrator
2. Execute "/opt/sequoiadb/uninstall"
3. Delete the database directory and files

- Fallback the system configuration params

1. delete the params in the file "/etc/security/limits.conf "

```
# <#domain>      <type>    <item>    <value>
# *               soft      core       0
# *               soft      data       unlimited
# *               soft      fsize      unlimited
# *               soft      rss        unlimited
       *               soft      as         unlimited
```

2.delete the params in the file "/etc/sysctl.conf"

```
vm.swappiness = 0
vm.dirty_ratio = 100
vm.dirty_background_ratio = 10
vm.dirty_expire_centisecs = 50000
vm.vfs_cache_pressure = 200
vm.min_free_kbytes = <5% of physical memory size,unit is KB>
```

DataBase Administration

database administration related content.

DataBase Management

This section provides information about basic management tasks.

Database Runtime Configuration

Parameter Instruction

Parameter Nmae	Acronym	Type	Description
--help	-h	—	Print help information
--confpath	-c	str	1.Specified configuration file path (doesn't contain fiel name). The system will find *sdb.conf* under confpath. 2. *sdb.conf* contains necessary configuration items. The format of a configuration item is "parameter name=value" like "svcname=50000" and "diaglevel=3". 3. If this parameter is not specified, system will automatically search for sdb.conf in current path. 4. There may not be sdb.conf.
--logpath	-l	str	1. Synchronous log file willbe generated when replication node does full sync. This parameter is used to specified the path of synchronous log file. 2. If is is not specified, the default path is "Data file path/replicalog".
--diagpath	—	str	1. It specifies the catalog of diagnostic log. 2. If is is not specified, the default path is "Data file path/diaglog".
--dbpath	-d	str	1. It specifies the path of data file. 2. If is is not specified, the default path is current path.
--bkuppath	—	str	1. It specifies the catalog of backup file. 2. If is is not specified, the default path is "Data file path/bakfile".
--maxpool	—	str	1. It specifies the amount of thread in thread pool.

Parameter Nmae	Acronym	Type	Description
			2. If is is not specified, the default value is 0.
--sycname	-p	str	1. It specifies local service port 2. If is is not specified, the default value is 50000.
--replname	-r	str	1. It specifies data synch port. 2. If is is not specified, the default value is "svcname+1".
--shardname	-a	str	1. It specifies shard port. 2. If is is not specified, the default value is "svcname+2".
--catalogname	-x	str	1. It specifies catalog port. 2. If is is not specified, the default value is "svcname+3".
--httpname	-s	str	1. It specifies http port. 2. If is is not specified, the default value is "svcname+4".
--diaglevel	-v	num	1. It specifies the print level of diagnostic log. Diagnostic log in sequoiaDB, 0-5 respectively represent: SEVERE, ERROR, EVENT,WARNING, INFO, DEBUG。 2. If is is not specified, the default value is "WARNING".
--role	-o	str	1. It specifies serving role. In sequoiaDB, "data/coord/catalog/ standalone" represents "data node/ coord node/catalog node/stand-alone machine". 2. If is is not specified, the default value is "independent".
--catalogaddr	-t	str	1. It specifies the address of catalog node. The format is "hostname1:catalogname1, hostname2:catalogname2,...". 2. It should specify address of at least one catalog.
--logfilesz	-f	num	1. It specifies the size of sync log file. It is between 32(MB) and 2048(MB). 2. If is is not specified, the default value is 32 (MB).
--logfilenum	-n	num	1. It specifies the amount of synchronous log file. 2. If is is not specified, the default value is 20.

Note:

SequoiaDB supports setting configuration with command line, configuration file, or both of them. When both of them are used, parameters in command line will overwrite those in configuration file.

The total size of synch log (logfilesz * logfilenum) determines the fault tolerance in the process of sync. If log is bigger, the possiblity of full sync is lower.

Monitoring

Concept

Monitorting is an approach to monitor the status of current system. In SequoiaDB, users can monitor system with SNAPSHOT command and LIST command.

Snapshot

Snapshot is a kind of command which can get the current status of system. There are different types of snapshots as follow:

Snapshot Sign	Snapshot Type	Proper Nodes	Description
SDB_SNAP_CONTEXTS	Context	Data node, catalog node	Context snapshot contains all the contexts corresponding to all the sessions in current data node.
SDB_SNAP_CONTEXTS_CURRENT	Context of current session	Data node, catalog node	Current contexts snapshot contains the session contexts corresponding to current connections in data nodes.
SDB_SNAP_SESSIONS	Session	Data node, catalog node	Session snapshot contains all the sessions between users and system on current database node.
SDB_SNAP_SESSIONS_CURRENT	Current session	Data node, catalog node	Current session snapshot contains the current sessions on current database node.
SDB_SNAP_COLLECTIONS	Collection	All nodes	Collections snapshot contains all the non-temporary collections in current data nodes or current cluster.
SDB_SNAP_COLLECTIONSPACES	Collection space	All nodes	Collection space snapshot contains all the collection space in the current data node or cluster. (expect from catalog collection space)
SDB_SNAP_DATABASE	Database	Data node, catalog node	Database snapshot contains database monitoring information of current database node.
SDB_SNAP_SYSTEM	System	Data node, catalog node	System snapshot contains system monitoring information of current database node.

List

List is a kind of lightweight command for getting system current status.There are different types of lists as follow:

List Sign	List Type	Proper Nodes	Description
SDB_LIST_CONTEXTS	Context	Data node, catalog node	Context list contains all the contexts corresponding to all the sessions in current data node.
SDB_LIST_CONTEXTS_CURRENT	Context of current session	Data node, catalog node	Current contexts list contains the session contexts corresponding to current connections in data nodes.
SDB_LIST_SESSIONS	Session	Data node, catalog node	Session list contains all the sessions between users and system on current database node.
SDB_LIST_SESSIONS_CURRENT	Current session	Data node, catalog node	Current session list contains the current sessions on current database node.
SDB_LIST_COLLECTIONS	Collection	All nodes	Collections list contains all the non-temporary collections in current data nodes or current cluster.
SDB_LIST_COLLECTIONSPACES	Collection space	All nodes	Collection space snapshot contains all the collection space in the current data node or current cluster. (expect from catalog collection space)
SDB_LIST_STORAGEUNITS	Storage unit	Data node, catalog node	Storage unit list contains information of all storage units on current database node.
SDB_LIST_GROUPS	Replset	Coord node	Replset list contains infromation of all shards in current cluster.

Snapshot

This section introduces information of all kinds of snapshots:

Contexts Snapshot

Current Contexts Snapshot

Session Snapshot

Current Session Snapshot

Collections Snapshot

Collection Spaces Snapshot

Database Snapshot

System Snapshots

Context Snapshot

Description

Context snapshot contains all the contexts corresponding to all the sessions in current data node.

A session is a record. If a session contains one or more contexts, the Contextd array field generate a for each of the contexts.

Note: The manipulation of snapshot will generate a context, so the result set will contains at least one context of current snapshot.

Sign

SDB_SNAP_CONTEXTS

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	String	Session ID(HostName:Port:ID)
Contexts.ContextID	Long integer	Context ID
Contexts.DataRead	Long integer	Data that is read.
Contexts.DataRead	Long integer	Data that is read.
Contexts.IndexRead	Long integer	Index that is read
Contexts.QueryTimeSpent	Float	Total time of query (second)
Contexts.StartTimestamp	Timeline	Start time.

Sample

```
> db.snapshot(SDB_SNAP_CONTEXTS)
{
  "SessionID": "vmsvr2-suse-x64:51000:28",
  "Contexts": [
    {
      "ContextID": 12,
      "DataRead": 0,
      "IndexRead": 0,
      "QueryTimeSpent": 0,
      "StartTimestamp": "2013-09-27-18.06.37.079570"
    }
  ]
}
```

Current Contexts Snapshot

Description

Current contexts snapshot contains the session contexts corresponding to current connections in data nodes.

It returns one record. In a record, Contests array field contains all the contexts in current session.

Note: The manipulation of snapshot will generate a context, so the result set of it at least contains one context.

Sign

SDB_SNAP_CONTEXTS_CURRENT

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	String	SessionID(HostName:Port:ID)
Contexts.ContextID	Long integer	Context ID

Field Name	Type	Description
Contexts.DataRead	Long integer	Data that is read
Contexts.IndexRead	Long integer	Index that is read
Contexts.QueryTimeSpent	Float	Total query time (second)
Contexts.StartTimestamp	Timeline	Creating time

Sample

```
> db.snapshot(SDB_SNAP_CONTEXTS_CURRENT)
{
  "SessionID": vmsvr2-suse-x64:51000:28,
  "Contexts": [
    {
      "ContextID": 13,
      "DataRead": 0,
      "IndexRead": 0,
      "QueryTimeSpent": 0,
      "StartTimestamp": "2013-09-27-18.25.17.311168"
    }
  ]
}
```

Session Snapshot

Description

Session snapshot contains all the sessions between users and system on current database node. Each session is recorded as one record.

Sign

SDB_SNAP_SESSIONS

Proper Node

data node,catalog node

Field Information

Field Information	Type	Description
SessionID	Integer or long integer	Session ID(Hostname:Port:ID)
TID	Integer	System thread ID corresponding to the session
Status	String	Session status: • Creating：creating status • Running：running status • Waiting：waiting status • Idle：idle status of thread pool • Destroying：destroying status
Type	String	EDU Type
Name	String	EDU name. Generally, it is null.
QueueSize	Integer	The length of queue of requests waiting to be processed
ProcessEventCount	Long integer	The amount of requests which have been processed.
Contexts	Long integer or array	Context ID array. It is a list containing all contexts in the session.

Field Information	Type	Description
TotalDataRead	Long integer	Total data read
TotalIndexRead	Long integer	Total index read
TotalDataWrite	Long integer	Tota data record write
TotalIndexWrite	Long integer	Total index write
TotalUpdate	Long integer	Total amount of updated records
TotalDelete	Long integer	Total amount of deleted records
TotalInsert	Long integer	Total amount of inserted records
TotalSelect	Long integer	Total amount of selected records
TotalRead	Long integer	Total data read
TotalReadTime	Long integer	Total data read time (millisecond)
TotalWriteTime	Long integer	Total data write time (millisecond)
ConnectTimestamp	Timeline	The time of beginning connection
UserCPU	Float	User CPU (second)
SysCPU	Float	System CPU (second)

示例

```
> db.snapshot(SDB_SNAP_SESSIONS)
{
  "SessionID": "vmsvr2-suse-x64:51000:1",
  "TID": 8680,
  "Status": "Running",
  "Type": "LogWriter",
  "Name": "",
  "QueueSize": 0,
  "ProcessEventCount": 1,
  "Contexts": [],
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
  "ConnectTimestamp": "2013-09-27-13.28.38.927465",
  "UserCPU": "0.410000",
  "SysCPU": "0.150000"
}
{
  "SessionID": "vmsvr2-suse-x64:51000:2",
  "TID": 8681,
  "Status": "Waiting",
  "Type": "DpsRollbackTask",
  "Name": "",
  "QueueSize": 0,
  "ProcessEventCount": 1,
  "Contexts": [],
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
  "ConnectTimestamp": "2013-09-27-13.28.38.927406",
  "UserCPU": "0.300000",
  "SysCPU": "0.130000"
```

```
}
{
  "SessionID": "vmsvr2-suse-x64:51000:3",
  "TID": 8682,
  "Status": "Waiting",
  "Type": "Cluster",
  "Name": "",
  "QueueSize": 0,
  "ProcessEventCount": 71977,
  "Contexts": [],
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
  "ConnectTimestamp": "2013-09-27-13.28.39.127611",
  "UserCPU": "2.050000",
  "SysCPU": "1.010000"
}...
{
  "SessionID": "vmsvr2-suse-x64:51000:28",
  "TID": 9430,
  "Status": "Running",
  "Type": "Agent",
  "Name": "127.0.0.1:60309",
  "QueueSize": 0,
  "ProcessEventCount": 9,
  "Contexts": [
    14
  ],
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
  "ConnectTimestamp": "2013-09-27-18.06.25.961090",
  "UserCPU": "0.030000",
  "SysCPU": "0.060000"
}
```


Current Session Snapshot

Description

Current session snapshot contains the current sessions on current database node. Each session is recorded as one record.

Sign

SDB_SNAP_SESSIONS_CURRENT

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	Integer or long integer	Session ID(Hostname:Port:ID)
TID	Integer	System thread ID corresponding to the session

Field Name	Type	Description
Status	String	Session status: • Creating：creating status • Running：running status • Waiting：waiting status • Idle：idle status of thread pool • Destroying：destroying status
Type	String	EDU Type
Name	String	EDU name. Generally, it is null.
QueueSize	Integer	The length of queue of requests waiting to be processed
ProcessEventCount	Long integer	The amount of requests which have been processed.
Contexts	Long integer or array	Context ID array. It is a list containing all contexts in the session.
TotalDataRead	Long integer	Total data read
TotalIndexRead	Long integer	Total index read
TotalDataWrite	Long integer	Tota data record write
TotalIndexWrite	Long integer	Total index write
TotalUpdate	Long integer	Total amount of updated records
TotalDelete	Long integer	Total amount of deleted records
TotalInsert	Long integer	Total amount of inserted records
TotalSelect	Long integer	Total amount of selected records
TotalRead	Long integer	Total data read
TotalReadTime	Long integer	Total data read time (millisecond)
TotalWriteTime	Long integer	Total data write time (millisecond)
ConnectTimestamp	Timeline	The time of beginning connection
UserCPU	Float	User CPU (second)
SysCPU	Float	System CPU (second)

Sample

```
> db.snapshot(SDB_SNAP_SESSIONS_CURRENT)
{
  "SessionID": "vmsvr2-suse-x64:51000:28",
  "TID": 9430,
  "Status": "Running",
  "Type": "Agent",
  "Name": "127.0.0.1:60309",
  "QueueSize": 0,
  "ProcessEventCount": 12,
  "Contexts": [
    15
  ],
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
```

```
  "ConnectTimestamp": "2013-09-27-18.06.25.961090",
  "UserCPU": "0.910000",
  "SysCPU": "2.060000"
}
```

Collection Snapshot

Description

Collections snapshot contains all the non-temporary collections in current data nodes (It contains user collection in coord node). Each collection is recorded as a record.

Sign

SDB_SNAP_COLLECTIONS

Proper Node

All nodes

Field Information

Since information stored in data nodes and catalog node is not the same, collections list will return different structures according to the kind of the node:

Coord Node Field Information

Field Name	Type	Description
Name	String	Full name of collection
Details.GroupName	String	Group name where the node in
Details.Group.ID	Integer	Collection ID，Range 0~4096，Unique in the collectionspace
Details.Group.LogicalID	Integer	Collection logical ID
Details.Group.Sequence	Integer	Sequence number
Details.Group.Indexes	Integer	The number of indexes in the collection
Details.Group.Status	String	The current status of the collection • Free • Normal • Dropped • Offline Reorg Shadow Copy Phase • Offline Reorg Truncate Phase • Offline Reorg Copy Back Phase • Offline Reorg Rebuild Phase
Details.Group.NodeName	String	Node name(hostname+port)

Other node field information

Field Name	Type	Description
Name	String	Full name of collection
Details.ID	Integer	Collection ID. Range from 0 to 4095. Unique in collection space.
Details.LogicalID	Integer	Collection logical ID
Details.Sequence	Integer	Sequence number

Field Name	Type	Description
Details.Indexes	Integer	The amount of indexes in the collection
Details.Status	String	The current status of the collection • Free • Normal • Dropped • Offline Reorg Shadow Copy Phase • Offline Reorg Truncate Phase • Offline Reorg Copy Back Phase • Offline Reorg Rebuild Phase
NodeName	String	NodeName(HostName+Port)
GroupName	String	The group name where the node in(Only display in the cluster mode)

Sample of coord node:

```
> coord.snapshot(SDB_SNAP_COLLECTIONS)
{
  "Name": "susefoo.susebar",
  "Details": [
    {
      "GroupName": "datagroup1",
      "Group": [
        {
          "ID": 0,
          "LogicalID": 0,
          "Sequence": 1,
          "Indexes": 1,
          "Status": "Normal",
          "NodeName": "vmsvr2-suse-x64:51000"
        }
      ]
    }
  ]
}
```

Sample of other nodes:

```
> db.snapshot(SDB_SNAP_COLLECTIONS)
{
  "Name": "foo.bar",
  "Details": [
    {
      "ID": 0,
      "LogicalID": 0,
      "Sequence": 1,
      "Indexes": 8,
      "Status": "Normal",
      "NodeName": "vmsvr2-suse-x64:51000",
      "GroupName": "datagroup1"
    }
  ]
}
```

Collection Space Snapshot

Description

Collection space snapshot contains all the collection space in the current data node. Each collection space is recorded as one record.

Sign

SDB_SNAP_COLLECTIONSPACES

Proper Node

All nodes

Field Information

Since information stored in data nodes and catalog node is not the same, collection space snap will return different structures according to the kind of the node:

Coord Node Field Information

Field Name	Type	Description
Name	String	Collection space name
Collection	String array	All the collection in the collection space
PageSize	Integer	Size of data page in collection space
Group.GroupID	Integer array	ID list of the replset of the collection space.
Group.GroupName	String	Group name list of the replset of the collection space.

Other node field information

Field Name	Type	Description
Name	String	Collection space name
Collection	String array	All the collections in collection space
PageSize	Integer	Size of data page in collection space
GroupName	String	group name where collectionspcae in

Coord node sample

```
> coord.snapshot(SDB_SNAP_COLLECTIONSPACES)
{
  "Collection": [
    {
      "Name": "bar"
    }
  ],
  "Group": [
    {
      "GroupID": 1000,
      "GroupName": "datagroup1"
    }
  ],
  "Name": "foo",
  "PageSize": 4096,
  "_id": {
    "$oid": "52451a6bf80be5d22b27489b"
  }
}
```

Other nodes sample

```
> db.snapshot(SDB_SNAP_COLLECTIONSPACES)
{
  "Collection": [
    {
      "Name": "bar"
    }
  ],
  "PageSize": 4096,
  "Name": "foo",
  "GroupName": "datagroup1"
}
```

Database Snapshot

Description

Database snapshot contains main status and performance monitoring parameter in current database node. It returns one record.

Sign

SDB_SNAP_DATABASE

Proper nodes

All Node

Field Information

Field Name	Type	Description
HostName	String	The host name of physical node of database node.
ServiceName	String	Service name specified by svcname. HostName and service name are the sign of a logical node.
NodeName	String	Node name, in the format of <HostName>:<ServiceName>
GroupName	String	The name of replset of the logical node. There is no this field under independent mode.
IsPrimary	Bool	IsPrimary shows whether the node is a main node.There is no this field under independent mode.
ServiceStatus	Bool	ServiceStatus represents the status of whether it is availible to serve. For example,Full Sync will make it *false*.
CurrentLSN.Offset	Long integer	Excursion of current LSN
CurrentLSN.Version	Integer	Verssion number of the current LSN
Version.Major	Integer	Major version number of database
Version.Minor	Integer	Minor version number of database
Version.Release	Integer	Released version number of database
CurrentActiveSessions	Integer	Current active session. It contains User EDU and system EDU.
CurrentIdleSessions	Integer	Current inactive session. Generally, inactive session means that EDU is stored in thread pool and waiting to run.
CurrentSystemSessions	Integer	Current system session. CurrentActiveSessions is the amount of current user EDUs.
CurrentContexts	Integer	Amount of current contexts
ReceivedEvents	Integer	Total amount of event requests received by current shard
Role	String	The role of current node
Disk.DatabasePath	String	The path of database
Disk.LoadPercent	Integer	The percentage of memory in databse path in disk
Disk.TotalSpace	Long integer	Total space (byte) in the database path

Field Name	Type	Description
Disk.FreeSpace	Long integer	Free space (byte) in the database path Ipmortant：this field and above all fields only display in data node and catalog node
TotalNumConnects	Integer	Total amount of database connection requests
TotalDataRead	Long integer	Total data read requests
TotalIndexRead	Long integer	Total index read requests
TotalDataWrite	Long integer	Total data write requests
TotalIndexWrite	Long integer	Total index write requests
TotalUpdate	Long integer	Total amount of updated records
TotalDelete	Long integer	Total amount of deleted records
TotalInsert	Long integer	Total amount of inserted records
ReplUpdate	Long integer	Amount of updated and replicated records
ReplDelete	Long integer	Amount of deleted and replicated records
ReplInsert	Long integer	Amount of inserted and replicated records
TotalSelect	Long integer	Total amount of selected records
TotalRead	Long integer	Total amount of read records
TotalReadTime	Long integer	Total data read time (millisecond)
TotalWriteTime	Long integer	Total data write time (millisecond)
ActivateTimestamp	Timeline	The time of starting data node
UserCPU	Float	User CPU (second)
SysCPU	Float	System CPU (second)
ErrNodes.NodeName	String	Exception node name (hostname +port) Ipmortant：this field only display in coord node and existing exception node
ErrNodes.Flag	Int	Rrror code this field only display in coord node and existing exception node

Sample

```
> db.snapshot(SDB_SNAP_DATABASE)
{
  "HostName": "vmsvr1-rhel-x64",
  "ServiceName": "51000",
  "GroupName": "foo",
  "NodeName": "vmsvr1-rhel-x64:51000",
  "IsPrimary": true,
  "ServiceStatus": false,
  "CurrentLSN": {
    "Offset": 84,
    "Version": 1
  },
  "Version": {
    "Major": 0,
    "Minor": 0,
    "Release": 3598
  },
  "CurrentActiveSessions": 8,
  "CurrentIdleSessions": 0,
  "CurrentSystemSessions": 6,
  "CurrentContexts": 1,
  "ReceiveCount": 3,
  "Role": "data",
  "Disk": {
    "DatabasePath": "/home/sequoiadb/sequoiadb/data1",
    "LoadPercent": 84,
    "TotalSpace": 37939249152,
    "FreeSpace": 5734342656
  },
  "TotalNumConnects": 1,
```

```
  "TotalDataRead": 0,
  "TotalIndexRead": 0,
  "TotalDataWrite": 0,
  "TotalIndexWrite": 0,
  "TotalUpdate": 0,
  "TotalDelete": 0,
  "TotalInsert": 0,
  "ReplUpdate": 0,
  "ReplDelete": 0,
  "ReplInsert": 0,
  "TotalSelect": 0,
  "TotalRead": 0,
  "TotalReadTime": 0,
  "TotalWriteTime": 0,
  "ActivateTimestamp": "2013-04-06-17.30.08.517367",
  "UserCPU": "0.240000",
  "SysCPU": "1.250000"
}
```


Operating System Snapshot

Description

Operating system snapshot contains main status and performance monitoring parameter of operating system of current database node. It returns one record.

Sign

SDB_SNAP_SYSTEM

Proper nodes

All Node

Field Information

Field Name	Type	Description
HostName	String	The host name of physical node of database node.
ServiceName	String	Service name specified by svcname. HostName and service name are the sign of a logical node.
NodeName	String	Node name, in the format of <HostName>:<ServiceName>
GroupName	String	The name of replset of the logical node. There is no this field under independent mode.
IsPrimary	Bool	IsPrimary shows whether the node is a main node.There is no this field under independent mode.
ServiceStatus	Bool	ServiceStatus represents the status of whether it is availible to serve. For example,Full Sync will make it *false*.
CurrentLSN.Offset	Long Integer	Excursion of current LSN
CurrentLSN.Version	Integer	Verssion number of the current LSN Ipmortant：this field and above all fields only display in data node and catalog node ,not in coord node.
CPU.User	Float Integer	Total user CPU time (seconds) cost after OS is started.
CPU.Sys	Float Integer	Total system CPU time (seconds) cost after OS is started.

Field Name	Type	Description
CPU.Idle	Float Integer	Total free CPU time (seconds) cost after OS is started.
CPU.Other	Float Integer	Total other CPU time (seconds) cost after OS is started.
Memory.LoadPercent	Integer	Percentage of used memory of the current operating system (including file system cache) Ipmortant：this field only display in data node
Memory.TotalRAM	Long In teger	Total memory space of the current operating system (byte)
Memory.FreeRAM	Long Integer	Free memory space of the current operating system (byte)
Memory.TotalSwap	Long Integer	Total swap space of the current operating system (byte)
Memory.FreeSwap	Long Integer	Free swap space of the current operating system (byte)
Memory.TotalVirtual	Long Integer	Total virtual space of the current operating system (byte)
Memory.FreeVirtual	Long Integer	Free virtual space of the current operating system (byte)
Disk.DatabasePath	String	Database Path Ipmortant：this field only display in data node
Disk.LoadPercent	Integer	The percentage of space of file system of data path . Ipmortant：this field only display in data node
Disk.TotalSpace	Long Integer	Total space in database path (byte)
Disk.FreeSpace	Long Integer	Free space in database path (byte)
ErrNodes.NodeName	String	Exception node name（hostname +port）Ipmortant：this field only display in coord node node and existing exception node
ErrNodes.Flag	Int	Rrror code Ipmortant：this field only display in coord node and existing exception node

Sample

```
> db.snapshot(SDB_SNAP_SYSTEM)
{
  "NodeName": "vmsvr2-suse-x64:51000",
  "HostName": "vmsvr2-suse-x64",
  "ServiceName": "51000",
  "GroupName": "datagroup1",
  "IsPrimary": false,
  "ServiceStatus": true,
  "CurrentLSN": {
    "Offset": 3764,
    "Version": 1
  },
  "NodeID": [
    1000,
    1000
  ],
  "CPU": {
    "User": 3947.31,
    "Sys": 715.11,
    "Idle": 331196.41,
    "Other": 771.14
  },
  "Memory": {
    "LoadPercent": 95,
    "TotalRAM": 4155072512,
```

```
    "FreeRAM": 202219520,
    "TotalSwap": 2153771008,
    "FreeSwap": 2137071616,
    "TotalVirtual": 6308843520,
    "FreeVirtual": 2339291136
  },
  "Disk": {
    "DatabasePath": "/opt/sequoiadb/database/data/51000",
    "LoadPercent": 78,
    "TotalSpace": 40704466944,
    "FreeSpace": 8615747584
  }
}
```

List

In this section, th following lists will be introduced:

Contexts list

Current session contexts list

Session list

Current session list

Collection list

Collection space list

Storage units list

Replset list

Context List

Description

Context list contains all the contexts corresponding to all the sessions in current data node.

A session is a record. If a session contains one or more contexts, the Contextd array field generate a for each of the contexts.

Note: The manipulation of list will generate a context, so the result set will contains at least one context of current list.

Sign

SDB_LIST_CONTEXTS

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	Long integer	Session ID
Contexts	Long integer array	Context ID array. It is a list of all the contexts of the session.

Sample

```
> db.list(SDB_LIST_CONTEXTS)
{
  "SessionID": 21,
  "Contexts": [
    182
  ]
```

```
}
```

Current Contexts List

Description

Current contexts list lists the session contexts corresponding to current connections in data nodes.

It returns one record. In a record, Contests array field contains all the contexts in current session.

Note: The manipulation of list will generate a context, so the result set of it at least contains one context.

Sign

SDB_LIST_CONTEXTS_CURRENT

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	Long integer	Session ID
Contexts	Long integer array	Context ID array. It is a list of all the contexts of the session.

Sample

```
> db.list(SDB_LIST_CONTEXTS_CURRENT)
{
  "SessionID": 21,
  "Contexts": [
    183
  ]
}
```

Session List

Description

Session list contains all the sessions between users and system on current database node. Each session is recorded as one record.

Sign

SDB_LIST_SESSIONS

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	Integer or long integer	Session ID
TID	Integer	System thread ID corresponding to the session
Status	String	Session status: • Creating：creating status • Running：running status

Field Name	Type	Description
		• Waiting：waiting status • Idle：idle status of thread pool • Destroying：destroying status
Type	String	EDU type
Name	String	EDU name. Generally, it is null.

Sample

```
> db.list(SDB_LIST_SESSIONS)
{
  "SessionID": 1,
  "TID": 6168,
  "Status": "Running",
  "Type": "TCPListener",
  "Name": ""
}
{
  "SessionID": 2,
  "TID": 6169,
  "Status": "Running",
  "Type": "HTTPListener",
  "Name": ""
}
...
{
  "SessionID": 21,
  "TID": 6691,
  "Status": "Running",
  "Type": "Agent",
  "Name": "192.168.20.101:52741"
}
```

Current Session List

Description

Current session list contains the current sessions on current database node. Each session is recorded as one record.

Sign

SDB_LIST_SESSIONS_CURRENT

Proper Node

data node,catalog node

Field Information

Field Name	Type	Description
SessionID	Integer or long integer	Session ID
TID	Integer	System thread ID corresponding to the session
Status	String	Session status: • Creating：creating status • Running：running status • Waiting：waiting status • Idle：idle status of thread pool • Destroying：destroying status
Type	String	EDU type

Field Name	Type	Description
Name	String	EDU name. Generally, it is null.

Sample

```
> db.list(SDB_LIST_SESSIONS_CURRENT)
{
  "SessionID": 21,
  "TID": 6691,
  "Status": "Running",
  "Type": "Agent",
  "Name": "192.168.20.101:52741"
}
```

Collections List

Description

Collections list contains all the non-temporary collections in current data nodes (It contains user collection in coord node). Each collection is recorded as a record.

Sign

SDB_LIST_COLLECTIONS

Proper Node

All nodes

Field Information

Since information stored in data nodes and catalog node is not the same, collections list will return different structures according to the kind of the node:

Coord Node Field Information

Field Name	Type	Description
Name	String	Full name of collection
Version	Integer	Collection version. The version number will increase by 1 when any metadata manipulation is done on the collection (such as data split).
ReplSize	Integer	The size of replication collection. Only when the amount of nodes, which are in the replset and have received a request of write operation, is greater than the value of ReplSize, can the request returns to applications.
ShardingKey	Object	Definition of sharding key
CataInfo	Array	Shard information of collection. Every shard range corresponds to an object element of a array.
CataInfo.GroupID	Integer	The ID of replset in the shard
CataInfo.LowBound	Object	The lower boundary of the shard. The range contains the lower boundary. The amount of fields in the object equals that in ShardingKey. Each field name is null.
CataInfo.UpBound	Object	The upper boundary of the shard. The range doesn't contain the upper boundary. The

Field Name	Type	Description
		amount of fields in the object equals that in ShardingKey. Each field name is null.

Other node field information

Field Name	Type	Description
Name	String	Full name of collection
Details.ID	Integer	Collection ID. Range from 0 to 4095. Unique in collection space.
Details.LogicalID	Integer	Collection logical ID
Details.Sequence	Integer	Sequence number
Details.Indexes	Integer	The amount of indexes in the collection
Details.Status	String	The current status of the collection • Free • Normal • Dropped • Offline Reorg Shadow Copy Phase • Offline Reorg Truncate Phase • Offline Reorg Copy Back Phase • Offline Reorg Rebuild Phase

Sample of coord node:

```
> coord.snapshot(SDB_LIST_COLLECTIONS)
      {
        "_id" : { "$oid" : "515bc719d3f60d7132a24bdc" },
        "ReplSize" : 2,
        "Name" : "foo.test",
        "Version" : 1,
        "ShardingKey" : {
          "Field1" : 1,
          "Field2" : -1
        },
        "CataInfo" : [
          {
            "GroupID" : 1000,
            "LowBound" : {
              "" : MinKey,
              "" : MaxKey
            },
            "UpBound" : {
              "" : MaxKey,
              "" : MinKey
            }
          }
        ]
      }
```

Sample of other nodes:

```
> db.list(SDB_LIST_COLLECTIONS)
{
  "Name": "foo.test",
  "Details": [
    {
      "ID": 0,
      "Logical ID": 0,
      "Sequence": 1,
      "Indexes": 2,
      "Status": "Normal"
    }
  ]
}
```

Collection Space List

Description

Collection space list contains all the collection space in the current data node. Each collection space is recorded as one record.

Sign

SDB_LIST_COLLECTIONSPACES

Proper Node

All nodes

Field Information

Since information stored in data nodes and catalog node is not the same, collection space list will return different structures according to the kind of the node:

Coord Node Field Information

Field Name	Type	Description
Name	String	Collection space name
Collection	String array	All the collection in the collection space
PageSize	Integer	Size of data page in collection space
Group	Integer array	ID list of the replset of the collection space.

Other node field information

Field Name	Type	Description
Name	String	Collection space name
Collection	String array	All the collections in collection space
PageSize	Integer	Size of data page in collection space

Coord node sample

```
> db.list(SDB_LIST_COLLECTIONSPACES)
{
  "Collection": [
    {
      "Name": "test"
    }
  ],
  "Group": [
    {
      "GroupID": 1000
    }
  ],
  "Name": "foo",
  "PageSize": 4096,
  "_id": {
    "$oid": "515fe97baf628545ac4234bb"
  }
}
```

Other nodes sample

```
> db.list(SDB_LIST_COLLECTIONSPACES)
{
  "Collection": [
    {
      "Name": "test"
    }
```

```
  ],
  "Name": "foo",
  "PageSize": 4096
}
```

Storage Units List

Description

Storage units list contains information of all the storage units on the current database node.

Sign

SDB_LIST_STORAGEUNITS

Proper Node

Data node,catalog node

Field Information

Field Name	Type	Description
Name	String	Collection space name.
ID	Integer	Collection space ID
LogicalID	String	Collection space logical ID, ascending order
PageSize	Integer	Size of data page in collection space
Sequence	Integer	Sequence. In current verssion, it is "1".
NumCollections	Integer	The amount of collections in collection space.
CollectionHWM	Integer	Collection high water marks. Generally, it is the amount of collections which has been generated in the collection space (including deleted collection).
Size	Long Integer	Size of storage unit (byte).

Sample

```
> db.list(SDB_LIST_STORAGEUNITS)
{
  "Name": "testCS",
  "ID": 4095,
  "LogicalID": 0,
  "PageSize": 4096,
  "Sequence": 1,
  "NumCollections": 1,
  "CollectionHWM": 1,
  "Size": 172032000
}
{
  "Name": "foo",
  "ID": 4094,
  "LogicalID": 1,
  "PageSize": 4096,
  "Sequence": 1,
  "NumCollections": 2,
  "CollectionHWM": 3,
  "Size": 172032000
}
```

Replset list

Description

Replset list contains information of all shards in th current cluster.

Sign

SDB_LIST_GROUPS

Proper node

coord node

Field Information

Field Name	Type	Description
Group.dbpath	String	The path of data file of nodes in replset
Group.HostName	String	Host name of node in replset
Group.Service.Type	Integer	Service type of node in replset • 0：direct-connection service, corresponding to database parameter *svcname* • 1：replication service, corresponding to database parameter *replname* • 2：shard service, corresponding to database parameter *shardname* • 3：catalog service, corresponding to database parameter catalogname
Group.Service.Name	String	Service name of node in replset. Service name may be port number or host name in service file.
Group.NodeID	Integer	Node ID in replset
GroupID	Integer	Replset ID
GroupName	String	Replset name
PrimaryNode	Integer	Master nodeID
Role	Integer	Replset role.It can be: • 0：data node • 2：catalog node
Status	String	Status of replset • 1：active repleset • 0：unactive repleset • does not exist：unactive repleset
Version	String	

Sample

```
> db.list(SDB_LIST_GROUPS)
{
  "Group":[
    {
      "dbpath": "/home/users/chenzichuan/sequoiadb/cata",
      "HostName": "ubuntu-dev2",
      "Service": [
        {
          "Type": 0,
          "Name": "30000"
        },
        {
          "Type": 1,
```

```
        "Name": "30001"
      },
      {
        "Type": 2,
        "Name": "30002"
      },
      {
        "Type": 3,
        "Name": "30003"
      }
    ],
    "NodeID": 1
  }
],
"GroupID": 1,
"GroupName": "SYSCatalogGroup",
"PrimaryNode": 1,
"Role": 2,
"Status": 1,
"Version": 1,
"_id": {
  "$oid": "51710981d8cb8fbc163d6350"
}
}
```


Engine Dispatchable Unit

Concept

Engine Dispatchable Unit is the carrier of tasks in SequoiaDB. Generally, EDU is an independent thread.

Each EDU can be used to execute users' requests or execute system internal maintenance tasks.

EDU is independent to each other. One EDU is charged for one user session. One user session is fixed on one EDU on one data node.

Each EDU has a thread-unique and 64-bit integer ID called "EDU ID".

There are two kinds of EDU: user EDU and system EDU, respectively representing threads that executes user task and threads that executes system task.

User EDU

User EDU is a thread executes user task. It is also called Agent

In SequoiaDB, ther are differnt kinds of agetns as follow:

Name	Type	Description
Agent	Agent	Agent thread is charged for requests of *svcname* service. Generally, the request is directly sent by user.
ShardAgent	Shard agent	Shard agent thread is charged for requests of *shardname* service. Generally, the request is directly sent from coord node to data node.
CoordAgent	Coord agent	Coord agent thread is charged for request of svcname service. Generally, the request is directly sent from user to coord node and only acts on coord node.
ReplAgent	Replication agent	Replication agent thread is charged for request of *replname* service. Generally, the request is directly sent from data master node to data slave node and acts on non-coord node.
HTTPAgent	HTTP agent	HTTP agent thread is charged for request of *httpname* service. Generally, the request is directly sent by user.

System EDU

System EDU is a kind of threads that maintains data structures in the system and guarantee the consistency of data and configuration. Generally, it is invisible to users.

In SequoiaDB, there are different kinds of EDU. The following EDU are just part of them:

Name	Type	Description
TCPListener	Service listener	The duty of this thread is to listen to *svcname* service, and start agent threads.
HTTPListener	HTTP listener	The duty of this thread is to listen to *httpname* service, and start agent threads.
Cluster	Cluster management	Cluster management thread is used to maintain the infrastructure of clusters, and start ReplReader threads and ShardReader threads.
ReplReader	Replication listener	*ReplReader* is charged for request of *replname* service, and start ReplAgent threads.
ShardReader	Shard listener	*ShardReader* is charged for request of *shardname* service, and start ShardAgent threads.
LogWriter	Log writer	LogWriter thread is used to write data from log cache to log file.
WindowsListener	Windows event listener	It is owned by Windows environment. It is used to listen to events defined by SequoiaDB.
Task	Background task	It is a kind of threads that cope with background tasks. For example,data split
CatalogMC	Catalog main controller	CatalogMC thread is used to receive and send requested sent to catalog nodes.
CatalogNM	Catalog node maintainer	CatalogNM thread is used to cope with requests relative to cluster information within catalog nodes.
CatalogManager	Catalog manager	CatalogManager thread is used to cope with requests relative to metadata information within catalog nodes.
CatalogNetwork	Catalog network listener	CatalogNetwork thread is used to listen to requests of *catalogname* service in coord network.
CoordNetwork	Coord network listner	CoordNetwork thread is used to listen to shard requests.

Monitorinf

Users can usesession snapshot to monitor system and user EDU.

Log

Sync Log

Sync-Log

Log Files

In SequoiaDB,it with log for data synchronization among data nodes,the log files is in the directory *replicalog*.The size and number of log file can be configured by setting *logfilesz* and *logfilenum*,default is *64M* and *20*.It can't be modified if the params has entryed into force.(if you want to modify,you must

delete all the log files off-line,reconfigure the params and restart SequoiaDB.But this will lead to full-sync.)

Sync

All slave nodes in data group will regularly packaged download the logs of other nodes to local for log replaying. Sync source is not limited to the master node.As we are hope the gap of data version in all nodes is in the range of a small window.When in the range of window,all the slave nodes synchronize to master node.But when the data version of some nodes is too larger differ from master node,they will select other slave nodes to synchronize.And if have conficted versions,be based on the current data verson of master node.If the confiction can't be solved,then enter into full-sync.No master node, no sync.

Full-sync

Causes of triggering full-sync:

1. downtime and restart.

2. data version of some nodes is too large differ from other nodes.

3. the data is inconsistent and can't be repaired.

Note: After restarting in mormal,If the data version is still in the range of synchronization,then it will not trigger full-sync.

the node which experienced full-sync will clear all of local data and logs,and copy all of data in another node(not limit master node) in the group to local,as well as the changed data in sync source.The node in the time of full-sync does not provide services to external,and no master node,no sync.Full-sync will greatly affect the performance of data group,and even lead to lower the synchronous performance of other slave nodes.So adding shardings or increating log size to avoid full-sync for proposing.

Backup And Recovery

Data Backup

Data Recovery

Data Backup

To advoid data unavailable caused by hardware,the users need to regularly backuping for the database.The backup file as a database "snapshot" at a time,contains all of the information including the backuping time.The users can use the backup files to restore the database to the point of backuping time,then the contents in database will be exactly consistent with the point.

In current versions,not to provide automatic backup-recovery tools,you need to manually complete the backuping and recoverying operations.

And it doesn't support backup-rollback ,all of the backups are the type of snapshot.

During the time of backuping,not do any changes to metadata,such as creating and dropping collections or data nodes,splitting.

Backup Configuration Files

Backup Data Files

Backup Configuration Files

There have configurable information for each nodes in configurable files,the configurable files in the directory of *<installed directory of SequoiaDB>/conf/local/<service name>*by default

Standalone Mode

Cluster Mode

Sample

Standalone Mode

The configuration file "sdb.conf" is existed in the direcroty of the users specified.if not specified,not existed.

It is only needed to file the "sdb.conf" that specified to backup the configuration file at the standalone mode.

Cluster Mode

In the cluster mode,the nodes in SequoiaDB are divided into three roles:data nodes,catalog nodes,coord nodes.The users need to file "sdb.conf" in the directory of configuration files for each nodes.

Note: In the cluster,the name of each node's configuration file is "sdb.conf",So,it is needed to specify different paths for each node to avoid overwritting.

Data Nodes

The users can get the physical machine and service names for each data node by querying coord nodes.

Catalog Nodes

To start the catalog nodes, you must specify the path of configuration file.

Coord Nodes

The users can get the physical machine and service names for each coord node by querying catalog nodes.

for more information

Sample

The users can get the physical machine and service names for each data node by the following steps.Assumming the directory of SequoiaDB is "/opt/sequoiadb/",and executing "/opt/sequoiadb/bin/sdb " to enter SequoiaDB Shell .

1. Connect to coord node

```
db=new Sdb('192.168.20.35', 60000)"
192.168.20.35:60000
```

2. Query the collection "SYSCAT.SYSNODES" to get the information of configuration files

```
db.SYSCAT.SYSNODES.find()
{
  "Group": [
    {
      "dbpath": "/home/sequoiadb/sequoiadb/cata",
      "HostName": "vmsvr1-rhel-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "60000"
        },
        {
          "Type": 1,
          "Name": "60001"
        },
        {
          "Type": 2,
          "Name": "60002"
        },
        {
          "Type": 3,
          "Name": "60003"
        }
      ],
      "NodeID": 1
    }
  ],
```

```
  "GroupID": 1,
  "GroupName": "SYSCatalogGroup",
  "PrimaryNode": 1,
  "Role": 2,
  "Status": 1,
  "Version": 1,
  "_id": {
    "$oid": "52147be0fe58d39ace53ef3a"
  }
}
{
  "Group": [
    {
      "HostName": "vmsvr1-rhel-x64",
      "dbpath": "/tmpfs/data1",
      "Service": [
        {
          "Type": 0,
          "Name": "51000"
        },
        {
          "Type": 1,
          "Name": "51001"
        },
        {
          "Type": 2,
          "Name": "51002"
        }
      ],
      "NodeID": 1000
    }
  ],
  "GroupID": 1000,
  "GroupName": "foo",
  "PrimaryNode": 1000,
  "Role": 0,
  "Status": 1,
  "Version": 2,
  "_id": {
    "$oid": "52147beffe58d39ace53ef3b"
  }
}
{
  "Group": [
    {
      "HostName": "vmsvr1-rhel-x64",
      "dbpath": "/tmpfs/data3",
      "Service": [
        {
          "Type": 0,
          "Name": "53000"
        },
        {
          "Type": 1,
          "Name": "53001"
        },
        {
          "Type": 2,
          "Name": "53002"
        }
      ],
      "NodeID": 1001
    }
  ],
  "GroupID": 1001,
  "GroupName": "foo1",
  "PrimaryNode": 1001,
  "Role": 0,
  "Status": 1,
  "Version": 2,
  "_id": {
    "$oid": "52147c1ffe58d39ace53ef3e"
  }
}
```


The output above shows three replicasets,each replicaset contains one node.The "SYSCatalogGroup" is coord replicaset,and "foo" and "foo1" are data replicasets.the name of physical machine where each node in is the value of "HostName" field and the service name is the value of "name" field in the "Service"

field.You can locate the configuration files'path of the specified server.The following sample shows the unique data node configuration file of the replicaset "foo".

```
[vmsvr1-rhel-x64]$ ls /opt/sequoiadb/conf/local/51000
sdb.conf
```

Data Files Backup

Standalone Mode

Cluster Mode

Sample

Standalone Mode

In standalone node,users must completely stop the database before archiving all the files in database and index direcroties.

In the case of specifying configuration files,the database and index directories are the specified by the dbpath and indexpath in the configuration file.By default,the value of them are same.

And if configuration file doesn't exist,database directory is specified at the time of starting database.

Cluster Mode

Data nodes and Coord nodes

In cluster mode, users can stop all of the replicasets and archive all files in the database and index directories for master node in each replicaset. The "PrimaryNode" field in the collection "SYSCAT.SYSNODES" marks the master node.

If users want to backuping without interrupting service,you can stop a slave node for each replicaset,and archive all files in the database and index directories for the node.At last,restarting the node after backuping,and the node will automatically sync with other nodes within the replicaset.

In the case of specifying configuration file,the database and index directories correspond with the directories that "dbpath" and "indexpath" specified in configuration file for each node.By default,the value of "indexpath" and "dbpath" are same.

Catalog nodes

As the catalog nodes does not contain any user data,so no backup.

for more information

Sample

1. Find the node's physical machine and service from catalog node
2. Open the node's configuration file,find the "dbpath" and "indexpath".By default,the value of them are the same.

```
$ cat /opt/sequoiadb/conf/local/51000/sdb.conf
svcname=51000
dbpath=/tmpfs/data1
diaglevel=5
transactionon=true
role=data
catalogaddr=vmsvr1-rhel-x64:60003
```

3. Connect to coord ,and stop node

```
$ ~/sequoiadb/bin/sdb
>db=new Sdb('192.168.20.35',50000)
192.168.20.35:50000
>rg=db.getRG('foo')
foo
>node=rg.getNode('vmsvr1-rhel-x64:51000')
vmsvr1-rhel-x64:51000
>node.stop()
```

4. archive the files in the database and index pathes.

```
$ tar -zcvf vmsvr1-rhel-x64.50000.2013-08-21.tar.gz /tmpfs/data1/*.dat /tmpfs/data1/*.idx
```

```
/tmpfs/data1/foo.dat
/tmpfs/data1/one.dat
/tmpfs/data1/SYSTEMP.dat
/tmpfs/data1/foo.idx
/tmpfs/data1/one.idx
/tmpfs/data1/SYSTEMP.idx
```

5. Restart node

```
$ ~/sequoiadb/bin/sdb
>db=new Sdb('192.168.20.35',50000)
192.168.20.35:50000
>rg=db.getRG('foo')
foo
>node=rg.getNode('vmsvr1-rhel-x64:51000')
vmsvr1-rhel-x64:51000
>node.start()
```

It is also applicable for catalog.

Note:

Please ensure the node is completely stopped before archiving data and index files,otherwise,the backuping files may be damaged.

Data Recovery

The operation of data recovery is to revert the nodes to a time after backuping.

During the time of recovery,not do any changes to metadata,such as creating and dropping collections or data nodes,splitting.

If have changes to metadata during backuping,then data recovery may result in data inconsistency or damaged

Standalone Mode

Cluster Mode

Sample

Standalone Mode

In standalone mode,users need to manually stop the database,and take the configuration files and data files to replace the current files.

Cluster Mode

In cluster mode,users need to stop the replicaset that to be restored and then recovery the configuration and data files for each node within the replicaset.

During the recovering,you must completely stop the replicaset that to be restored,and ensure

1)each of the nodes within the replicaset uses the same backuping files to recovery.

2)delete the log files for each nodes within the replicaset.

for more information

Sample

1. Stop the replicaset that to be restored

```
$ /opt/sequoiadb/bin/sdb
>db=new Sdb('192.168.20.35',50000)
192.168.20.35:50000
>rg=db.getRG('foo')
foo
>rg.stop()
```

2. Recover configuration file

```
$ cp vmsvr1-rhel-x64.51000.sdb.conf /opt/sequoiadb/conf/local/51000/sdb.conf
```

3. Recover data files

```
$ tar -zxvf vmsvr1-rhel-x64.51000.2013-08-21.tar.gz
/tmpfs/data1/foo.dat
/tmpfs/data1/one.dat
```

```
/tmpfs/data1/SYSTEMP.dat
/tmpfs/data1/foo.idx
/tmpfs/data1/one. idx
/tmpfs/data1/SYSTEMP.idx
```

4. Delete log files.The log files are saved in the "logpath" directory in the configuration file,*<SequoiaDB installed path >/replicalog* by default.

```
$ rm -rf /tmpfs/data1/replicalog
```

5. Start replicaset

```
$ /opt/sequoiadb/bin/sdb
>db=new Sdb('192.168.20.35',50000)
192.168.20.35:50000
>rg=db.getRG('foo')
foo
>rg.start()
```

Note: It will result in data inconsistency among data nodes within replicaset if only recover one node.

DataBase Tool

contents：

Data Migration--Import

sdbimprt

sdbimprt is a import tool in SequoiaDB database, it can achieve from a JSON format or a CSV format data stored file import into Sequoiadb database.

The record of JSON format must meet the json definition:Left and right curly braces ({})as a record delimiter; and the string data type must be contained in doublequotation marks;and escape character is defined as the backslash(₩).

CSV is called for short Comma-Separated Value, each record is separate by 0x0D in default,and fields separated by commas.Users can specify the delimiter between fields,as well as the records.Delimiters defined(only support ASCII characters):

usage	default	be equipped
string delimiter	"	yes
character delimiter	,	yes
record delimiter	'₩n'(0X0D)	yes

Options

Param	Description
--help,-h	return the basic help and usage information
--hostname,-h	import data from Sequoiadb of the specified hostname.In default,the sdbexprt try to connect to local hostname.
--svcname,-p	specify port.In default,the sdbimprttry to connect to the host which port number is 50000.
--type,-t	specify the format of import data.It can be CSV or JSON.
--input,-i	specify the name of file want to import. note:don't write extensions name.

Param	Description
--delchar,-a	specify the character delimiter." " is default,it is no action for JSON format.
--delfield,-e	specify the field delimiter. "," is default,no action for JSON format.
--delrecord,-r	specify the record delimiter."₩n" is default,also no action for JSON format.
--csname,-c	specify the collectionspace name of exported data.
--clname,-l	specify the collection name of exported data.

Usage

In the following sample,sdbimprt will import data into local database which port is 50000,and collectionspace name is *foo*,collection name is *bar*,and import type is csv,import file is *test.csv*.

Note: the file must hava extensions name *csv*,but can't write the extensions name of file when impotting.

```
sdbimprt #s localhost #p 50000 #t csv #itest #c foo #l bar
```

sdbload

sdbload is a batch import tool in SequoiaDB database, it can achieve from a JSON format data stored file import into Sequoiadb database.

The record of JSON format must meet the json definition:Left and right curly braces ({})as a record delimiter; and the string data type must be contained in doublequotation marks;and escape character is defined as the backslash(₩).。

Options

Params	Description
--help,-h	return the basic help and usage information
--svcname,-p	specify port.In default,the sdbloadtry to connect to the host which port number is 50000.
--filename,-f	specify the imported file name.
--input,-i	specify the imported file name. Note:don't write extensions name.
--user,-u	user name(optional)
--password,-w	password(optional)
--asynchronous,-a	whether asynchronous load. Note:if run asynchronous load,the result of traversal query or index query may be inconsistent in a short time.
--CollectionSpace,-c	specify the collectionspace name of imported data.
--Collection,-l	specify the collection name of imported data.

Usage

In the following sample,sdbload will import data into local database which port is 50000,and collectionspace name is *foo*,collection name is *bar*,and import file is *test.json* with asynchronous loading.

```
sdbload #p 50000 #f test.json #c foo #l bar #a true
```

Data Migration--Export

sdbexprt

sdbexprt is a practical tool, it can export a JSON format or a CSV format data stored file from SequoiaDB database .

Options

Param	Description
--help,-h	return the basic help and usage information
--hostname,-s	export data from Sequoiadb of the specified hostname.In default,the sdbexprt try to connect to local hostname.
--svcname,-p	specify port.In default,the sdbexprttry to connect to the host which port number is 50000.
--type,-t	specify the format of export data.It can be CSV or JSON.
--output,-o	specify the name of file want to export. note:don't write extensions name.
--delfield,-e	specify the field delimiter. "," is default,no action for JSON format.
--delrecord,-r	specify the record delimiter."₩n" is default.
--fieldlist,-f	specify one or more field names to export,separated by commas(,).
--fieldincluded,-i	specify if export field name.no action for JSON format.
--csname,-c	specify the collectionspace name of exported data.
--clname,-l	specify the collectionspace of exported data.

Usage

In the following sample,sdbexport will export data from the local database which port is 50000,and collectionspace name is *foo*,collection name is *bar*,and export type is csv,export file name is *contact*,export field are *field1* and *field2*.

```
sdbexprt -s localhost #p 50000 #t csv #o contace -f field1,field2 #c foo #l bar
```

Database detecting tool--sdbinspt

sdbinspt is a data file detection tool in SequoiaDB , it can detect the structure of data file true or false and present the result report.

Authorization

The user must have read permission for the data and index in database when running sdbinspt command.

Required connection

None

Options

Params	Description
--help, -h	return the basic help and usage information

Params	Description
--dbpath, -d	specify the path of data file, If not be specified, the default path is *current path*.
--output, -o	specify the output file,If not be specified, default is *screen output*
--verbose, -v	whether for ASCII text output(*true/false*),default is *true* .
--csname, -c	specify the collectionspace name,If not be specified,all collectionspaces.
--clname, -l	specify the collection name,If not be specified,all collections.
--action, -a	specify a action in one of inspect,dump and all，it can't be null, inspect:to check and report any data corruption. dump:to format data page,and output. all :to check data page corruption and formatted output data page.
--dumpdata -t	specify whether to formatted output data or not(t*true/false*),default is *false*.
--dumpindex, -i	specify whether to formatted output *index* or not(t*true/false*),*false* is default.
--pagestart, -s	specify the starting data page,*0* is default.
--numpage, -n	specify whether need to check or format the number of data page,default is *all*

Usage

The following sample shows sdbinspt will detect in current directory and formatted output the data and indexes in all of collectionspaces and collections to *output.txt* file .

```
sdbinspt #d . #o output.txt #v true #a all #t true #i true
```

Cluster Management

This section describe some management conceptions and operations about cluster,including create/ delete of catalog/data sharding and increasing of coord node.

By understanding these concepts and methodss of operation,to better achieve expand or adjust the cluster.

catalog：

create master in cluster

catalog sharding management

data sharding management

coord sharding management

sample

Add master to cluster

If want to add a new master(Physical Machine or Virtual Machine) to deploy catalog node or data node,then please configure the master system according to the following steps:

1.Install the same operating system with other master,and configure IP address.

2.According to System configuration requirement section configure master/kernel param,and add the corresponding relationship between master and IP address to the directory /etc/hosts.

3.Modify each cluster master file in the directory /etc/hosts,add the corresponding relationship between new master and IP address to the directory /etc/hosts.

4.According to System configuration requirement section verify the correctness of configuation.

5.According to SequoiaDB system install section install SequoiaDB software.please keep the configuration management service port and the port existing systems consistency.

Catalog Shardings Management

This section introduce how to add shardings or nodes in catalog shardings

directoty:

Create New Catalog Sharding

Note: If creating new nodes relate to create new master,please refer to the section Add master to cluster to complete the configuration of host name and params about master.

There is only one catalog sharding in the database cluster.So create new catalog sharding have already completed in the time of installed and there is no need execute creating new sharding operation after installing.Please refer to the section Configuration and startup of cluster mode. operation methods:

```
>db.createCataRG(<host>,<service>,<dbpath>,[config])
```

This command is to create a new catalog sharding,at the same time create and start a catalog node,amongs:

host ：specify the host name of catalog node.

service ：specify the service port of catalog node,please ensure this port and the next three ports are unoccupied;for example, if set port 30000,then ensure 30000/30001/30002/30003 are unoccupied.

dbpath ：data file path,be used to store catalog data files,please ensure that the data administrator user has write access.(created when installed,dafault *sdbadmin*).

config ： optional param,be used to configure more detail params.the form is json.param list refer to the section database configuration ,for example configure the param of log size{logfilesz:64}。

Create New Node In Catalog Sharding

Note: If creating new nodes relate to create new master,please refer to the section Add master to cluster to complete the configuration of host name and params about master.

With the expansion of physical devices in the cluster,it can create more catalog nodes to improve the reliability of catalog service.

operation methods:

```
>var catarg = db.getRG("SYSCatalogGroup");
>var node1 = catarg.createNode(<host>,<service>,<dbpath>,[config]);
>node1.start()
```

The first command is used to get catalog sharding,"SYSCatalogGroup" is the name of catalog sharding.

The second command is used to create a new catalog node,amongs:

params:

host ：specify the host name of catalog node.

service ：specify the service port of catalog node,please ensure this port and the next three ports are unoccupied;for example, if set port 30000,then ensure 30000/30001/30002/30003 are unoccupied.

dbpath ：data file path,be used to store catalog data files,please ensure that the data administrator user has write access.(created when installed,dafault *sdbadmin*).

config ： optional param,be used to configure more detail params.the form is json.param list refer to the section database configuration ,for example configure the param of log size{logfilesz:64}。

The third command is used to start the added catalog node.

Data Sharding Management

This section introduce how to create data shardings and data nodes in the data sharding .

Directory:

Create New Data Sharding

Note: If creating new nodes relate to create new master,please refer to the section Add master to cluster to complete the configuration of host name and params about master.

It can configure many data shardings in a cluster,the max configurable is 60000.By adding shardings,you can take advantage of the physical device for horizontal expansion,Theoretically,SequoiaDB can do linear horizontal scalability.

operation methods:

```
>var datarg = db.createRG("datagroup1")
>datarg.createNode("sdbserver1", 51000, "/opt/sequoiadb/database/data/51000")
>datarg.start()
```

The first command is used to create data sharding,and what different from catalog sharding is that this operation will not create any data node.The param is the name of data sharding.

The second command is used to create a new node in the data sharding,many times as needed to execute this command to create more data nodes.

amongs:

host ：specify the host name of catalog node.

service ：specify the service port of catalog node,please ensure this port and the next three ports are unoccupied;for example, if set port 51000,then ensure 51000/51001/51002/51003 are unoccupied.

dbpath ：data file path,be used to store catalog data files,please ensure that the data administrator user has write access.(created when installed,dafault *sdbadmin*).

config ： optional param,be used to configure more detail params.the form is json.param list refer to the section database configuration ,for example configure the param of log size{logfilesz:64}。

The third command is used to start data sharding,and it will start all the data nodes and provide sercices.

Create New Nodes In Data Sharding

Note: If creating new nodes relate to create new master,please refer to the section Add master to cluster to complete the configuration of host name and params about master.

Some data shardings may set few copies when creating,but with the increase of physical device,it may need to create the number of copies to improve the reliability of data shardings.

operation methods:

```
>var datarg = db.getRG(<groupname>);
>var node1 = datarg.createNode(<host>,<service>,<dbpath>,[config]);
>node1.start();
```

The first command is used to get the data sharding,the param of *groupname* is the name of data sharding.

The second command is used to create a new data node ,for the params,please refer to create new node in catalog sharding.

The third command is used to start data nodes.

Create coord node

When the size of cluster expanding,coord node is alo need to added with the expansion of cluster.For advice,one physical node configure one coord node.

1. create directory for the coord node configuration.

```
mkdir -p /opt/sequoiadb/conf/local/50000 ;
```

50000 is the service port of coord node,users can configure with the need.

2.copy the sample configuration file of coord node.

3. modify the configuration file.

```
cp /opt/sequoiadb/conf/samples/sdb.conf.coord     /opt/sequoiadb/conf/local/50000/sdb.conf
vi /opt/sequoiadb/conf/local/50000/sdb.conf
modify content
  # database path dbpath=/opt/sequoiadb/database/coord ;
  this param is the path of database,can modify with need,please ensure the path is already exists(if not,manually
 create)
the following line
  # catalog addr(hostname1:servicename1,hostname2:servicename2,...)
  # catalogaddr=
modify to
  # catalog addr(hostname1:servicename1,hostname2:servicename2,...)
  catalogaddr=sdbserver1:30003,sdbserver2:30003,sdbserver3:30003 ;
  this param is the service address and port of catalog
```

4. press:wq,save and quit vi.

5. create the path where storage the data file.

```
mkdir -p /opt/sequoiadb/database/coord ;
```

the path is the previous step configured.

6. start catalog node process.

```
/opt/sequoiadb/bin/sdbstart -c /opt/sequoiadb/conf/local/50000/
```

Sample

Case one:add new master,add a new data duplicate node in current data sharding

- current environment:

Configure items	Configure Situation
master	contain one master: master one:OS is suse 11 SP2 64bit, host name is vmsvr1-suse-x64,IP is 192.168.1.10
catalog sharding group	the group contain one catalog node,service port is 30000
data sharding group	one data sharding group,group name is datagroup1,contain one data node: data node one:service host name is vmsvr1-suse-x64,port is 51000
coord node	one coord node,port is 50000

- expected result after adjustment

Configure items	Configure Situation
master	contain two masters:

Configure items	Configure Situation
	master one:OS is suse 11 SP2 64bit, host name is vmsvr1-suse-x64,IP is 192.168.1.10 master two:OS is suse 11 SP2 64bit, host name is vmsvr2-suse-x64,IP is 192.168.1.11
catalog sharding	the group contain one catalog node,service port is 30000
data sharding group	one data sharding group,group name is datagroup1,contain two data nodes: data node one:service host name is vmsvr1-suse-x64,port is 51000 data node two:service host name is vmsvr2-suse-x64,port is 51000
coord node	one coord node,port is 50000

Operation steps

- step one:configure the new added master

  1.install operation system SUSE 11 SP2 64bit(keep consistent with the original system).

  2.login system with root.

  3.configure IP address:192.168.1.11.

  4.execute the following command,configuration master is vmsvr2-suse-x64

  ```
  hostname vmsvr2-suse-x64
  ```

  modify the file in /etc/hosts,add two lines:

  ```
  192.168.1.10 vmsvr1-suse-x64
  192.168.1.11 vmsvr2-suse-x64
  ```

  press :wq,save and quit.

  5.execute the following command,to check if the master configuration is work:

  ```
  ping vmsvr1-suse-x64  //check if the comman ping is ok;
  ping vmsvr2-suse-x64  //check if the comman ping is ok;
  ```

  6.configure the system kernel param:

  open the file /etc/profile,add the following lines at the end of the file:

  ```
  ulimit -Sf unlimited   //file size limit
  ulimit -St unlimited   //CPU time limit
  ulimit -Sv unlimited   //virtual memory limit
  ulimit -Sn 64000       //the number of files limit
  ulimit -Sm unlimited   //memory size limit
  ulimit #Su 32000      //the number of threads limit
  ```

  press:wq,save and quit;

  execute source /etc/profile，make the configuration work;

  7.execute the following command,to check if the system kernel param configuration is work:

  ```
  ulimit #a; check if the configuration is right;
  ```

- step two:configure the master of original cluster

  modify the file of original master *vmsvr1-suse-x64* in the /etc/hosts,add a new line in the configuration file:

  ```
  192.168.1.11   vmsvr2-suse-x64
  ```

  press :wq save and quit

- step three:install Sequoiadb in the added master

  refer toSequoiaDB system installsection to insatall the software.

- step four:add data nodes to current sharding

1. executed the following command in the name of master *vmsvr2-suse-x64*,started *sequoiadb shell* command line:

```
/opt/sequoiadb/bin/sdb
```

2. In the *SequoiaDB shell* command line,execute the following command to add a copy data node to current data sharding group:

```
> var db = new Sdb("localhost",50000)
> var datarg = db.getRG("datagroup1")
>node=rg.createNode("vmsvr2-suse-x64",51000,"/opt/sequoiadb /database/data/51000")
> node.start()
```

3．In the *SequoiaDB shell* command line,execute the following command to check the configuration information of the sharding group,you can see a new data node have added to the data group:

```
>db.listReplicaGroups();
...
{
  "Group": [
    {
      "dbpath": "/opt/sequoiadb/database/data/51000",
      "HostName": " vmsvr1-suse-x64",
      "Service": [
        {
          "Type": 0,
        {
          "Type": 1,
          "Name": "51001"
        },
        {
          "Type": 2,
          "Name": "51002"
        },
        {
          "Type": 3,
          "Name": "51003"
        }
      ],
      "NodeID": 1000
    },
    {
      "dbpath": "/opt/sequoiadb/database/data/51000",
      "HostName": " vmsvr2-suse-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "51000"
        },
        {
          "Type": 1,
          "Name": "51001"
        },
        {
          "Type": 2,
          "Name": "51002"
        },
        {
          "Type": 3,
          "Name": "51003"
        }
      ],
      "NodeID": 1001
    }
  ],
  "GroupID": 1000,
  "GroupName": "datagroup1",
  "PrimaryNode": 1000,
  "Role": 0,
  "Status": 1,
  "Version": 3,
  "_id": {
    "$oid": "51d673c1fde96799fbac6aad"
  }
}
```

System Security

Concept

The permission to log in the system is specified in security module. If a user without correct user name and password cannot visit database.

Details

1. When the system is started, security function is closed by default. Only when a user si created, security function can be automatically activated

2. User name and password should be specified when creating a user. User name is unique in the whole system

3. User name and password should be specified when deleting a user.

4. After security funciton is activated, user name and password should be specified when visiting database. In one session, identity verification should be fulfilled once. When a new session is started, identity verification should be fulfilled again.

5. Users are equal to 1each other. There is no super user. A user can delete any users.

6. When all users are deleted, security function is closed.

7.All passwords are transmitted and stored in the format of cipher text.

Development Instruction

Sequoiadb relative development infromation

SequoiaDB shell

Catalog

SequoiaDB Shell Introduction

SequoiaDB has a built-in javascript shell, users can communicate with SequoiaDB instance through command lines.It is very useful. Through it, users can manage system, check and run instance or try other things. Shell is a vital tool in SequoiaDB.

Run shell

Run sequoiadb(./sequoiadb) and start shell：

```
$ ./sdb
Welcome to SequoiaDB shell!
help() for help, Ctrl+c or quit to exit
>
```

Shell will automatically connect to SequoiaDB when SequoiaDB starts, so users should start SequoiaDB before using shell.

Shell is a javascript resolver with complete functions, which can run any javascript program. For example:

```
> y=200
200
> y/20
10
>
```

Users can also use javascript stdlib:

```
> new Date("2013/04/17");
Wed Apr 17 2013 00:00:00 GMT+0800 (CST)
> "hello,world".replace("world","SequoiaDB")
hello,SequoiaDB
>
```

Uses can define and invoke javascript function:

```
> function sdb(n){
... if(n<=1)return 1;
... else return n*sdb(n-1);
... }
> sdb(4);
24
>
```

Note: We can use multi-line commands. Shell will check out whether javascript statement is complete. If it is not complete, users can continue to input next line.

SequoiaDB Client

Shell can run any javascript program. But the particular feature of shell is that is a unique SequoiaDB client. When the system is started, with the command "db=new Sdb("localhost",50000)", users can connect to local database in sequoiaDB server and assign it to the global variable "db". This variable is the a main entrance to visit SequoiaDB in shell.

Shell also has some non-javascript commands. For example:

```
>db
localhost:50000
>db.create("foo")//create colleciton space
localhost:50000.foo
```

Users can visit the collection space in it through the variable "db". "db.foo" returns the collection space "foo" in current database. Users can visit collection in it like "db.foo.bar" which returns the collection "bar" in collection "foo". Since users can visit collections in shell, they can execute almost all manipulations in database.

Tips when using shell

Sequoiadb shell has built-in help documents. It can be read through the command of help.

```
> help
function () {
    println("   var db = new Sdb('localhost', 50000)   connect to database");
    println("   db.help()                              help on db methods");
    println("   db.cs.help()                           help on collection space
                                                                        cs");
    println("   db.cs.cl                               access collection cl on c
                                                                        ollection space cs");
    println("   db.cs.cl.help()                        help on collection cl");
    println("   db.cs.cl.find()                        list all records");
    println("   db.cs.cl.find({a:1})                   list records where a=1");
    println("   print(x), println(x)                   print out x");
    println("   getErr(ret)                          print error description for r
                                                                        eturn code");
    println("   quit                                   exit");
    }
```

- Database Help

  The command to check help document on database level:

  ```
  db.help()
  ```

- CollectionSpace Help

  The command to check help document of methods under collection space:

  ```
  db.<collectionspace>.help()
  ```

- Collection Help

  The command to check help document of methods under collection :

  ```
  db.<collectionspace>.<collection>.help()
  ```

- Cursor Help

  In shell, when using find() to read data in database, it uses all kinds of cursor methods and javascript method to cope with result set returned by the method find().

  The command to check help document of methods manipulating cursor:

  ```
  db.<collectionspace>.<collection>.find().help()
  ```

- Methods Help

  Additionally, there is a skill to understand the function of functions. When you type in the function name without brakets, it will show the javascript source code of the function. For example:

  ```
  > db.foo.bar.find
  function (query, select) {
      return new SdbQuery(this, query, select);
      }
  ```

Please visit http:// to check all the automatically generated javascript function API documents provide by shell.

Basic Manipulation in Shell

In shell, there are 4 basic manipulaitons :create, read, update and delete (CRUD).

- create
- read
- update
- delete

Create

In sequoiaDB, "create" means adding new document record into collections. We can add new record into collections through the method "insert". In sequoiaDB, the insert manipulations are as follow:

- If there is no "_id" field in the document record which is about to be inserted, the client will automatically add "_id" field and insert an unique value.
- If "_id" field is specified, the "_id" field in that collection should be unique, or exception will occur.
- The maximum length of BSON document is 16MB.
- The limits of fields are as follow:

  The field "_id" should be stored as primary key in a collection. The value of it should be unique and unchangable. The type of "_id" can be ordinary types except for array.

  The name of a field should not be null, start with "$" or contain (.).

Note: In this document, all examples use shell interface in sequoiadb.

insert()

The method insert() is used to insert record into insert records into sequoiaDB collection.The grammar of it is :

```
db.collectionspace.collection.insert(<doc|docs>, [flag])
```

Insert the first document

If there is no collection space and collection. Firstly, create collection (For example, "db.createCS("foo")" create collection space "foo") and collection (For example, "db.foo.createCL("bar")" create collection under collection space "bar"), then you are able to insert records.

```
db.foo.bar.insert(
  {
     _id:1,
     name:{fist:"Jhon",last:"Black"},
     phone:[1853742XXX,1802321XXX],
     remark:[
               {
                  position:"manager",
                  year:2000
                },
               {
                  position:"CEO",
                  year:2012
                }
             ]
  }
   )
```

Users can check out whether the insert is successful through the method "find()"

```
db.foo.bar.find()
```

The result is as follow:

```
 {
     _id:1,
     name:{fist:"Jhon",last:"Black"},
     phone:[1853742XXX,1802321XXX],
     remark:[
               {
                  position:"manager",
                  year:2000
```

```
            },
            {
              position: "CEO",
              year: 2012
            }
        ]
}
```

unspecified _id field

If a new document record doesn't conain "_id" field, the method insert() will add "_id" field into a document and generate an unique "$oid" value.

```
db.foo.bar.insert({name: "Tom", age: 20})
```

This command insert a newe record into collection "bar". In the record, the value of "name" is "Tom". The value of "age" is 20. The "_id" field is unique:

```
{ "_id": { "$oid": "515152ba49af395200000000" }, "name": "Tom", "age": 20 }
```

Insert more than one record

If users insert an array document in the method "insert", the method "insert()" will execute batch inserts in a collection.

The following command inserts 2 records into collection "bar". This manipulation demonstrates the feature of dynamic mode in sequoiaDB. Although the record with "_id:20" contains field name "phone", but the other one doesn't contain that, sequoiaDB doesn't require all records to contains this field.

```
db.foo.bar.insert([{name:" Mike" ,age:15}, {_id:20, name:" John" ,age:25, phone:123}])
```

Read

Of the basic database operations(i.e.CRUD),read operatios are these those retrieve records or documents from a collection in Sequoiadb, and include all operations that return a cursor in response to application request data.

This dacument describes the syntax and structure of the queries applications use to request data from Sequoiadb and how different factors affect the efficiency of reads.

Note:

All of the examples in this document use the sequoiadb shell interface.

find()

we use *find*method to read records from Sequoiadb。the *find*method is the main method to select records from collections and it returns a cursor which contains a lot of records.it's grammatical structure as follows:

```
db.collectionspace.collection.find([cond], [sel])
```

The corresponding opration in sql: the method of *find()* is simalar with *select* statement :

.the *[cond]* parameter corresponds to *where* statement

.the [sel] parameter corresponds to the list of fields in the result set

there is a record in collection like this：

```
   {
  "_id":1,
  "name":{
     "first" : "Tom",
     "second":"David"
      },
  "age":23
  "birth":"1990-04-01",
  "phone":[
      10086,
      10010,
      10000
```

```
    ],
 "family": [
     {
    "Dad": "Kobe",
    "phone": 139123456
     },
     {
     "Mom": "Julie",
     "phone": 189123456
     }
   ]
}
```

return all the records from the collection

If there is no *cond* parameter,the method of *db.collectionspace.colletion.find()* returns all the records from the collection.The following operation select all records in the collection named *bar* which in the collectionspace named *foo*:

```
db.foo.bar.find()
```

return records that match query conditions

- Equality match

  The following operation select the records in the collection *bar* where the *age* filed has the value 23

  ```
  db.foo.bar.find({age: 23})
  ```
- use *operators*

  The following operation select the records in the collection *bar* where the *age* field is more than($gt)20

  ```
  db.foo.bar.find({age: {$gt: 20}})
  ```
- On array

  (1)query an element,The following operation returns a cursor to all records in the *bar* collection where the array field *phone* contains the element '10086':

  ```
  db.foo.bar.find({"phone": 10086})
  ```

  (2)query array element which is BSON Object. The following operation return a cursor to all records in the *bar* collection where *family* array contains a subdocument element that contains the *Dad* field equal to 'Kobe' and the *phone* field equal to '139123456':

  ```
  db.foo.bar.find(
    {
   "family": {
     $elemMatch:  {
      "Dad": "Kobe",
      "phone": 139123456
    }
   }
   })
  ```
- query on subdocument

  The following operation returns a cursor to all documents in the *bar* collection where the subdocument name is exactly {first:"Tom"}

  ```
  db.foo.bar.find(
     {
      "name": {
      "first": "Tom"
       }
    }
    )
  ```

  It can also be written as：

  ```
  db.foo.bar.find(
      {
      "name.first": "Tom"
      }
    )
  ```

specity the field of the returned records

If specity the *sel* parameter of *find* method, then only return the field within sel parameter:

```
db.foo.bar.find(null, {name: ""})
```

Note:

If a records doesn't contain the specity field(as:people),Sequoiadb will default return. For example:

```
db.foo.bar.find({}, {name: "", people: ""})
return:  {
        "name": {
                  "fist": "Tom",
                  "second": "David"
                },
         "people": ""
         }
```

find() more information

Execute db.foo.bar.find().help() , then will see more methods of *find()*

- cursor.sort() on page 144sort(<sort>)

  The method of sort() is used to order by the specified field, syntax: sort({"field1":1|-1,"field2":1|-1,...}), '1' stand for Ascending,'-1' stand for Descending. If the the method of *find* doesn't specity the content of sel parameter, then *sort()* will order by the *sort* parameter, but if *sel* parameter specity the returned field, then *sort()* can only order by the field included in the *sel* parameter. For example:

  ```
  db.foo.bar.find().sort({age:1})   //return all the records in the bar collection and order by the value of age
   field in ASC
  ```

  ```
  db.foo.bar.find(null, {name: ""}).sort({age:1})  //this operation actually can't reach the purpose of sorting.
  ```

- cursor.hint() on page 144hint(<hint>)

  add indexes to speed up the query speed, assuming there is a index named 'testIndex':

  ```
  db.foo.bar.find().hint({"": "testIndex"})
  ```

- cursor.limit() on page 145limit(<num>)

  limit the number of records in the result set:

  ```
  db.foo.bar.find().limit(3) //return the first three records in the result set
  ```

- cursor.skip() on page 146skip(<num>)

  the skip() method controls the point of the results set, that is skipping the first 'num' records, begin to return from the 'num+1' records:

  ```
  db.foo.bar.find().skip(5)  //skipping the first 5 records in the bar collection
  ```

- use cursor to control the returned records

  ```
  db.foo.bar.find().current() //return the record that the current cursor point to
  db.foo.bar.find().deleteCurrent() //delete the record that the current cursor point to
  db.foo.bar.find().next() //return the next record that the current cursor point to
  db.foo.bar.find().updateCurrent() //update the record that the current cursor point to
  db.foo.bar.find().count()//return the number of records that the current cursor point to
  db.foo.bar.find().size() //return the distance from the current cursor to the final cursor
  db.foo.bar.find().toArray() //return the result sets with array form
  ```

Update

Of the basic database operations(i.e.CRUD),update operation are those that modify the existing records or documents in a collection, using *update()* method to modify documents in Sequoiadb.

Note:

All of the examples in this document use the sequoiadb shell interface.

update()

the update() method is the primary method used to modify documents in Sequoiadb, it has the following syntax :

```
db.collectionspace.collection.update(<rule>, [cond], [hint])
```

The corresponding opration in sql: the method of *update* is simalar with *update...set* statement :

.the [rule] parameter corresponds to *set* statement

.the *[cond]* parameter corresponds to *where* statement

.the [hint] parameter corresponds to the index name in index table

modify with update operator

If the *update()* method contains only *rule* parameter expression(such as the *$set* operator expression), the *update()* method updates the corresponding fields in the document. To update fileds in the subdocuments,use dot notation(.).

- update a field in a document

  use*$set* to update a value of a field. The following operation queries the bar collection for the document that has an _id field equal to 1 and sets the value of field *first* ,in the subdocument *name*, to 'Mike':

  ```
  db.foo.bar.update({$set: {"name.first": "Mike"}}, {_id: 1})
  ```

  Note:

  if the *rule* parameter containas fields not in the current records,the *update()* method will add the new fields to the records.

- remove a field from a record

  use*$unset* to remove the field from a record.The following operation queries the *bar* collection for all records where contains *age* field and remove the *age* filed , if a record doesn't contain the age field,skipping.

  ```
  db.foo.bar.update({$unset: {age: ""}})
  ```

- Update array

  if the update operation requires an update of an element in an array field, SequoiaDB use dot notation(.), array are zero-based. The following operation is to update second element in the *arr* array, it's value will increate 5:

  ```
  db.foo.bar.update({$inc: {"arr.1": 5}})
  ```

- hint parameter

  The following operation will traversal all the records through index,and update the value of field *name* to 'Tom', the name of index is 'textIndex'

  ```
  db.foo.bar.update({$set: {name: "Tom"}}, null, {"": "textIndex"})
  ```

Note: more update operator

Delete

Delete means deleting records in collection. In sequoiadb,remove() method is used to delete data.

Note:

In this document, all examples use shell interface in sequoiadb.

remove()

The method "remove()" is used to delete records in collection. The structure of it is:

```
db.collectionspace.collection.remove([cond],[hint])
```

The method "remove()" corresponds to the statemend "DELETE" in SQL:

.[cond] corresponds to the statement containing "where" in SQL

. [hint] corresponds to the name in index table in SQL

Delete Collection Record

- Delete all records in collection

  The following command will delete all records in collection "bar":

  ```
  db.foo.bar.remove()
  ```

- Delete all records that match the condition in collection

  The following command will delete all records that contain the value "Tom" in the field "name".

  ```
  db.foo.bar.remove({name:"Tom"})
  ```

- hint

  The following command will rapidly delete all records that contain the value "Tom" in the field "name" after searching them with index. The index name is "textIndex".

  ```
  db.foo.bar.remove({name:"Tom"},{"":"testIndex"})
  ```

Sequoiadb Application Development

Sequoiadb allows users to connect it in many ways such as C, C++, Java, PHP. This chapter introduces drivers for different programming language.

Java Driver

This section introduce java driver.

Overview

Overview

Java Driver Introduction

SQL to sequoiadb shell to java

java API

Java Client

Java Client Interface

Add jar package in Eclipse: sequoiadb.jar

Functions by Java Client Interface

Java client interface provide manipulations including database connection, data inserting, deleting, altering and querying, index creating and deleting, collection creating and deleting, collection space creating and deleting, etc.

- Class sample

  There are 3 degree of data in SequoiaDB:

  1)Database

2)Collection space

3)Collection

Logically, every collection space in SequoiaDB has no physical upper boundary. Every collection space is stored in a stand-alone file. So the amount of collection space depends on the size of disk and memory.

A collection space contains at most 4096 collections. Every collection can contain more than one record. In a physical system, the size of a collection space is at most 256GB.

Compared with relation database, we can regard record as line in relation database. We can regard collection as table in relation database.

So in client, there are 3 classes representing 3 kinds of objects. The other class represents cursor:

Sequoiadb	Database instance	It represents a stand-alone connection
CollectionSpace	Collection space instance	It represents a stand-alone collection space
DBCollection	Collection instance	It represents a stand-alone collection
DBCursor	Cursor Instance	It represents a result set of a query

Note:

1. As for a connection, in its collection space, collections share one socket. So in mutiple-thread system, the system should guarantee that the socket can be used merely by one thread whenever.

2. Generally, it is not recommended to manipulate a connection instance and other instance, which it generates, with more than threads.

3、If each thread merely use its own connection instance and other instances it generates, it is thread-safe.

- Exception

  Every interface may throw com.sequoiadb.exception.BaseException and java.lang.Exception, repectively corresponding to exceptions returned by database engine and client local exception. The information of BaseException can be read from its attributes: errorCode，errorType and errorDescription.

Topic title

Topic paragraph

java Driver Introduction

This page is a brief introduction to java driver of Sequoiadb. Please refer to .java API for more.

It is easy to use java driver. Firstly, download Java driver jar package "sequoiadb.jar". Then, import it to your project and use relative API.

The following program samples can be found in the catalog "com.sequoiadb.examples" in "smaples".

Create connection

In order to connect to SequoiaDB, we need at least one database name. If there's no database name, SequoiaDB will automatically generate one. Additionally, we should specify the server address and port. Here are 3 ways of connecting database

```
// the database server address, IP:PORT
String connString = "192.168.20.111:40000";
Sequoiadb sdb = null;
// connect to specifical db, may cause some baseException, e.g. Network error
try {
 sdb = new Sequoiadb(connString, "username", "password");
// or
// sdb = new Sequoiadb("username", "password");//Default connecting approach: local 50000 port
```

```
// or
// sdb = new Sequoiadb("IP", port, "username", "password");
} catch (BaseException e) {
 System.out.println("Failed to connect to database: " + connString + ", error description" + e.getErrorType());
 e.printStackTrace();
 System.exit(1);
}
```

- Get collection space

```
// create collectionspace, if collectionspace exists get it
CollectionSpace cs = null;
if(sdb.isCollectionSpaceExist(Constants.CS_NAME))
 cs = sdb.getCollectionSpace(Constants.CS_NAME);
else
 cs = sdb.createCollectionSpace(Constants.CS_NAME);
// or sdb.createCollectionSpace(csName, pageSize), need to specify the pageSize
```

- Get collection

```
// create collection, if collection exists get it
DBCollection cl = null;
if(cs.isCollectionExist(Constants.CL_NAME))
 cl = cs.getCollection(Constants.CL_NAME);
else
 cl = cs.createCollection(Constants.CL_NAME);
// or cs.createCollection(collectionName, options), create collection with some options
```

Insert record

Once collection object is gained, we can insert data into it or create a BSON Object to insert it (String corresponding to BSON structure canbe directly inserted).

```
// inserted data
BSONObject insertor1 = Constants.createChineseRecord();
BSONObject insertor2 = Constants.createEnglishRecord();
List <BSONObject> list = Constants.createNameList(200);

// insert operation
try {
insert a Chinese record
// chinese record
cl.insert(insertor1);

 insert a English record
// english record
cl.insert(insertor2);

 insert more than one record
// bulk insert
cl.bulkInsert(list, DBCollection.FLG_INSERT_CONTONDUP);

System.out.println("Successfully insert records");
} catch (BaseException e) {
 System.out.println("Failed to insert chinese record, ErrorType = "+ e.getErrorType());
} catch (Exception e) {
 e.printStackTrace();
 sdb.disconnect();
 System.exit(1);
}
```

Index

```
// create index key, index on attribute 'Id' by ASC(1)/DESC(-1)
BSONObject key = new BasicBSONObject();
key.put("Id", 1);
boolean isUnique = true;
boolean enforced = true;
try {
  Create an index
  // you can refer to the API to know more about this method
  cl.createIndex(Constants.INDEX_NAME, key, isUnique, enforced);
  // or
  // cl.createIndex(Constants.INDEX_NAME, "{'Id':1}", isUnique, enforced);

  Delete an index
  // drop index
  cl.dropIndex(Constants.INDEX_NAME);
} catch (BaseException e) {
```

```
    e.printStackTrace();
  }
```

Query

In order to show inserted records, the function find() is used to return a cursor object so that we can go through all results in the result set.

```
  import com.sequoiadb.base.DBCursor;
  try {
  Get curretn index of Collection. If it is not null, it can be used in a query.
  BSONObject index = null;
  DBCursor indexCursor = cl.getIndex(Constants.INDEX_NAME);
  if (indexCursor.hasNext())
  index = indexCursor.getNext();

  We get a Cursor from the query. It is used to go through result set.
  // result cursor
  DBCursor dataCursor = null;

  //Create corrresponding BSON Object. It is used to search for data, including condition (query), domain selection
 (selector), order rule (orderBy), application of index (index),
  //the amount of skipped records (0), the amount of returned record (-1:return all data).
  // query condition
  BSONObject query = new BasicBSONObject();
  BSONObject condition = new BasicBSONObject();
  condition.put("$gte", 0);
  condition.put("$lte", 9);
  query.put("Id", condition);
  // return fields
  BSONObject selector = new BasicBSONObject();
  selector.put("Id", null);
  selector.put("Age", null);
  // order by ASC(1)/DESC(-1)
  BSONObject orderBy = new BasicBSONObject();
  orderBy.put("Id", -1);
  if (index == null)
    dataCursor = cl.query(query, selector, orderBy, null, 0, -1);
    // or
    // dataCursor = cl.query("{'Id':{'$gte':0,'$lte':9}}", "{'Id':null,'Age':null}", "{'Id':-1}", null, 0, -1);
  else
    dataCursor = cl.query(query, selector, orderBy, index, 0, -1);
    // or
    // dataCursor = cl.query("{'Id':{'$gte':0,'$lte':9}}", "{'Id':null,'Age':null}", "{'Id':-1}", index, 0, -1);

  //Use Cursor, go through query result set
  // operate data by cursor
  while (dataCursor.hasNext()) {
  System.out.println(dataCursor.getNext());
  }
  // get count by match condition
  long count = cl.getCount(query);
  System.out.println("Get count by condition: " + query+ ", count = " + count);

  // see more details about query function in API document
    } catch (BaseException e) {
       e.printStackTrace();
    }
```

Delete

```
  Create corresponding BSON Object. It is used to specify deleting condition or the String corresponding to it.
  // create the delete condition
  BSONObject condition = new BasicBSONObject();
  condition.put("Id", 0 );
  try {
   cl.delete(condition);
   // if you want to delete all the data in current collection, you can use like: cl.delete(null)
   // or
   // cl.delete("{'Id':0}");
  } catch(BaseException e) {
   System.out.println("Failed to delete data, condition: " + condition);
   e.printStackTrace();
  }
  System.out.println("Delete data successfully");
```

Update

```
Create corresponding BSON Object. It is used to specify updating condition or the String corresponding to it.
// create match condition and modify rule
BSONObject matcher = new BasicBSONObject();
BSONObject modifier = new BasicBSONObject();
BSONObject m = new BasicBSONObject();

matcher.put("Id", 20);
m.put("Age", 80);
modifier.put("$set", m);

Update. If there is no matcher conditions, insert this record.
cl.upsert(matcher, modifier, null);
// query the updated data from db
DBCursor cursor = cl.query(matcher, null, null, null);
if(cursor.hasNext())
 System.out.println(cursor.getNext());
```

Disconnect

```
  // do not forget to disconnect from db
  sdb.disconnect();
```

SQL Statement

```
try {
   String csFullName = Constants.SQL_CS_NAME + "." + Constants.SQL_CL_NAME;
   String sql = "";

   Create CollectionSpace
   // create collectionspace
   sql = "create collectionspace " + Constants.SQL_CS_NAME;
   sdb.exec(sql);

   Create Collection
   // create table
   sql = "create table " + csFullName;
   sdb.execUpdate(sql);
   // insert data into table

   Insert data
   sql = "insert into " + csFullName + " (a,b,c)" + " values(1,\"John\",20)";
   sdb.execUpdate(sql);

   Search for data
   // select from table
   sql = "select * from " + csFullName;
   DBCursor cursor = sdb.exec(sql);
   while(cursor.hasNext())
     System.out.println(cursor.getNext());
   } catch (BaseException e) {
   e.printStackTrace();
   }
```

SQL to sequoiadb shell to java

The query in Sequoiadb is in the format of json(bson). The following table shows examples comparing sql statements, sequoiadb shell statements and sequoiadb java driver application statements.

SQL	sequoiadb shell	java Driver
insert into students(a,b) values(1,-1)	db.foo.bar.insert({a:1,b:-1})	bar.insert("{'a':1,'b':-1}")
select a,b from students	db.foo.bar.find(null,{a:"",b:""})	bar.query("", "{'a':'','b':''}", "", "")
select * from students	db.foo.bar.find()	bar.query()
select * from students where age=20	db.foo.bar.find({age:20})	bar.query("{'age':20}", "", "", "")
select * from students where age=20 order by name	db.foo.bar.find({age:20}).sort({name:1})	bar.query("{'age':20}", "", "{'name':1}", "")
select * from students where age>20 and age<30	db.foo.bar.find({age:{$gt:20,$lt:30}})	bar.query("{'age':{'$gt':20,'$lt':30}}", "", "", "")
create index testIndex on students(name)	db.foo.bar.createIndex("testIndex", {name:1},false)	bar.createIndex("testIndex", "{'name':1}", false, false)
select * from students limit 20 skip 10	db.foo.bar.find().limit(20).skip(10)	bar.query("", "", "", "", 10, 20)
select count(*) from students where age>20	db.foo.bar.find({age:{$gt:20}}).count()	bar.getCount("{'age':{'$gt':20}}")
update students set a=a+2 where b=-1	db.foo.bar.update({$set:{a:2}},{b:-1})	bar.update("{'b':-1}", "{'$inc':{'a':2}}", "")

SQL	sequoiadb shell	java Driver
delete from students where a=1	db.foo.bar.remove({a:1})	bar.delete("{'a':1}")

JAVA API

This part is related to java API document.

java API

C Driver

This section introduce C driver

C Client

C Client Compiling

C driver getting start

SQL to sequoiadb shell to C

C API

C Client

C client application is mainly to provide database connection,data CRUD and create or drop indexes,collectionspaces, collections.More refer to the online C API。

Handle

The handle of C client-driven is divided into two types.One is used to operate databases,the other is used to operate clusters.

- Database Operation Handle

  There are 3 degrees of data in SequoiaDB:

  1)Database

  2)Collection space

  3)Collection

  Logically, every collection space in SequoiaDB has no physical upper boundary. Every collection space is stored in a stand-alone file. So the amount of collection space depends on the size of disk and memory.

  A collection space contains at most 4096 collections. Every collection can contain more than one record. In a physical system, the size of a collection space is at most 256GB.

  Compared with relation database, we can regard record as line in relation database. We can regard collection as table in relation database.

  So in client, there are 3 Handles representing connection,collectionspace and collection. The other Handle represents cursor:

Sequoiadb	Database instance	It represents a stand-alone connection
CollectionSpace	CollectionSpace instance	It represents a stand-alone collection space
DBCollection	Collection instance	It represents a stand-alone collection
DBCursor	Cursor instance	It represents a result set of a query

  C client-driven application operate with different handles.For example,reading data will use Cursor Handle,then creating collectionspace uses database connection Handle.

  Note:

1. As for a connection, in its collection space, collections share one socket. So in mutiple-thread system, the system should guarantee that the socket can be used merely by one thread whenever.

2. Generally, it is not recommended to manipulate a connection instance and other instance, which it generates, with more than threads.

3、If each thread merely use its own connection instance and other instances it generates, it is thread-safe.

- Cluster Operation Handle

  There are two degrees of cluster operation in SequoiaDB:

  1)ReplicaSets

  2)Data Node

  Note:

  ReplicaSet contain three tpyes:Coord ReplicaSets,Catalog ReplicaSet,Data ReplicaSet.

  ReplicaSet instances and Data Node instances can be represented by the following two types of Handles.

sdbReplicaSetHandle	ReplicaSet Handle	It represents a stand-alone ReplicaSet
sdbReplicaNodeHandle	Data Node Handle	It represents a stand-alone Data Node

  Cluster-related operations need to use ReplicaSet Handle and Data Node Handle.

  The sdbReplicaSetHandle instants is to manage replicasets,including starting and stopping replicasets,getting the status,name-information and number-information of nodes in the replicaset.

  The sdbReplicaNodeHandle instants is to manage Data Nodes,including starting and stopping the specified data node,getting the specified data node instants and maste-slave data node instants and address-information of data node.

Error Information

Every function has a returned-value.If a function is successfully invoked,then return SDB_OK,else return error information.If the returned-value is less than 0,it is database error,if greater than 0,is system error.For the database errors,you can find the error code in the file of *ossErr.h*.For the system errors,error code can be finded in the file of */usr/include/errno.h*.It can also use *getErr()* method to get error information in Sequoiadb shell.

```
(nofile):0 uncaught exception: -34
> getErr(-34)
collection space not exist
```

In the client-driven application,you can find the error information in the file *ossErr.h* by the function returned-value *rc*.SDB_OK stands for operating successfully.

```
INT32 rc = SDB_OK ;
rc = sdbDisconnect( connection ) ;
```

C Client Building

First,press here to load C client-driven.

C Client development interface is in the directory "sequoiadb/client/C"(corresponding windows "sequoiadb₩client₩C"),including following files:

```
./include:
     base64.h
     bson/
     client.h
     core.h
     jstobs.h
     ossErr.h
     ossFeat.h
     ossTypes.h
```

```
    ./include/bson:
    bcon.h
    bson.h
    encoding.h

    ./lib:
    in Linux
    libsdbcpp.so
    in Windous
    sdbc.dll
    sdbc.exp
    sdbc.lib
```

Illustrating:in the above files,".h" and ".hpp" are head files,".so" are dynamic runtime files in Linux,".dll" and ".lib" are dynamic runtime files in Windows.

Compiling and linking in Linux

Please add the "include" directory to head directory when compiling programs,and add the "lib" directory to link diretory when linking,at the same time,add the "lib" directory to the environment variables "LD_LIBRARY_PATH".The following shows how to compile a C program in Linux:

```
cc testClient.c -o testClientC -I/path/C/include -L /path/C/lib -lsdbc
```

"-o" assign the output file,"-I" specify the head path,"-L" specify the runtime library path,"-l" assign the runtime library name.

When running the programs,users can specify the path of *LD_LIBRARY_PATH* as the path which contain *libsdbc.so* dynamic library.If the path is specified uncorrectly,run-time errors message will appear like this:

```
[sequoiadb@vmsvr1-rhel-x64 test]$ export | grep LD_LIBRARY_PATH
[sequoiadb@vmsvr1-rhel-x64 test]$ ldd testClientC
        linux-vdso.so.1 =>  (0x00007fff099ff000)
        libsdbc.so => not found
        libc.so.6 => /lib64/libc.so.6 (0x00000033c1200000)
        /lib64/ld-linux-x86-64.so.2 (0x00000033c0a00000)
[sequoiadb@vmsvr1-rhel-x64 test]$ ./testClientC
./testClientC: error while loading shared libraries: libsdbc.so: cannot open shared object file: No such file or
 directory
```

Using the command "ldd" can list the dynamic libraries which the executable program needed.In the above example,program "testClientC" need four dynamic libraries,including linux-vdso,libsdbc,libc and ld-linux-x86-64.Except the "libsdbc" library,the other two linux library are system library,"libc" is run-time library for C program.

As in the above example is not defined "LD_LIBRARY_PATH",so the program can't find "libsdbc" library,as shown above in italics tag:"libsdbc.so => not found".so set "LD_LIBRARY_PATH" :

```
$ export LD_LIBRARY_PATH=/home/sequoiadb/sequoiadb/client/C/lib
$ ldd testClientC
        linux-vdso.so.1 =>  (0x00007fffe45c1000)
        libsdbc.so => /home/sequoiadb/sequoiadb/client/C/lib/libsdbc.so (0x00007f00ff0b9000)
        libc.so.6 => /lib64/libc.so.6 (0x00000033c1200000)
        libm.so.6 => /lib64/libm.so.6 (0x00000033c1e00000)
        libdl.so.2 => /lib64/libdl.so.2 (0x00000033c0e00000)
        libpthread.so.0 => /lib64/libpthread.so.0 (0x00000033c1600000)
        libcrypto.so.10 => /usr/lib64/libcrypto.so.10 (0x00000033c7e00000)
        /lib64/ld-linux-x86-64.so.2 (0x00000033c0a00000)
        libz.so.1 => /lib64/libz.so.1 (0x00000033c1a00000)
$ ls /home/sequoiadb/sequoiadb/client/C/lib
libsdbc.so
[sequoiadb@vmsvr1-rhel-x64 test]$ ./testClientC
        name : 2         tom
        address : 3
                city : 2         Toronto
                provice : 2      Ontario
```

When referring to the libsdbc library,you can run the program successfully.

Compiling and linking in Windows

Please add the "include" directory to head directory when compiling programs,and add the "lib" directory to link diretory when linking，before running,copy the file "sdbc.dll" in the "lib" directory to the directory which generated the file ".exe".The following shows how to compile a C program: in Windows:

```
cl /Fo testClient.obj /c testClient.c /I <PATH>\sequoiadb\client\C\include /wd4047
link /OUT:testClient.exe /LIBPATH:<PATH>\sequoiadb\client\C\lib sdbc.lib testClient.obj
copy <PATH>\sequoiadb\client\C\lib\sdbc.dll .
```

"/Fo" represents creating-object file,"/c" means compiling but not link,"/I" expresses searching needed file in the specified directory,"/OUT" specifies the output file,"/LIBPATH" specified the environment-library path.

C Driver Introduction

This section introduce the approach to run SequiaDB in C application. First, you should make sure that you have install. Please refer to the installation page for more about installation information.

Edit client code

For the purpose of simpleness, the following sample is not complete code. Please get complete source code in the path "/sequoiadb/client/samples/C".

- Connecting Database

  There is a complete client file called "connect.c" to show how to connect to database in a C applicaiton. The file includes a head file called "client.h".

```
#include <stdio.h>
#include "client.h"

// Display Syntax Error
void displaySyntax ( CHAR *pCommand )
{
   printf ( "Syntax: %s<hostname> <servicename> <username> <password>"
            OSS_NEWLINE, pCommand ) ;
}

INT32 main ( INT32 argc, CHAR **argv )
{
   // define a connecion handle; use to connect to database
   sdbConnectionHandle connection    = 0 ;
   INT32 rc = SDB_OK ;

   // verify syntax
   if ( 5 != argc )
   {
      displaySyntax ( (CHAR*)argv[0] ) ;
      exit ( 0 ) ;
   }

   // read argument
   CHAR *pHostName = (CHAR*)argv[1] ;
   CHAR *pServiceName = (CHAR*)argv[2] ;
   CHAR *pUsr = (CHAR*)argv[3] ;
   CHAR *pPasswd = (CHAR*)argv[4] ;

   // connect to database
   rc = sdbConnect ( pHostName, pServiceName, pUsr, pPasswd, &connection ) ;
   if( rc!=SDB_OK )
   {
      printf("Fail to connet to database, rc = %d" OSS_NEWLINE, rc ) ;
      goto error ;
   }

done:
   // disconnect from database
   sdbDisconnect ( connection ) ;
   // release connection
   sdbReleaseConnection ( connection ) ;
   return 0 ;
error:
   goto done ;
   }
```

In Linux we can compile and link to dynamically linked librarie file called "libsdbc.so" as follow:

```
$gcc -o connect connect.c -I /path_to/C/include -lsdbc -L/path-to/client/C/lib
$ ./connect localhost 50000 "" ""
connect success!
```

Note:

This sample link to the port of 50000 of local database. It log in with null user name and null password. Users need to configurate parameters according to their own situation. But if database has created a user, it can connect to database with it and its password.

- Inserting Data

  SequoiaDB stores data in the format of BSON. BSON is a binary object similar to JSON. In order to store data, we should create a BSON object. The following sample insert the record {name:"Tom",age:24} into a collection:

```
// Firstly, a BSON object is created.
bson obj;
bson_init( &obj ) ;
bson_append_string( &obj, “name” , “tom” ) ;
bson_append_int( &obj, “age” , 24 ) ;
bson_finish( &obj ) ;
// Secondly, insert it into a collection.
dbInsert ( collection, &obj ) ;
```

- Query

  In a query, a cursor handle is used to store the result set of a query. You can get results by manipulating the cursor. This sample uses the *sdbNext* interface of cursor manipulation to get one record in the result set.

```
  // Define a cursor handle.
  dbCursorHandle cursor    = 0 ;
  ...
  // Search for all records and put the results in the cursor handle.
  sdbQuery(collection, NULL, NULL, NULL, NULL, 0, -1, &cursor ) ;
  // Show every record in cursor.
  bson_init(obj);
  while( !( rc=sdbNext( cursor, &obj ) ) )
  {
     bson_print( &obj ) ;
     bson_destroy(&obj) ;
     bson_init(&obj);
  }
  bson_destroy(obj) ;
```

- Index

  Here, we create an index in the collection specified by the collection handle *collection* . It is in ascending order on "name" and descending order on "age".

```
  #define INDEX_NAME "index"
  ...
  // Firstly we create a bson that contains the information of the index.
  bson_init( &obj ) ;
  bson_append_int( &obj, "name", 1 ) ;
  bson_append_int( &obj, "age", -1 ) ;
  bson_finish( &obj ) ;
  // Create an index with the BOSN object.
  sdbCreateIndex ( collection, &obj, INDEX_NAME, FALSE, FALSE ) ;
  bson_destroy ( &obj ) ;
```

- Updating data

  Here, we create an index in the collection specified by the collection handle *collection* This sample doesn't contain any matching condition, so it will update all the records in the collection.

```
  // Create a BSON object that contains updating rules
  bson_init( &rule ) ;
  bson_append_start_object ( &rule, "$set" ) ;
  bson_append_int ( &rule, "age", 19 ) ;
  bson_append_finish_object ( &rule ) ;
  bson_finish ( &rule ) ;
 // Print updating rule.
  bson_print( &rule ) ;
  // Update records.
  sdbUpdate( collection, &rule, NULL, NULL ) ;
```

```
bson_destroy(&rule);
```

Here,as no record matching conditions have specified,then this example will update all records in the collection which collection handle specified.

Cluster Operations

- ReplicaSet Operations

  ReplicaSet Operations contains creating ReplicaSets(sdbCreateReplicaSet)、getting ReplicaSet handles(sdbGetReplicaSet)、starting ReplicaSet(sdbStartReplicaSet) and stopping ReplicaSet(sdbStopReplicaSet).The following shows the ReplicaSet Operations,and real application should include error detecting ,etc.

```
// define a ReplicaSet Handle
   sdbReplicaSetHandle rg   = 0 ;
   ...
// create a catalog ReplicaSet at first
   sdbCreateReplicaCataSet ( connection, HOST_NAME, SERVICE_NAME,
                             CATALOG_SET_PATH , NULL ) ;
// sleep 5s to wait starting catalog ReplicaSet and electing master and slave.
   sleep( 5 ) ;
// create data ReplicaSet
   sdbCreateReplicaSet ( connection, REPLICA_SET_NAME, &rg ) ;

// create data nodes
   sdbCreateReplicaNode ( rg, HOST_NAME1, SERVICE_NAME1, DATABASE_PATH1, NULL ) ;
// start ReplicaSet
   sdbStartReplicaSet( rg ) ;
```

- Data Node Operations

  Data Node Operations contains create data node(sdbCreateReplicaNode)、get master data node(sdbGetReplicaNodeMaster)、get slave data node(sdbGetReplicaNodeSlave)、start data node (sdbStartReplicaNode)and stop data node(sdbStopReplicaNode),etc.The following shows the Data Node Operations,and real application should include error detecting ,etc.

```
   // define a data node handle
   sdbReplicaNodeHandle masternode   = 0 ;
   sdbReplicaNodeHandle slavenode    = 0 ;
   ...
   // sleep 10s to wait starting data ReplicaSet and electing master and slave
   sleep( 10 ) ;

   //get master data node
   sdbGetReplicaNodeMaster ( rg, &masternode ) ;
   //get slave data node
   sdbGetReplicaNodeSlave ( rg, &slavenode ) ;
```

SQL to sequoiadb shell to C

The query in Sequoiadb is in the format of json(bson). The following table shows examples comparing sql statements, sequoiadb shell statements and sequoiadb C driver application statements.

SQL	sequoiadb shell	java Driver
insert into students(a,b) values(1,-1)	db.foo.bar.insert({a:1,b:-1})	const char *r ="{a:1,b:-1}" ; // *JsonToBso* transforms a json string into a bson object jsonToBson (&obj, r) ; // *Collection* is the handle of collection *bar* sdbInsert (collection, &obj) ;
select a,b from students	db.foo.bar.find(null,{a:"",b:""})	const char *r ="{a:₩"₩",b:₩"₩"}" ; // *JsonToBso* transforms a json string into bson object jsonToBson (& select, r) ; // *Collection* is the handle of collection *bar*. *Cursor* is the handle that returns query results. sdbQuery (collection, NULL, &select, NULL, NULL, 0, -1, cursor) ;

SQL	sequoiadb shell	java Driver
select * from students	db.foo.bar.find()	// *Collection* is the handle of collection *bar*.*Cursor* is the handle that returns query results. sdbQuery (collection, NULL, NULL, NULL, NULL, 0, -1, cursor) ;
select * from students where age=20	db.foo.bar.find({age:20})	const char *r ="{age:20}"; // JsonToBso transforms a json string into a bson object. jsonToBson (&condition, r) ; // *Collection* is the handle of collection *bar*. *Cursor* is the handle that returns query results. sdbQuery (collection, & condition , NULL, NULL, NULL, 0, -1, cursor) ;
select * from students where age=20 order by name	db.foo.bar.find({age:20}).sort({name:1})	const char *r1 ="{age:20}" ; const char *r2 ="{name:1}" ; // *JsonToBso* transforms a json string into a bson object jsonToBson (& condition, r1) ; jsonToBson (&orderBy, r2) ; // *Collection* is the handle of collection *bar*. *Cursor* is the handle that returns query results. sdbQuery (collection, & condition , NULL, & orderBy , NULL, 0, -1, cursor) ;
select * from students where age>20 and age<30	db.foo.bar.find({age:{$gt:20,$lt:30}})	const char *r =”{age:{$gt:20,$lt:30}}” ; // *JsonToBso* transforms a json string into a bson object jsonToBson (&condition, r) ; // *Collection* is the handle of collection *bar*. *Cursor* is the handle that returns query results. sdbQuery (collection, & condition , NULL, NULL, NULL, 0, -1, cursor) ;
create index testIndex on students(name)	db.foo.bar.createIndex("testIndex", {name:1},false)	const char *r =”{name:1}” ; // *jsonToBso* transforms a json string into a bson object jsonToBson (&obj, r) ; // *Collection* is the handle of collection *bar* sdbCreateIndex (collection, &obj, “testIndex”, FALSE, FALSE)
select * from students limit 20 skip 10	db.foo.bar.find().limit(20).skip(10)	// *Collection* is the handle of collection *bar*. *Cursor* is the handle that returns query results. sdbQuery (collection, NULL, NULL, NULL,NULL, 10, 20, cursor) ;
select count(*) from students where age>20	db.foo.bar.find({age:{$gt:20}}).count()	const char *r =”{age:{$gt:20}}” ; // jsonToBson transforms a json string into a bson object jsonToBson (&condition, r) ; // *Collection* is the handle of collection *bar*. Count is the total amount of results. sdbGetCount (collection, & condition , &count);
update students set a=a+2 where b=-1	db.foo.bar.update({$set:{a:2}},{b:-1})	const char *r1 =”{$set:{a:2}}” ; const char *r2 =”{b:-1}” ; // jsonToBso transforms a json string into a bson object jsonToBson (&rule, r1) ; jsonToBson (&condition, r2) ; // *Collection* is the handle of collection *bar*. sdbUpdate (collection, &rule, &condition, NULL) ;
delete from students where a=1	db.foo.bar.remove({a:1})	const char *r =”{a:1}” ;

SQL	sequoiadb shell	java Driver
		// jsonToBso transforms a json string into a bson object jsonToBson (&condition, r) ; // *Collection* is the handle of collection *bar*. sdbDelete (collection, & condition , NULL) ;

C API

This part is related to C API document.

C API

C++ Driver

This section introduce C++ driver.

C++ Client

C++ Client Compiling

C++ Driver Getting Start

SQL to sequoiadb shell to C++

C++ API

C++ Client

C++ client-driven provide interfaces of database operations and cluster operations.Database operations include database connection、create or drop users、data CRUD、create or drop indexes and collectionspace and collections、get or reset snapshots,etc.cluster operations contain all kinds of managing replicaset and data node,such as starting or stopping replicasets、starting or stopping data nodes、getting master nodes、slave nodes or collection shardings,etc.

For more please refer to online C++ API。

C++ Class Instances
C++ Client-Driven has two class instances.One is used to operate database,the other is used to operate cluster.

- Database operation instance

  There are 3 degrees of data in SequoiaDB:

  1)Database

  2)Collection space

  3)Collection

  So in client, there are 3 Handles representing connection,collectionspace and collection. The other Handle represents cursor:

Sdb	Database class	it represents a stand-alone database connection
sdbCollectionSpace	CollectionSpace class	it represents a stand-alone collection space
sdbCollection	Collection class	it represents a stand-alone collection
sdbCursor	Cursor class	it represents a result set of a query

  C++ client-driven application operate with different instances.For example,reading data will use Cursor instance,then creating collectionspace uses database connection instance.
- Cluster Operation Instance

  There are two degrees of cluster operation in SequoiaDB:

1)ReplicaSets

2)Data Node

Note:

ReplicaSets contain three tpyes:Coord ReplicaSets,Catalog ReplicaSet,Data ReplicaSet.

ReplicaSet instances and Data Node instances can be represented by the following two types of classes.

sdbReplicaSet	ReplicaSet class	It represents a stand-alone ReplicaSet
sdbReplicaNode	data node clsaa	It represents a stand-alone Data Node

Cluster-related operations need to use ReplicaSet instance and Data Node instance.

The sdbReplicaSet instants is to manage replicasets,including starting and stopping replicasets,getting the status,name-information and number-information of nodes in the replicaset.

The sdbReplicaNode instants is to manage Data Nodes,including starting and stopping the specified data node,getting the specified data node instants and maste-slave data node instants and address-information of data node.

Error Information

Every function has a returned-value.If a function is successfully invoked,then return SDB_OK,else return error information.If the returned-value is less than 0,it is database error,if greater than 0,is system error.For the database errors,you can find the error code in the file of *ossErr.h*.For the system errors,error code can be finded in the file of */usr/include/errno.h*.It can also use *getErr()* method to get error information in Sequoiadb shell.

```
(nofile):0 uncaught exception: -34
> getErr(-34)
collection space not exist
```

In the client-driven application,you can find the error information in the file *ossErr.h* by the function returned-value *rc*.SDB_OK stands for operating successfully.

```
INT32 rc = SDB_OK ;
rc = sdbDisconnect( connection ) ;
```

C++ Client Building

First,press here to load C++ client-driven.

C++ Client development interface is in the directory "sequoiadb/client/CPP"(corresponding windows "sequoiadb₩client₩CPP"),including following files:

```
./include:
     base64.h
     bson/
     client.hpp
     core.h
     core.hpp
     jstobs.h
     ossErr.h
     ossFeat.h
     ossFeat.hpp
     ossTypes.h
     ossTypes.hpp

     ./include/bson:
     bcon.h
     bson.h
     encoding.h

     ./lib:
     in Linux
     libsdbcpp.so
     in Windous
     sdbcpp.dll
     sdbcpp.exp
     sdbcpp.lib
```

Illustrating:in the above files,".h" and ".hpp" are head files,".so" are dynamic runtime files in Linux,".dll" and ".lib" are dynamic runtime files in Windows.

Compiling and linking in Linux

Please add the "include" directory to head directory when compiling programs,and add the "lib" directory to link diretory when linking,at the same time,add the "lib" directory to the environment variables "LD_LIBRARY_PATH".The following shows how to compile a C++ program in Linux:

```
cc testClient.c -o testClientC -I/path/C/include -L /path/C/lib -lsdbc
```

"-o" assign the output file,"-I" specify the head path,"-L" specify the runtime library path,"-l" assign the runtime library name.

When running the programs,users can specify the path of *LD_LIBRARY_PATH* as the path which contain *libsdbcpp.so* dynamic library.If the path is specified uncorrectly,run-time errors message will appear like this:

```
[sequoiadb@vmsvr1-rhel-x64 test]$ export | grep LD_LIBRARY_PATH
[sequoiadb@vmsvr1-rhel-x64 test]$ ldd testClient
        linux-vdso.so.1 =>  (0x00007fff099ff000)
        libsdbcpp.so => not found
        libc.so.6 => /lib64/libc.so.6 (0x00000033c1200000)
        /lib64/ld-linux-x86-64.so.2 (0x00000033c0a00000)
[sequoiadb@vmsvr1-rhel-x64 test]$ ./testClientC
./testClient: error while loading shared libraries: libsdbcpp.so: cannot open shared object file: No such file or
 directory
```

Using the command "ldd" can list the dynamic libraries which the executable program needed.In the above example,program "testClientC" need four dynamic libraries,including linux-vdso,libsdbcpp,libc and ld-linux-x86-64.Except the "libsdbcpp" library,the other two linux library are system library,"libc" is run-time library for C program.

As in the above example is not defined "LD_LIBRARY_PATH",so the program can't find "libsdbcpp" library,as shown above in italics tag:"libsdbcpp.so => not found".so set "LD_LIBRARY_PATH" :

```
$ export LD_LIBRARY_PATH=/home/sequoiadb/sequoiadb/client/CPP/lib
$ ldd testClient
        linux-vdso.so.1 =>  (0x00007fffe45c1000)
        libsdbcpp.so => /home/sequoiadb/sequoiadb/client/CPP/lib/libsdbcpp.so (0x00007f00ff0b9000)
        libc.so.6 => /lib64/libc.so.6 (0x00000033c1200000)
        libm.so.6 => /lib64/libm.so.6 (0x00000033c1e00000)
        libdl.so.2 => /lib64/libdl.so.2 (0x00000033c0e00000)
        libpthread.so.0 => /lib64/libpthread.so.0 (0x00000033c1600000)
        libcrypto.so.10 => /usr/lib64/libcrypto.so.10 (0x00000033c7e00000)
        /lib64/ld-linux-x86-64.so.2 (0x00000033c0a00000)
        libz.so.1 => /lib64/libz.so.1 (0x00000033c1a00000)
$ ls /home/sequoiadb/sequoiadb/client/CPP/lib
libsdbcpp.so
[sequoiadb@vmsvr1-rhel-x64 test]$ ./testClient
        name : 2         tom
        address : 3
                city : 2         Toronto
                provice : 2      Ontario
```

When referring to the libsdbcpp library,you can run the program successfully.

Compiling and linking in Windows

Please add the "include" directory to head directory when compiling programs,and add the "lib" directory to link diretory when linking，before running,copy the file "sdbcpp.dll" in the "lib" directory to the directory which generated the file ".exe".The following shows how to compile a C++ program: in Windows:

```
cl /Fo testClient.obj /c testClient.cpp /I <PATH>\sequoiadb\client\CPP\include /wd4047
link /OUT:testClient.exe /LIBPATH:<PATH>\sequoiadb\client\CPP \lib sdbcpp.lib testClient.obj
copy <PATH>\sequoiadb\client\CPP\lib\sdbcpp.dll .
```

"/Fo" represents creating-object file,"/c" means compiling but not link,"/I" expresses searching needed file in the specified directory,"/OUT" specifies the output file,"/LIBPATH" specified the environment-library path.

C++ Driver Getting Start

This section introduce how to use C++ client-driven interfaces to run C++ application.for simple,the following examples are not full code,only play an exemplary role.You can get the full code in the directory "/sequoiadb/client/samples/CPP",for more,please refer to online C++ API

Database operations

- Connecting Database

  "connect.cpp" shows how to connect database.The file must contain the head file "client.hpp" and use namespace "sdbclient".

```
#include <stdio.h>
#include <iostream>
#include "client.hpp"

using namespace std ;
using namespace sdbclient ;

// Display Syntax Error
void displaySyntax ( CHAR *pCommand ) ;

INT32 main ( INT32 argc, CHAR **argv )
{
   // define a sdb object
   // use to connect to database
   sdb connection ;
   INT32 rc = SDB_OK ;

   // verify syntax
   if ( 5 != argc )
   {
      displaySyntax ( (CHAR*)argv[0] ) ;
      exit ( 0 ) ;
   }

   // read argument
   CHAR *pHostName    = (CHAR*)argv[1] ;
   UINT16 Port        =  atoi(argv[2]) ;
   CHAR *pUsr         = (CHAR*)argv[3] ;
   CHAR *pPasswd      = (CHAR*)argv[4] ;

   // connect to database
   rc = connection.connect ( pHostName, Port, pUsr, pPasswd ) ;
   if( rc!=SDB_OK )
   {
      cout<<"Fail to connet to database, rc = "<<rc<<endl ;
      goto error ;
   }
   else
      cout<<"connect success!"<<endl ;
done:
   // disconnect from database
   connection.disconnect () ;
   return 0 ;
error:
   goto done ;
}

// Display Syntax Error
void displaySyntax ( CHAR *pCommand )
{
   cout<<"Syntax:"<<pCommand<<" <host><name><servicename>\
 <username><password>"<<endl ;
 }
```

  In Linux,you can compile and link the dynamic library "libsdbc.so" like this:

```
$ gcc -o connect connect.cpp -I <PATH>/sequoiadb/client/CPP/include -lsdbcpp -L <PATH>/sequoiadb/client/CPP/lib
Execution results:
$ ./connect localhost 50000 "" ""
connect success!
```

In Windows,compiling and linking the dynamic library "sdbcpp.lib" and "sdbcpp.dll" like this:

```
> cl /Fo connect.obj /c connect.cpp /I <PATH>\sequoiadb\client\CPP\include /wd4047
> link /OUT:connect.exe /LIBPATH:<PATH>\sequoiadb\client\CPP \lib sdbcpp.lib connect.obj
> copy <PATH>\sequoiadb\client\CPP\lib\sdbcpp.dll .
Execution results:
> connect.exe localhost 50000 "" ""
> connect success!
```

Note:

This sample connects to the port "50000" of local database,usename and password are null,users can configure params according to needs.For example,when the user have created usename and password,you must use the correct usename and password to connect the database.

- Inserting Data

```
// Firstly , a BSON object is created
bson obj;
bson_init( &obj ) ;
bson_append_string( &obj,  “name” ,  “tom”  ) ;
bson_append_int( &obj,  “age” , 24 ) ;
bson_finish( &obj ) ;
// Then,insert it into a collection
collection.insert ( &obj ) ;
bson_destroy( &obj ) ;
```

obj is inputed param and inserted data.

- Query

```
// Define a cursor object
   sdbCursor cursor ;
   ...
   // Search for all records and put the results in the cursor object
   collection.query ( cursor, NULL, NULL, NULL, NULL, 0, -1 ) ;
   // Show every record in cursor
   bson_init(&obj);
   while( !( rc=cursor.next( obj ) ) )
   {
      bson_print( &obj ) ;
      bson_destroy(&obj) ;
      bson_init(&obj);
   }
   bson_destroy(&obj) ;
```

In a query, a cursor object is used to store the result set of a query . You can get results by manipulating the cursor. This sample uses the *next* interface of cursor manipulation to get one record in the result set. This sample does not set querying conditions,filtering conditions and ordering,only use default index.

- Index

```
   #define INDEX_NAME "index"
   ...
   // Firstly we create a bson that contains the information of the index.
   bson_init( &obj ) ;
   bson_append_int( &obj, "name", 1 ) ;
   bson_append_int( &obj, "age", -1 ) ;
   bson_finish( &obj ) ;
   // Create an index with the BOSN object.
   collection.createIndex ( &obj, INDEX_NAME, FALSE, FALSE ) ;
   bson_destroy ( &obj ) ;
```

Here, we create an index in the collection specified by the collection object *collection* . It is in ascending order on "name" and descending order on "age"

- Update

```
   // Create a BSON object that contains updating rules
   bson_init( &rule ) ;
   bson_append_start_object ( &rule, "$set" ) ;
   bson_append_int ( &rule, "age", 19 ) ;
   bson_append_finish_object ( &rule ) ;
   bson_finish ( &rule ) ;
   //  Print updating rule.
   bson_print( &rule ) ;
```

```
// Update records.
collection.update( &rule, NULL, NULL ) ;
bson_destroy(&rule);
```

Here,as no record matching conditions have specified,then this example will update all records in the collection which collection handle specified.

Cluster Operations

- ReplicaSet Operations

  ReplicaSet Operations contains creating ReplicaSets(sdbCreateReplicaSet)、getting ReplicaSet handles(sdbGetReplicaSet)、starting ReplicaSet(sdbStartReplicaSet) and stopping ReplicaSet(sdbStopReplicaSet).The following shows the ReplicaSet Operations,and real application should include error detecting ,etc.

```
// define a ReplicaSet Instance
sdbReplicaSet rg  ;
// define a null map object,it means that no more configuration information when create data nodes
map<string,string> config ;
...
// create a catalog ReplicaSet at first
connection.createReplicaCataSet ( HOST_NAME, SERVICE_NAME,
                    CATALOG_SET_PATH , NULL ) ;
// sleep 5s to wait starting catalog ReplicaSet and electing master and slave.
sleep( 5 ) ;
// create data ReplicaSet
connection.createReplicaSet ( REPLICA_SET_NAME, rg ) ;

// create data nodes
rg.createNode ( HOST_NAME1, SERVICE_NAME1, DATABASE_PATH1, config ) ;
...
// start ReplicaSet
rg.start () ;
```

- Data Node Operations

  Data Node Operations contains create data node(sdbCreateReplicaNode)、get master data node(sdbGetReplicaNodeMaster)、get slave data node(sdbGetReplicaNodeSlave)、start data node (sdbStartReplicaNode)and stop data node(sdbStopReplicaNode),etc.The following shows the Data Node Operations,and real application should include error detecting ,etc.

```
// define a data node instance
sdbReplicaNode masternode ;
sdbReplicaNode slavenode ;
...
// sleep 15s to wait starting data ReplicaSet and electing master and slave
sleep( 15 ) ;

// get master data node
rg.getMaster( masternode ) ;
//get slave data node
rg.getSlave( slavenode ) ;
```

SQL to sequoiadb shell to C++

The query in Sequoiadb is in the format of json(bson). The following table shows examples comparing sql statements, sequoiadb shell statements and sequoiadb C++ driver application statements.

SQL	sequoiadb shell	C++ Driver
insert into students(a,b) values(1,-1)	db.foo.bar.insert({a:1,b:-1})	sdbCollection collection ; const char *r =”{a:1,b:-1}” ; jsonToBson (&obj, r) ; collection.insert(&obj) ;
select a,b from students	db.foo.bar.find(null,{a:"",b:""})	sdbCollection collection ; sdbCursor cursor ; const char *r =”{a:₩”₩”,b:₩”₩”}” ; jsonToBson (& select, r) ; collection .query(cursor, NULL, &select, NULL, NULL, 0, -1) ;
select * from students	db.foo.bar.find()	sdbCollection collection ;

SQL	sequoiadb shell	C++ Driver
		sdbCursor cursor ; collection .query (cursor, NULL, NULL, NULL, NULL, 0, -1) ;
select * from students where age=20	db.foo.bar.find({age:20})	sdbCollection collection ; sdbCursor cursor ; const char *r =”{age:20}” ; jsonToBson (&condition, r) ; collection .query (cursor, & condition , NULL, NULL, NULL, 0, -1) ;
select * from students where age=20 order by name	db.foo.bar.find({age:20}).sort({name:1})	sdbCollection collection ; sdbCursor cursor ; const char *r1 =”{age:20}” ; const char *r2 =”{name:1}” ; jsonToBson (& condition, r1) ; jsonToBson (&orderBy, r2) ; collection .query (cursor, & condition , NULL, & orderBy , NULL, 0, -1) ;
select * from students where age>20 and age<30	db.foo.bar.find({age:{$gt:20,$lt:30}})	sdbCollection collection ; sdbCursor cursor ; const char *r =”{age:{$gt:20,$lt:30}}” ; jsonToBson (&condition, r) ; collection .query (cursor, & condition , NULL, NULL, NULL, 0, -1) ;
create index testIndex on students(name)	db.foo.bar.createIndex("testIndex", {name:1},false)	sdbCollection collection ; const char *r =”{name:1}” ; jsonToBson (&obj, r) ; collection.createIndex (&obj, "testIndex", FALSE, FALSE)
select * from students limit 20 skip 10	db.foo.bar.find().limit(20).skip(10)	sdbCollection collection ; sdbCursor cursor ; collection .query (cursor, NULL, NULL, NULL, NULL, 10, 20) ;
select count(*) from students where age>20	db.foo.bar.find({age:{$gt:20}}).count()	sdbCollection collection ; SINT64 count = 0 ; const char *r =”{age:{$gt:20}}” ; jsonToBson (&condition, r) ; collection.getCount (collection, & condition , &count);
update students set a=a+2 where b=-1	db.foo.bar.update({$set:{a:2}},{b:-1})	sdbCollection collection ; const char *r1 =”{$set:{a:2}}” ; const char *r2 =”{b:-1}” ; jsonToBson (&rule, r1) ; jsonToBson (&condition, r2) ; collection.update (&rule, &condition, NULL) ;
delete from students where a=1	db.foo.bar.remove({a:1})	sdbCollection collection ; const char *r =”{a:1}” ; jsonToBson (&condition, r) ; collection.del (& condition , NULL) ;

C++ API

This part is related to the C++ API document.

C++ API

PHP Driiver

This section introduce PHP driver.

Built PHP Environment

Windows

Firstly,install apache and PHP programs.

Assuming:

1.the apache installed path is :"C:₩Apache"

2.the PHP installed path is:"C:₩php"

Steps:

- Add the PHP path in the system environment variable,rename the file "php.ini-recommended" in the root directory of PHP as "php.ini".
- Modify "php.ini"

```
extension_dir = "C:\php\ext"
```

- Modify C:₩Apache₩conf₩httpd.conf,add the following information at the end of file

```
LoadModule php5_module "C:\php\php5apache2_2.dll"
AddType application/x-httpd-php .php
```

- Restart Apache Service

Ubuntu Linux

- Install Apache

```
sudo apt-get install apache2
```

- Install PHP

```
sudo apt-get install php5
sudo apt-get install libapache2-mod-php5
```

- Modify /etc/apache2/apache2.conf ,add the following information at the end of file

```
AddType application/x-httpd-php .php
```

- Restart apache

```
sudo /etc/init.d/apache2 restart
```

- Install php5-dev

```
sudo apt-get install php5-dev
```

Centos Linux

- Install Apacha and PHP

```
yum -y install httpd php httpd-manual mod_perl php-gd php-xml php-mbstring php-ldap php-pear php-xmlrpc
```

- Configure startup service

```
/sbin/chkconfig httpd on
```

- Modify configuration file

```
vi /ect/httpd/conf/httpd.conf
```

find > #AddType application/x-tar .tgz in this line add the following information > AddType application/x-httpd-php .php

- Start Apacha

```
/etc/init.d/httpd start
```

PHP Client

PHP driver provide manipulations including database connection, data inserting, deleting, altering and querying, index creating and deleting, collection creating and deleting, collection space creating and deleting, etc.

Object

Users donot need care how the database work with PHP driver,only create a sequoiadb object can operate the database.

Error Information

1.The value of most functions returned are Boolean type.

2.If a function is successfully invocated,then return "true"(or 1),otherwise,return "false"(or 0).

3.If the users want to kown the detail error information,can call the method "getErr()",if no error,it will output "No error message".

SQL to sequoiadb shell to php

The query in Sequoiadb is in the format of json(bson). The following table shows examples comparing sql statements, sequoiadb shell statements and sequoiadb PHP driver application statements.

SQL	sequoiadb shell	php Driver
insert into students(a,b) values(1,-1)	db.foo.bar.insert({a:1,b:-1})	$bar->insert("{a:1,b:-1}")
select a,b from students	db.foo.bar.find(null,{a:"",b:""})	$bar->find(NULL,'{a:"",b:""}
select * from students	db.foo.bar.find()	$bar->find()
select * from students where age=20	db.foo.bar.find({age:20})	$bar->find("{age:20}")
select * from students where age=20 order by name	db.foo.bar.find({age:20}).sort({name:1})	$bar->find("{age:20}",NULL,"{name:1}")
select * from students where age>20 and age<30	db.foo.bar.find({age:{$gt:20,$lt:30}})	$bar->find("{age:{$gt:20,$lt:30}}")
create index testIndex on students(name)	db.foo.bar.createIndex("testIndex", {name:1},false)	$bar->createIndex("{name:1}","testIndex",false)
select * from students limit 20 skip 10	db.foo.bar.find().limit(20).skip(10)	$bar->find(NULL,NULL,NULL,NULL,10,20)
select count(*) from students where age>20	db.foo.bar.find({age:{$gt:20}}).count()	$bar->count("{age:{$gt:20}}")
update students set a=a+2 where b=-1	db.foo.bar.update({$set:{a:2}},{b:-1})	$bar->update("{$set:{a:2}}","{b:-1}")
delete from students where a=1	db.foo.bar.remove({a:1})	$bar->remove("{a:1}")

PHP API

This part is related to the PHP API document.

PHP API

SequoiaDB Cluster Manager

sdbcm overview

SequoiaDB Cluster Manager is a daemon process，in Windows, it disposes the system background in the way of service. All of the Cluster Manager operations must have the participation of sdbcm in Sequoiadb, at present, each physical machine can only start a sdbcm process, responsible for executing the remote commands and Monitoring the local Sequoiadb 。sdbcm mainly has two big functions：

- Remote starting、closing、creating and modifying nodes: Through Sequoiadb client or driving to connect to the database, then we can execute the operations of starting、closing、creating and modifying nodes, those operations send remote commands to sdbcm that the physical machine specified, and obtaining performance results.

- local monitoring：for the starting nodes by sdbcm,will maintain a table about nodes, it save all the service name of the local nodes and starting information, such as starting time、running status etc. If a node terminate in normal case, such as the process is forced to terminate、engine quit unexpectedly etc., then sdbcm will try restart this node.

sdbcm operator

- conf files

  in the database installtion directory, hava a conf file named *sdbcm.conf*, this file gives the conf information of starting sdbcm, as follows:

param	desciption	example
defaultPort	the default monitoring port of sdbcm	defaultPort=50010
<hostname>_Port	monitoring port of sdbcm in physical host.if it can't find the corresponding param of host in the conf file,sdbcm will start with defaultPort; if defaultPort is also not exist ,then sdbcm will start the default port 50010	<hostname>_Port=50010
RestartCount	number of restarts. That is define the max number of restarting nodes for sdbcm. If the param doesn't exist,then default set to -1,that is constantly restart.	RestartCount=5
RestartInterval	restart interval, that is difine the max restart interval of sdbcm,the unit is minutes. This param combine with RestartCount define the max number of restarting nodes for sdbcm in the restart interval, when beyond,no longer restart. If the param doesn't exist,then default set to 0, that is doesn'tconsider the restart interval .	RestartInterval=0

- start sdbcm

  running *sdbcmart* commands can start sdbcm。
- close sdbcm

  running *sdbcmtop* command can close sdbcm。

Reference

It introduces javascript methods and operator, vocabulary, exception information and name limits in sequoiadb database.

Sequoiadb javascript Method List

Sequoiadb is a kind of file-based non-relation database. It usesJSON data module.

- If users want to match elements nested in an object. They can use "." ("0x2E" in ASCII) to connect parent element and child element. For example, in this object:

  ```
  { name:"tom",address:{ street:{ street1:"1024 Wall Street",street2:"University Drive"},zipcode: 100000 } }
  ```

  If users want to match the element "street" in the object "street" in the object "address", they can get it in this way:

  ```
  { "address.street.street1":{ $exists : 1 } }
  ```

- If there is an array in an object, users can get the 1st, 2nd, and 3rd element by "0", "1" and "2" respectively. For example, this object cantains am array:

  ```
  { name: "tom", phone: [ "13175824", "1528345"] }
  ```

  Users can get the 1st element in the array in this way:

  ```
  { "phone.0 ":"13175824" }
  ```

Database Operation Methods

Method Name	Description	Sample
db.createCS()	Create a collectin space	db.createCS("foo")
db.dropCS()	Delete a collection space	db.dropCS("foo")
db.getCS()	Get the quotation of a collection space	db.getCS("foo")
db.createRG()	Create a replset	db.createRG("group")
db.getRG()	Get the quotation of a replset	db.getRG("group")
db.list()	Enumerate lists	db.list()
db.listCollectionSpaces()	Enumerate collection spaces	db.listCollectionSpaces()
db.listCollections()	Enumerate collections	db.listCollections()
db.listReplicaGroups()	Enumerate replsets	db.listReplicaGroups()
db.snapshot()	Enumerate snapshots	db.snapshot(0)
db.resetSnapshot()	Reset a snapshot	db.resetSnapshot(0)
db.startRG()	Start a replset	db.startRG("groupName")
db.createUsr()	Create a database user	db.createUsr("root","admin")
db.dropUsr()	Delete a databse user	db.dropUsr("root","admin")

Collection Space Method

Method Name	Description	Sample
db.collectionspace.createCL()	Create a collection	db.foo.createCL("bar")
db.collectionspace.dropCL()	Delete a collection	db.foo.dropCL("bar")
db.collectionspace.getCL()	Get the quotation of a collection	db.foo.getCL("bar")

Collection Method

Method Name	Description	Sample
db.collectionspace.collection.insert()	Insert a record into a collection	db.foo.bar.insert({name:"Tom",age:20,phone:[123,345]})
db.collectionspace.collection.count()	Count the records in a collection	db.foo.bar.count({age:{$gt:20}})
db.collectionspace.collection.find()	Find records	db.foo.bar.find()
db.collectionspace.collection.remove()	Delete records	db.foo.bar.remove()
db.collectionspace.collection.createIndex()	Create an index	db.foo.bar.createIndex("myIndex",{age:-1},false)
db.collectionspace.collection.dropIndex()	Delete an index	db.foo.bar.dropIndex("myIndex")
db.collectionspace.collection.getIndex()	Get the quotation of an index	db.foo.bar.getIndex("myIndex")
db.collectionspace.collection.listIndexes()	Enumerate indexes	db.foo.bar.listIndexes()
db.collectionspace.collection.update()	Update records	db.foo.bar.update({$set:{age:25}},{age:{$lte:20}},{"":"myIndex"})
db.collectionspace.collection.upset()	Update records	db.foo.bar.update({$set:{age:25}},{a:1},{"":"myIndex"})
db.collectionspace.collection.split()	Split a collection	db.foo.bar.split("group1","group2",{age:20})

Cluster Method

Method Name	Description	Sample
rg.start()	Start replset	rg.start()
rg.getDetail()	Get information of replset	rg.getDetail()
rg.createNode()	Create node	rg.createNode(<hostname>,<port>,<path>)
rg.getMaster()	Get information of master node	rg.getMaster()
rg.getSlave()	Get information of slave node	rg.getSlave()
rg.getNode()	Get information of all nodes	rg.getNode()
rg.stop()	Stop a replset	rg.stop()

Node Method

Method Name	Description	Sample
node.start()	Start a node	node.start()
node.connect()	Get the hostname and port of a connected node	node.connect()
node.getHostName()	Get the host name of a node	node.getHostName()
node.getServiceName()	Get the server name of a node	node.getServiceName()
node.getNodeDetail()	Get information of a node	node.getNodeDetail()
node.stop()	Stop a node	node.stop()

Cursor Method

Method Name	Description	Sample
cursor.sort()	Sort result set according to the specified field	db.foo.bar.find().sort({age:1})
cursor.hint()	Go through result set according to the specified index	db.foo.bar.find().hint({"":"Index"})
cursor.limit()	Specify the amount of records to be returned in the result set	db.foo.bar.find().limit(10)
cursor.skip()	Specify the first record to be returned in the result set	db.foo.bar.find().skip(10)
cursor.current()	Return the record that the current cursor currently pionts to	db.foo.bar.find().current()

Method Name	Description	Sample
cursor.deleteCurrent()	Delete the record that the current cursor currently pionts to	db.foo.bar.find().deleteCurrent()
cursor.next()	Return the next record in the cursor	db.foo.bar.find().next()
cursor.updateCurrent()	Update the record that the current cursor currently pionts to	db.foo.bar.find.()updateCurrent({$set:{age:25}})
cursor.size()	Return the distance from the record that the current cusor pionts to to the final record	db.foo.bar.find().size()
cursor.toArray()	Return result set in the format of array	db.foo.bar.find.toArray()

db.createCS()

db.createCS(<name>,[pageSize])

Create a collection space in a database instance.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
name	string	Collection space name. Collection space names should be unique to each other in a database object.	yes
pageSize	int(32)	Size of data page. The default value is 4096B.	no

Format

Users can create a collection space with 2 parameters: name and pageSize.The parameter "name" is in the type of string. The parameter "pageSize" is in the type of 32-bit integer

```
("<collection space name>",[pagesize])
```

Note:

- The parameter "name" should not be null string. It should not contain "." or "$". The length of it should not be greater than 127B.
- Collection space names should be unique to each other in a database object.
- When creating a collection space, users can specify the size of data page. It is unchangable afterward. The default value of it is 4096B.

Sample

- Create a collection space named "foo" without specifying the size of data page.

```
db.createCS("foo")
```

- Create a collection space named "foo" and specify the size of data page as 1024.

```
db.createCS("foo",1024)
```

db.dropCS()

db.dropCS(<name>)

Delete a specified collection space in a database instance.

Parameter Descrioption

Parameter name	Parameter type	Description	Not null
name	string	Collection space name. All the collection sapces in a database instance is unique to each other.	yes

Format

The parameter "name" should be specified in the method "dropCS()". The type of collection space name should be string. It should be ensured that the collcection space name exists in the database instance, or operation exception will occur.

```
("<Collection space name>")
```

Note:

- The value of "name" should not be null string. It should not contain "." or "$". And the length of it should not be greater than 127B. If the collection space name is invalid , the operation will fail.
- It should be ensured that the collection space name exists, or exception will occur.

Sample

- Supposing that a collection space called "foo" exists, the following command will delete it.

```
db.dropCS("foo")
```

db.getCS()

db.getCS(<name>)

Return the quotation of a specified collection space.

Parameter description

Parameter name	Parameter type	Description	Not null
name	string	Collection space name. Collection space names are unique to each other in a database instance.	yes

Format

The definition formant of *getCS()* method only have *name* param,it is a string type.

```
("<collectionspace name>")
```

Note:

- the value of param *name* can't be null 、 contain dot(.) or '$',the length of it must be not more than 127B.
- The collectionspace must be existed in the database.

Sample

- Supposing that the collection space "foo" exists, it will return the quotation of it.

```
db.getCS( “foo” )
```

db.createRG()

db.createRG(<groupname>)

Create a replset. When creating a replset, the system will automatically generate a "GroupID" for a replset.

Parameter Description

Parameter name	Parameter type	Description	Not null
groupname	string	Replset name. All replset names are unique to each other in a database object.	Yes

Format

The method "createRG()" contains the parameter "groupname", which is in the type od string. It should not be null.

```
(<"Replset name">)
```

Note:

- Replset is not null string. It should not contains "." or "$". The length of it should not be greater than 127B.

Sample

- Create a replset named "group".

```
db.createRG( “group” )
```

db.getRG()

db.getRG(parameters)

Return the quotation of a replset.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
name	string	Replset name. Replset names are unique to each other in a database object.	Either "name"or "id" is not null
id	int	Replset id. The system will automatically generate a replset id.	Either "name"or "id" is not null

Format

The method "getRG()" contains "name" or "id". The type of "name" is string. The type of "id" is "int". It the replset specified by "name" or "id" doesn't exist, operation exception will occur.

```
("<Replset name>"|<id>)
```

Note:

- The value of "name" should not contain null string, "." or "$". The length of it should not be greater than 127B.

Sample

- Specify the value of "name", the returnthe quotation of replset "group".

```
db.getRG("group")
```

- Specify the value of "id", the returnthe quotation of replset "group".

```
db.getRG("1000")
```

db.list()

db.list(<listType>,[con],[sel],[sort])

Enumerate list. List is a lightweight command to get the current system status.Click here to learn more about list.

Parameter Description

Parameter Name	Parameter Type	Description	Not Null
listType	Enumeration	List type. The value of it is in the range from 0 to 7.	yes
con	json object	Selecting condition. It just returns the records that match conditions in "con". When it is null, return all records.	no
sel	json object	Select fields to be returned. When it is null, return all records.	no
sort	json object	Sort records. "1" means ascending order. "-1" means descending order.	no

Format

The method "list()" contians 4 parameters: listType, con, sel and sort. The parameter "listType" is in the type of enumeration. Other parameters are in the type of json object. The format is as follow:

```
{"listType":"<List type>",["con":"{"field name 1":{"operator 1":"value 1"},"field name 2":{"operator 2":"value
2"}...}"],["sel":"{"field name 1":"","field name 2":"",...}"],["sort":"{"field name 1":-1|1,"field name
2":1|-1,...}"]}
```

Note:

- The value of "listType" is in the range from 0 to 7.
- The value of "sel" is in the type of json object. The value of each field in it is always null string. If it is specified as "{"field name 1": "value 1", "field name 2":"value 2",...}", the values of existing fields are invalid. As for fields that doesn't exist, it will return "{"field name 1": "value 1", "field name 2":"value 2",...}".
- If a value of a field is in the type of array, users can use "." to invoke it and add double quotation.

Sample

- Specify the value of "listType" as 0 :

```
db.list(0)
```

Return :

```
    {
       "SessionID": 4,
       "Contexts": [ 0 ]
    } ...
```

- Specify the value of "listType" as 6：

```
db.list(6)
```

Return：

```
    {
        "Name": "foo",
        "ID": 4094,
        "Logical ID": 1,
        "PageSize": 4096,
        "Sequence": 1,
        "NumCollections": 1,
        "CollectionHWM": 3,
        "Size": 172032000
      }
```

- `db.list(6, {"Logical ID": {$gt:1}}, {Name:"space", ID:2}, {Name: 1})`

  Return records that contains the value of "Logical" greater than 1. Each record just return the 2 fields: Name and ID. The values of "Name" in these records are in ascending order.

```
      {
        "ID": 4091,
        "Name": "foo"
      }
      {
        "ID": 4093,
        "Name": "name"
       }...
```

db.listCollectionSpaces()

db.listCollectionSpaces()

Emurate all the collection spaces in the database.

Sample

- `db.listCollectionSpace()`

  Returns:

```
      {
        "Name":"foo"
      }...
```

db.listCollections()

db.listCollections()

Enumerate collection. It will show information of all collections in a collection space.

Sample

- `db.listCollections()`

  Return：

```
    {
        "Name": "foo.bar",
        "Details": [
          {
            "ID": 0,
            "Logical ID": 1,
            "Sequence": 1,
            "Indexes": 2,
            "Status": "Normal"
           }
        ]
      }
```

db.listReplicaGroups()

db.listReplicaGroups()

List all replsets:

Sample

- Return information of all replsets:

```
> db.listReplicaGroups()
Return: {
  "Group": [
    {
      "dbpath": "/opt/sequoiadb/data/60000",
      "HostName": "vmsvr2-suse-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "60000"
        },
        {
          "Type": 1,
          "Name": "60001"
        },
        {
          "Type": 2,
          "Name": "60002"
        },
        {
          "Type": 3,
          "Name": "60003"
        }
      ],
      "NodeID": 1000
    },
    {
      "dbpath": "/opt/sequoiadb/data/60010",
      "HostName": "vmsvr2-suse-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "60010"
        },
        {
          "Type": 1,
          "Name": "60011"
        },
        {
          "Type": 2,
          "Name": "60012"
        },
        {
          "Type": 3,
          "Name": "60013"
        }
      ],
      "NodeID": 1001
    }
  ],
  "GroupID": 1001,
  "GroupName": "group",
  "PrimaryNode": 1001,
  "Role": 0,
  "Status": 1,
  "Version": 5,
  "_id": {
    "$oid": "517b2fc33d7e6f820fc0eb57"
  }
}
```

This replset has 2 nodes: 60000 and 60010. Node 60010 is the master node.Click here to know more about details of replset

db.snapshot()

db.snapshot(<snapType>,[con],[sel],[sort])

It lists snapshots. Snapshot is a kind of command used to get the current status of system.Click here to learn more about snapshot.

Parameter Description

Parameter name	Parameter type	Description	Not null
snapType	List	Snapshot type, the value of snapshot type is in the range from 0 to 7.	Yes
con	json object	Selectable condition. It just returns records that matches the value of "con". If it is null, it returns all records.	No
sel	json object	Select returned field name. If it is null, return all the fields	No
sort	json object	Sort records according to specified field. "1" means ascending order. "-1" means descending order.	No

Format

The defination of the method "snapshot()" contains four fields: listType、con、sel、sort. The type of "listType" is enumeration. Other fields are json objects. The format is as follow:

```
{"snapType":"<snapshot type>",["con":"{"Field name 1":{"operation 1":"value 1"},"Field name 2":{"Operation 2":"value 2"}...}"],
["sel":"{"Field name 1":"","Field name 2":"",...}"],["sort":"{Order:-1|1}"]}
```

Note:

- The value of snapType is in the range from 0 to 7.
- The parameter "sel" is a json object. The value of field name is specified as null string. It is defined as: {"Field name 1":"value 1","Field name 2":"value 2",...}.If the records contain the specified field names, the values are invalid. If the records don't contain specified fields, it returns "{"Field name 1":"value 1", "Field name 2":"value 2"}".
- If the value of a field is in the type of array. We invoke the elements in it through "." and double quotations.

Sample

- Set the value of "snapType" as 0:

```
db.snapshot(0)
```

Returns context snapshot:

```
{
        "SessionID": 3,
        "Contexts": [
          {
            "ContextID": 1,
            "DataRead": 0,
            "IndexRead": 0,
            "QueryTimeSpent": 0,
            "StartTimestamp": "2013-04-08-17.17.02.135620"
          }
        ]
      }
```

- ```
  db.snapshot(0,{"Contexts.0.ContextID":{$gte:0}},{SessiomID:"","Contexts.0.StartTimestamp":""},{SessionID: 1})
  ```

  Return records that contain the value of "Contexts.0.ContextID" equal to 0, and just return values in the 2 field, SessiomID and Contexts.StartTimestamp. The returned records are in ascending order:

  ```
  {
          "Contexts": [
           {
             "StartTimestamp": "2013-04-09-09.31.03.270965"
           }
          ],
          "SessionID": 5
        }
  ```

db.resetSnapshot()

db.resetSnapshot([cond])

Reset snapshot.

Parameter description

Parameter name	Parameter type	Description	Not null
con	json object	Selecting condition. It merely resets snapshot records that match the conditions in "con". If it is null, it will reset all the snapshot records.]	no

Format

The method "resetSnapshot()" contains the parameter "con". It is a json object.

```
{["con":"{"Field name 1":{"match 1":"value 1"},"Field name 2":{"match 2":"value 2"}...}"]}
```

Sample

- Reset snapshots that contain the value of "SessionID" greater than 1.

  ```
  db.resetSnapshot({SessionID:{$gt:1}})
  ```

db.startRG()

db.startRG(<name>)

Start a specified replset. Only after it is started can a node be created on it. This method is equal to "rg.start()".

Parameter Description

Parameter Name	Parameter Type	Description	Not Null
name	sring	The name of a replset	yes

Format

The method "db.startRG()" contains a parameter called "name". The type of it is string. It is the name of the replset which is about to be started.

```
("<Replset name>")
```

Note:

Exception will occur when the specified replset doesn't exist or it has been started.
```

Sample

- The following command starts a replset called "group":

```
db.startRG("group")
```

db.createUsr()

db.createUsr(<name>,<password>)

Create a databse user with a user name and a password.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
name	string	User name	yes
password	string	Password	yes

Format

The method "createUsr()" contains 2 parameters: name and password. Both of them are in the type of string. And they can be null string.

```
("<user name>","<password>")
```

Note:

Only when users log in database with correct user name and password can they enter database and make some operations. The comamd to log in database is as follow:

```
db = new Sdb("<hostname>","<port>","<name>","<password>")
```

Sample

- Create a user with the user name "root" and the password "admin":

```
db.createUsr("root","admin")
```

db.dropUsr()

db.dropUsr(<name>,<password>)

Delete an existing user and the corresponding password in the database.

Parameter Description

Parameter name	Parameter type	Description	Not null
name	string	User name	Yes
password	string	Password	Yes

Format

The method "dropUsr()" contains 2 parameter: "name" and "password". Both of them are in the type of string.

```
("<User name>","<password>")
```

Sample

- Delete a user through the user name "root" and the password "admin".

```
db.dropUsr("root","admin")
```

db.collectionspace.createCL()

db.collectionspace.createCL(<name>,[options])

Create a collection in a specified collection space. Collection is a logical object which stores records. Each record should belong to on and only one collection.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
name	string	Collection name. Collection names should be unique to each other in a collection space.	yes
options	option	when create collection,you can set other properties about the collection through the param *options*,such as assigning the ShardingKey or whether storing the data in compressed form.	no

Format

The method "createCL()" contains 2 parameters: name and shardingkey. The value of "name" is a *string*, which should not be null. the param *options* be used to set other properties for the collection,at present,you can set the following properties by the *options*:

(1)ShardingKey: a json form param,the format is:{ShardingKey:{field name1:1|-1,field name2:1|-1,...}, {ShardingType:"hash"} [, {Partition:<number of slices>}]} , ShardingType specify the way of sharding,"Range Sharding" is in default;"Partition" specify the number of slices,only fill in Hash Sharding.

(2)copy number: a json form param,the format is:{ReplSize:<int num>}

(3)data commpressed:a json form param,the format is:{Compressed:true|false}

```
{"name":"<collection name>",[options]}
```

Note:

- The parameter "shardingkey" is a JSON object. Each of the fields in the JSON object corresponds to that in shard key. The value is 1 or -1, representing ascending and descending order.
- The parameter "ReplSize" is a JSON object,the value of it is a *int* type,Be used to set the number of writed data nodes, the value can not be greater than the number of nodes in the current data group.
- The parameter "Compressed" is a JSON object,the value of it is *boolean* type,if the value is "true",then standing for the data of the collection will be stored in compressed form ,otherwise,data will be stored in normal .
- when the *options* have one or more params,please use comma(,) to separate.
- The value of "name" should not be null string, "." or "$". The length of it should not be greater than 127B, or exception will occur.

Sample

- Create collection "bar" in collection space "foo" without shard key. If the collection space "foo" doesn't exist, it will automatically generate collection space "foo".

```
db.foo.createCL("foo")
```

- Create collection "bar" in collection space "foo" with the shard key "age" and in ascending order.

```
db.foo.createCL("bar", {age:1})
```

db.collectionspace.dropCL()

db.collectionspace.dropCL(<name>)

Delete a specified collection in a specified collection space.

Parameter Description

Parameter name	Parameter type	Description	Not null
name	string	Collection name. Collection names should be unique to each other in a collection space.	Yes

Format

The parameter "name" should be specified in the method "dropCL()". It should be ensured that the collcection name exists in the collection space, or operation exception will occur.

```
{"name":"<collection name>"}
```

Note:

- The value of "name" should not be null string. It should not contain "." or "$". And the length of it should not be greater than 127B. If the collection name is invalid , the operation will fail.
- It should be ensured that the collection name exists in a collection space, or exception will occur.

Sample

- Supposing that the collection "bar" exists in the collection space "foo", the following command will delete it.

```
db.foo.dropCL("bar")
```

db.collectionspace.getCL()

db.collectionspace.getCL(<name>)

Returns the quotation of collection in a collection space.

Parameter description

Parameter name	Parameter type	Description	Not null
name	string	Collection name. In a collection space, collection name should be unique to each other.	Yes

Format

The parameter "name" in the method "getCL()" should be specified and the "name" should be valid, or exception will occur.

```
{"name":"<collection name>"}
```

Note:

- The value of "name" should not be null string. It should not contain "." or "$". The length of should not be greater than 127B. If the value of "name" is invalid, exception will occur.
- Make sure that the collection name exists in the collection space, or operation exception will occur.

Sample

- Supposing that the collection name "bar", the command returns the quotation of the collection "bar" in the collection space "foo".

```
db.foo.getCL("bar")
```

db.collectionspace.collection.insert()

db.collectionspace.collection.insert(<doc|docs>,[flag])

Insert records into specified collection. If collection space or collection doesn't exist, please create a new collection space. For example, the command "db.createCS("foo")" creates a new collection space named "foo". Then create a new collection in collection space. For instance, the command "db.foo.createCL("bar")" creates a collection named "bar" under the collection space "foo" so that users can insert data into the collection "bar".

Parameter Description

Parameter Name	Parameter Type	Description	Not Null
doc|docs	json object	Document record. The parameter "doc" means one record. The parameter "docs" means more than one record.	yes
flag	Int	The paremeter "flag" means whether the value of "_id" should be showed. The value of it is SDB_INSERT_CONTONDUP or SDB_INSERT_RETURN_ID. It is only be effective when insert one document.	no

Format

The definition of "insert()" contains two fields: doc|docs and flag. The parameter "doc|docs" is in the type of JSON object. The parameter "flag" is in the type of int. The value of "flag" is SDB_INSERT_CONTONDUP or SDB_INSERT_RETURN_ID,when you want to insert one document and set it as SDB_INSERT_RETURN_ID,then it will return a value of _id of the document;if you set SDB_INSERT_CONTONDUP,return nothing;

doc：

```
({"< field name 1>":"< value >","< field name 2>":"< value >",...} [,SDB_INSERT_CONTONDUP|SDB_INSERT_RETURN_ID ])
```

docs：

```
([
  {"< field name 1>":"< value >","< field name 2>":"< value >",...},
  {"< field name 1>":"< value >","< field name 2>":"< value >",...},...
 ])
```

Note:

- If the value of "_id" is not specified in a record that is about to be inserted, the system will generate the value of "_id" automatically to ensure the uniqueness of the record.

Sample

- Insert a record without the value of "_id".

```
db.foo.bar.insert({name:"Tom",age:20})
```

This operation inserts a new record into collection "bar". In this record, the value of "name" is "Tom", and the value of "age" is 20. The value of "_id" is uniquely generated:

```
{ "_id": { "$oid": "515152ba49af395200000000" }, "name": "Tom", "age": 20 }
```

- Insert a record that contains the value of "_id".

```
db.foo.bar.insert({_id:10,age:20})
```

This operatin inserts a new record into the collection "bar". The value of "_id" is 10. The value of "age" is 20:

```
{ "_id": 10, "age": 20 }
```

- Insert more than one record.

```
db.foo.bar.insert([{_id:20,name:"Mike",age:15},{name:"John",age:25,phone:123}])
```

This operation will insert 2 records into the collection "bar":

1)In the first record, the value of "_id" is 20. The value of "name" is "Mike". The value of "age" is 15.

2)In the second record, teh value of "_id" is generated by the system. The value of "name" is "John". The value of "age" is 25. The value of "phone" is 123.

```
{
"_id": 20,
"name": "Mike",
"age": 15
}
```

```
{
"_id": { "$oid": "5151557a49af395200000001" },
"name": "John",
"age": 25,
"phone": 123
}
```

db.collectionspace.collection.count()

db.collectionspace.collection.count([cond])

Return the total amount of records in a specified collection in a collection space.

Parameter Description

Parameter name	Parameter type	Description	Not null
cond	json object	Selecting condition. If it is null, it will count all the records in the collection. If it is not null, it will count records that match the conditions.	no

Format

The method "count()" contains field named "cond". It is a JSON object.

```
{[{"Field name 1":{"match 1":"value 1"},"Field name 2":{"match 2":"value 2"},...}]}
```

Sample

- Count all the records in collection "bar" without specifying "cond".

```
db.foo.bar.count()
```

- Count records that contain the value "Tom" on the field "name" and the value on the field "age" greater than 25.

```
db.foo.bar.count({name:"Tom",age:{$gt:25}})
```

db.collectionspace.collection.find()

db.collectionspace.collection.find([cond],[sel])

The method "find()" selects records that users need. It will return a cursor. In Sequoiadb, a cursor is a pointer which point to a query result set. The client can go through teh query results.

Parameter Description

Parameter Name	Parameter Type	Description	Not Null
cond	json object	Selecting condtion. If it is null, it will find all the records. If it is not null, it will find records that matches the condition.	no
sel	json object	It chooses fields to be returned. If it is null, it will return all the records.If a field doesn't exist, it will return null string.	no

Format

The definitin of "find()" contains 2 parameters: cond and sel, which are both in the type of JSON object. The parameter "cond" specifys the matching condition. The parameter "sel" specifys the fields to be returned.

```
{[{"field name 1":{"match 1":"value 1","field name 2":{"match 2":"value 2"},...}],[{"field name 1":"","field name 2":"",..}]}
```

Note:

The parameter "sel" is in the type of a json object. The value of the field is null string. But if it is specified as :{"field name 1":"value 1","field name 2":"value 2",...} and records conain choosed field, the values are invalid. If fields specified in "sel" don't exist, it will return "{"field name 1":"value 1","field name 2":"value 2",...}".

Sample

- Find all the records without specifying the field "cond" and "sel".

  ```
  db.foo.bar.find()
  ```

- Find records that matches the value of the parameter "cond".

  ```
  db.foo.bar.find({age: {$gt: 25}, name: "Tom"})
  ```

  This operation will return records that contains the value of "age" greater than 25 and the value "Tom" on the field "name".

- Specify the fields to return by setting the content of the parameter "sel".For example, there are 2 records: "{age:25,type:"system"}" and "{age:20,name:"Tom",type:"normal"}".

  ```
  db.foo.bar.find(null, {age: "", name: ""})
  ```

  This operation return the value of "age" and "name". So it will return:"{age:25,name:""} , {age:20,name:"Tom"}". Although the first record doesn't contain the field, it will return "name:"".

db.collectionspace.collection.remove()

db.collectionspace.collection.remove([cond],[hint])

Delete record in a collection

Parameter description

Parameter name	Parameter type	Description	Not Null
cond	json object	Selectable condition. If it is null, it will delete all the records. If it is not null, delete records that match the condition.	no
hint	json object	Specify visiting plan	no

Format

The "cond" is a json object. When it is null (For example, "{}".), delete all records in the collection. "hint" is a json object that contains a single element. The field name of this element is ignored. But the value of the field in it is specified as the name of index which is needed to visit. When the value of field is null, it will visit all the records in the collection.

```
{[{"Field name 1":{"Matching character 1":"value 1","Field name 2":{"Matching character 2":"value 2"},...}],
[{"":"index name"|null}]}
```

Sample

- Delete all the records in the collection "bar".

  ```
  db.foo.bar.remove()
  ```

- Delete records that match the condition in "cond" according to visit plan.

  ```
  db.foo.bar.remove({age:{$gte:20}},{"":"myIndex"})
  ```

  This manipulation visits records in a collection named "bar" according to the index named "myIndex" and delete records which contains the value of "age" greater than 20.

db.collectionspace.collection.createIndex()

db.collectionspace.collection.createIndex(<name>,<indexDef>,[isUnique])

Create index in a collection to accelerate query.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
name	string	Index name. It should be unique in a collection.	yes
indexDef	json object	Index key. It contains one or more objects that specify index fields and order direction. "1" means ascending order. "-1" means descending order.	yes
isUnique	Boolean	It shows whether the index is unique. The default value is "false". When it is "true", the index is unique.	no

Format

The method "createIndex()" contains three parameters: "name", "indexDef" and "isUnique". The value of "name" should be in the type of string. The field "indexDef" is defined as a JSON object, which contains at least one field. The field name is index name. The value of it is 1 or -1. "1" means ascending order. "-1" means descending order. The parameter "isUnique" is in the type of bool. Its default value is "false".

```
{"name":"<index name>","indexDef":{"<index field 1>":<1|-1> [,"<index field 2>":<1|-1>...] },["isUnique":<true|
false>]}
```

Note:

- Values on the field that an unique index specified should not be the same in defferent records.
- Index name should not be null string. It should not contain "." or "$". The length of it should not be lesser than 127B.

Sample

- Create an unique index named "ageIndex" on the field "age" in collection "bar". The records are in ascending order on the field "age".

```
db.foo.bar.createIndex("ageIndex", {age:1}, true)
```

db.collectionspace.collection.dropIndex()

db.collectionspace.collection.dropIndex(<name>)

Delete specified index in a collection

Parameter description

Parameter name	Parameter type	Description	Not null
name	string	Index name. Index names should be unique to each other in the same collection.	Yes

Format

Then method "dropIndex()" contains a field named "name". The value of "name" should be string.

```
{"name":"<index name>"}
```

Note:

- When deleting index, it should be ensured that the index name is in the collection.
- The index name can't be bull、contain dot(.) or '$',and the length of it must be not more than 127B.

Sample

- Supposing that the index "ageIndex" exists in the collection "bar", the following command will delete it.

```
db.foo.bar.dropIndex("ageIndex")
```

db.snapshot()

db.snapshot(<snapType>,[con],[sel],[sort])

It lists snapshots. Snapshot is a kind of command used to get the current status of system.Click here to learn more about snapshot.

Parameter Description

Parameter name	Parameter type	Description	Not null
snapType	List	Snapshot type, the value of snapshot type is in the range from 0 to 7.	Yes
con	json object	Selectable condition. It just returns records that matches the value	No

Parameter name	Parameter type	Description	Not null
		of "con". If it is null, it returns all records.	
sel	json object	Select returned field name. If it is null, return all the fields	No
sort	json object	Sort records according to specified field. "1" means ascending order. "-1" means descending order.	No

Format

The defination of the method "snapshot()" contains four fields: listType、con、sel、sort. The type of "listType" is enumeration. Other fields are json objects. The format is as follow:

```
{"snapType":"<snapshot type>",["con":"{"Field name 1":{"operation 1":"value 1"},"Field name 2":{"Operation 2":"value
 2"}...}"],
["sel":"{"Field name 1":"","Field name 2":"",...}"],["sort":"{Order:-1|1}"]}
```

Note:

- The value of snapType is in the range from 0 to 7.
- The parameter "sel" is a json object. The value of field name is specified as null string. It is defined as: {"Field name 1":"value 1","Field name 2":"value 2",...}.If the records contain the specified field names, the values are invalid. If the records don't contain specified fields, it returns "{"Field name 1":"value 1", "Field name 2":"value 2"}".
- If the value of a field is in the type of array. We invoke the elements in it through "." and double quotations.

Sample

- Set the value of "snapType" as 0:

```
db.snapshot(0)
```

Returns context snapshot:

```
{
        "SessionID": 3,
        "Contexts": [
          {
            "ContextID": 1,
            "DataRead": 0,
            "IndexRead": 0,
            "QueryTimeSpent": 0,
            "StartTimestamp": "2013-04-08-17.17.02.135620"
          }
        ]
      }
```

- `db.snapshot(0,{"Contexts.0.ContextID":{$gte:0}},{SessiomID:"","Contexts.0.StartTimestamp":""},{SessionID:1})`

  Return records that contain the value of "Contexts.0.ContextID" equal to 0, and just return values in the 2 field, SessiomID and Contexts.StartTimestamp. The returned records are in ascending order:

```
{
        "Contexts": [
          {
            "StartTimestamp": "2013-04-09-09.31.03.270965"
          }
        ],
        "SessionID": 5
      }
```

db.collectionspace.collection.getIndex()

db.collectionspace.collection.getIndex(<name>)

Return the quotation of specified index.

Parameter description

Parameter name	Parameter type	Description	Not null
name	string	Index name, all the index name in a collection should be unique to each other.	Yes

Format

The method "getIndex()" contains the field "name", which is in the type of string.

```
{"name":"<index name>"}
```

Note:

- It should be ensured that the index name exists in the collection before users get index quotation.
- Index name should not contain null string, "." or "$". The length of it should not be greater than 127B.

Sample

- Supposing that the index "ageIndex" exists in collection "bar", the following command will return the quotation of it.

```
db.foo.bar.getIndex("ageIndex")
```

db.collectionspace.collection.listIndexes()

db.collectionspace.collection.listIndexes()

Enumerate indexes. It will show information of all indexes in the specified collection.

Sample

- The following command will return information of all the indexes in the collection "bar".

```
db.foo.bar.listIndexes()
```

db.collectionspace.collection.update()

db.collectionspace.collection.update(<rule>,[cond],[hint])

Update records in a collection.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
rule	json object	Update rule. Records will be updated according the value of "rule".	yes
cond	json object	Selecting condition. If it is null, update all the records. If it is not null, it will update records that match the condition.	no
hint	json object	It specify the visiting plan by giving an index.	no

Format

The method "update()" should contains the field "rule". It is a json object. The parameter "cond" and "hint" are selectable. The parameter "hint"

contains one json object that contains one field. The "name" in this field is ignored, but the value specify the index to visit records. When the value is

null, it will visit all the records in the collection. The format is "{"":null}" or "{"":"<indexname>"}".

```
{<{""Update character 1"": {field name 1:"value"},"Update character 2": {"field name 2":"value 2"},...}>,[{"field name 1": {"match 1":"value 1"},"field name 2": {"match 2":"value 2"},...}],[{"":"Index name"|null}]}
```

Sample

- Update all the records according to the updating rule. That's to say, we merely set the value of "rule", but not "cond" or "hint".

  ```
  db.foo.bar.update({$inc: {age:1}})
  ```

  This operation updates the field "age" in collection "bar" by adding 1 to the value of "age".
- Select records that match the condition. Update these records according to update rules by setting the values of "rule" and "cond".

  ```
  db.foo.bar.update({$unset: {age:""}}, {age: {$exists:1},name: {$exists:0}})
  ```

  This operation will update records that contains the field "age" but not the field "name" in collection "bar" and delete the field "age" in these records.
- Update records according to visiting plan, supposing that the collection contains the specified index name.

  ```
  db.foo.bar.update({$inc: {age:1}}, {age: {$gt:20}}, {"":"testIndex"})>
  ```

  This operation visit records that contains the value of "age" greater than 20 in the collection "bar" through the index "testIndex". Add 1 to the field of "age".

db.collectionspace.collection.upsert()

db.collectionspace.collection.upsert(<rule>,[cond],[hint])

Update collection records. The method "upsert" and the method "update" are used to update records. But when no records is

matched according to the parameter "cond", "update" does nothing, but "upset" will insert data once.

Parameter description

Parameter name	Parameter type	Description	Not null
rule	json object	Update rule. It will update record according to the value of "rule".	Yes
cond	json object	Selectable condition. When it is null, update all the records. If it is not null, update records that match the condition in "cond".	No
hint	json object	Specify visiting plan.	No

Format

The method "upsert()" should contians the parameter "rule". The parameter "rule" is a json object. The parameters "cond"

and "hint" are selectable.The parameter "cond" and "hint" are selectable. The parameter "hint"

contains one json object that contains one field. The "name" in this field is ignored, but the value specify the index to visit records. When the value is

null, it will visit all the records in the collection. The format is "{"":null}" or "{"":"<indexname>"}".

```
{<{""Update character 1"": {"Field name 1": "value"}, "Update character 2": {"Field name 2": "value 2"},...}>, [{"Field name
 1": {"match 1": "value 1"}, "field name 2": {match 2": "value 2"},...}], [{"": "index name "|null}]}
```

Sample

Supposing that there are 2 records in the collection "bar".

```
{
    "_id": {
      "$oid": "516a76a1c9565daf06030000"
    },
    "age": 10,
    "name": "Tom"
  }
  {
    "_id": {
      "$oid": "516a76a1c9565daf06050000"
    },
    "a": 10,
    "age": 21
  }
```

- Update all the records according to the updating rule. That's to say, we merely set the value of "rule", but not "cond" or "hint".

  ```
  db.foo.bar.upsert({$inc: {age: 1}, $set: {name: "Mike"}})
  ```

  This operation is equivalent to that of the method "update()". It updates all the records in the collection ''bar". It adds 1 to the value of "age" and changes the value of "name" into "Mike". If a record doesn't contain the field "name", "$set" will insert the field of "name" and its value into it and return with the method "find".

  ```
  {
    "_id": {
      "$oid": "516a76a1c9565daf06030000"
    },
    "age": 11,
    "name": "Mike"
  }
  {
    "_id": {
      "$oid": "516a76a1c9565daf06050000"
    },
    "a": 10,
    "age": 22,
    "name": "Mike"
  }
  ```

- Update all the records according to the updating rule. That's to say, we set the value of "rule", "cond" and "hint".

  ```
  db.foo.bar.upsert({$inc: {age: 3}}, {type: {$exists: 1}})
  ```

  This operation will update records that contains the field "type" in the collection "bar" and add 3 to the value of "age" in them. The 2 records above don't contain the field "type", so it will insert a new record. This new record contains only the field "_id" and the field "age". The value of "_id" is automatically generated by the system. The value of "age" is 23.

  ```
  {
    "_id": {
      "$oid": "516a76a1c9565daf06030000"
    },
    "age": 11,
    "name": "Mike"
  }
  {
    "_id": {
      "$oid": "516a76a1c9565daf06050000"
    },
    "a": 10,
    "age": 22,
    "name": "Mike"
  ```

```
  }
{
  "_id": {
    "$oid": "516cfc334630a7f338c169b0"
  },
  "age": 3
}
```

- Update records according to visiting plan, supposing that the index name "testIndex" exists in the collection.

```
db.foo.bar.upsert({$inc: {age:1}}, {age: {$gt:20}}, {"":"testIndex"})>
```

This operation is equal to that of "update". It visits records that contains the value of "age" greater than 20 through the index "testIndex" in collection "bar". Then it adds 1 to the value of the field "age" in these records. Then it returns:

```
{
   "_id": {
     "$oid": "516a76a1c9565daf06050000"
   },
   "a": 10,
   "age": 23,
   "name":"Mike"
}
```

db.collectionspace.collection.split()

db.collectionspace.collection.split(<source group>,<target group>,<condition>|<percent(0~100)>)

At least two sharding groups environment,the data records will be cut into different sharding group according to the specified conditions.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
source group	string	The source sharding group.	yes
target group	string	The target sharding group.	yes
condition	json object	Range split condition	condition or percent
percent(0~100)	int(32)	percent split condition	condition or percent

Format

Data spliting is divided into range-spliting and percent-spliting,"source group" and "target group"are common params,and are string type;the param "condition" is filled when range spliting,it is a object of json structure;param "percent" is filled when percent-spliting,it is a int type.

- range-spliting

  Range Sharding use exact conditions when range-spliting,while Hash Sharding use "Partition"(number of slices) conditions.The starting condition is a required field and the ending condition is a optional condition when spliting,if the ending condition doesn't fill,then it default as the max range of data that "source group" contains.

```
("<source group>","<target group>", <condition>)
```

- percent-spliting

```
db.foo.bar.split("<source group>","<target group>", <percent>)
```

Sample

- Range-spliting of Hash Sharding

```
db.foo.bar.split("group1", "group2", {Partition:10}, {Partition:20})
```

- Range-spliting of Range Sharding

```
db.foo.bar.split("group1", "group2", {$and: [{a: {$lt:10}}, {b: {$lt:20}}]})
```

- Percent-spliting

```
db.foo.bar.split("group1", "group2", 50)
```

rg.start()

rg.start()

It starts replset. After a replset is started, operations like creating nodes can be executed. Users can start specified node with the method db.startRG(<name>)db.startRG(<name>).

Sample

- Start replset:

```
rg.start() //Equal to db.startRG("rg")
```

rg.createNode()

rg.createNode(<HostName>,<Port>,<Path>)

Create a node in a replset.

Note:

Only after a replset is started can a node be created.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
HostName	string	Hostname	yes
Port	string	Node port	yes
Path	string	Node path	yes

Format

The method "rg.createNode()" contains 3 parameters: HostName、Port、Path. All of them in the type of string, should not be null.

```
("<Hostname>","<Port>","<Node path>")
```

Sample

- Create a node in port 60000 in replset "rg".

```
rg.createNode("vmsvr2-suse-x64",60000,"/opt/sequoiadb/data/60000")
```

Note:

In a replset, more than one replset can be created. But the margin between two ports of them should not be greater than 5. Because the system controls 5 communication interfaces for each node.

rg.getDetail()

rg.getDetail()
Return information of replsets.

Sample

-

```
> rg.getDetail()
{
  "Group": [
```

```
    {
      "dbpath": "/opt/sequoiadb/data/60000",
      "HostName": "vmsvr2-suse-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "60000"
        },
        {
          "Type": 1,
          "Name": "60001"
        },
        {
          "Type": 2,
          "Name": "60002"
        },
        {
          "Type": 3,
          "Name": "60003"
        }
      ],
      "NodeID": 1000
    },
    {
      "dbpath": "/opt/sequoiadb/data/60010",
      "HostName": "vmsvr2-suse-x64",
      "Service": [
        {
          "Type": 0,
          "Name": "60010"
        },
        {
          "Type": 1,
          "Name": "60011"
        },
        {
          "Type": 2,
          "Name": "60012"
        },
        {
          "Type": 3,
          "Name": "60013"
        }
      ],
      "NodeID": 1001
    }
  ],
  "GroupID": 1001,
  "GroupName": "group",
  "PrimaryNode": 1001,
  "Role": 0,
  "Status": 1,
  "Version": 3,
  "_id": {
    "$oid": "517b2fc33d7e6f820fc0eb57"
  }
}
```


rg.getMaster()

rg.getMaster()

Return the master node of a replset.

Sample

- Return the master node of a replset.

```
> rg.getMaster()
Return: vmsvr2-suse-x64:60010
```

rg.getSlave()

rg.getSlave()

Return the slave nodes in replset.

Sample

- Return slave nodes in replset:

```
> rg.getSlave()
Return: vmsvr2-suse-x64:70010
```

rg.getNode()

rg.getNode(<nodename>|<hostname>,<servicename>)

Return information of specified node.

Parameter Description

Parameter name	Parameter type	Description	Not null
nodename	string	Node name	Either "nodename" or "hostname"
hostname	string	Hostname	Either "nodename" or "hostname"
servicename	string	Server name	Yes

Format

The method "rg.getNode()" has 2 parameter. The first one is node name or hostname. The secend one is server name. The types of the 2 parameters are both string. And they should not be null.

```
("<Node name>|<hostname>","<server name>")
```

Sample

- Return node according to specified hostname and server name

```
> rg.getNode("vmsvr2-suse-x64","70000")
Return: vmsvr2-suse-x64:70000
```

rg.stop()

rg.stop()

Stop replset. After stopping replset, manipulations lilke creating nodes cannot be executed on the replset.

Sampel

- Stop replset.

```
rg.stop()
```

node.start()

node.start()

Start the current node.

Sample

- Start a current node named "node".

```
node.start()
```

node.connect()

node.connect()

Connect the database to specified node. After connecting them, users can execute a set of operations. Users can check relative manipulations with the method "node.connect().help()".

Sample

- Connect the database to a node named "node".

```
> node.connect()
Return: vmsvr2-suse-x64:60000
```

node.getHostName()

node.getHostName()

Return the hostname of a node.

Sample

- Return the hostname of a node called "node".

```
> node.getHostName()
Return: vmsvr2-suse-x64
```

node.getServiceName()

node.getServiceName()

Return the server name of a node.

Sample

- Return the server name of a node called "node".

```
> node.getServiceName()
Return: 60000
```

node.getNodeDetail()

node.getNodeDetail()

Return information of current node.

Sample

- Return information of a node named "node".

```
> node.getNodeDetail()
Return: 1000:vmsvr2-suse-x64:60000(group)
Here "1000" is node ID(NodeID); "vmsvr2-suse-x64" is hostname(HostName);
"60000" is service name(ServiceName), "(group)" is the replset name of the node.
```

node.stop()

node.stop()

Stop the current node.

Sample

- Stop the current node named "node".

```
node.stop()
```

cursor.sort()

cousor.sort(<sort>)

Sort result set according to a specified field.

Parameter Description

Parameter Name	Parameter Type	Description	Not null
sort	json object	Sort result set according to a specified field. The value of it is 1 or -1. 1 represents ascending order. -1 represents descending order.	no

Format

The method "cousor.sort()" contains a parameter called "sort". It is a json object. If it is not specified, result set will not be sorted.

The format is as follow:

```
{<Field name 1>:<-1|1>,<Field name 2>:<-1|1>,...}
```

Sample

- Return records that contains the value of "age" greater than 20. And it just return the fields named "name" and "age" in them with ascending order on the field "age".

```
db.foo.bar.find({age:{$gt:20}},{age:"",name:""}).sort({age:1})
```

Note:

Through the method "find()", we can select the fields we need. In the example above, it returns "age" and "name", so it we use "sort()" to sort the records, we can only sort according to "age" and "name". If we don't specify the parameter "sel" in the method "find", we can sort the records according to any field.

cursor.hint()

cursor.hint(<hint>)

Visit result set using specified index.

Parameter Description

Parameter name	Parameter type	Description	Not null
hint	json object	It specifies the visiting plan to accelarate query speed.	no

Format

The method "cursor.hint()" contains parameter "hint". If it is not specified, the query will not use index to visit records.

The parameter "hint" contain a one-field json object. The field name is ignored. But the field value is specified as the index

which will be visited. When it is null, it will visit all the records in the collection.

The format is as follow:

```
{"":null} or {"":"<indexname>"}
```

Sample

- Find records that contains the field "age" in the collection "bar" using the index "ageIndex" and return results.

```
db.foo.bar.find({age: {$exists:1}}).hint({"":"ageIndex"})
```

cursor.limit()

cursor.limit(<num>)

Set the maximum amount of records in result set.

Parameter description

Parameter name	Parameter type	Description	Not null
num	int	The parameter "num" sets the maximum amount of records in result set.	no

Format

The method "cousor.limit()" contains the parameter "num". It is in the type of "int". If the value of "num" is not specified, it will return all the records in the result set. If users need the first 5 records, the value of "num"should be set as 5.

Sample

- Select records with the value of "age" greater than 10 in the collection "bar" and return the first 10 records.

```
db.foo.bar.find({age: {$gt:10}}).limit(10)
```

Note:

If the amount of records in the result set is lesser than 10, it will return all the records. It it is greater than 10, it will return the first 10 records.

cursor.skip()

cursor.skip(<num>)

It is used to specify the first returned record.

Parameter description

Parameter name	Parameter type	Description	Not null
num	int	Specify the amount of records in returned result set.	No

Format

The method "cousor.skip()" contains the parameter "num". It is in the type of "int". If the value of "num" is null, or it is 0, the method will return the whole result set.

If you want to get a result set that begin with the 3rd record, you should set the value of "num" as "2".

Sample

- Select records with the value of "age" greater than 10 and return records which begin with the 5th record. That's to say, it will skip the fisrt 4 records.

```
db.foo.bar.find({age: {$gt:10}}).skip(4)
```

Note:

If the amount of records in a result set is lesser than 5, no record will be returned. If the amount of records in a result set is greater than 5, it will return records which starts with the 5th record.

cursor.current()

cursor.current()

Return the record that current cursor currently point to. Click here to learn more about the method "cursor.next()".

Sample

- Select records with the value of "age" greater than 10 in the collection "bar" and return the record that current cursor currently point to.

```
db.foo.bar.find({age: {$gt:10}}).current()
```

cursor.next()

cursor.next()

Return the next record in current cursor. Click here to know more about the method "cursor.current()".

Sample

- Select records with the value of "age" greater than 10 in the collection "bar" and return the next record in current cursor.

```
db.foo.bar.find({age: {$gt:10}}).next()
```

cursor.size()

cursor.size()

Return the amount of records from current cursor to final cursor.

Sample

- Select records with the value of "age" greater than 10 in the collection "bar" and return the amount of record in the range from current cursor to final cursor.

```
db.foo.bar.find({age: {$gt:10}}).size()
```

cursor.toArray()

cursor.toArray()

Return result set in the type of array.

Sample

- Select records with the value of "age" greater than 5 in the collection "bar" and return the amount of record in the range from current cursor to final cursor.

```
db.foo.bar.find({age: {$gt:10}}).size()
```

```
Return:{
  "_id": {
    "$oid": "516a76a1c9565daf06030000"
  },
  "age": 10,
  "name": "Tom"
},{
  "_id": {
    "$oid": "516a76a1c9565daf06050000"
  },
  "age": 20,
  "a": 10
},{
  "_id": {
    "$oid": "516a76a1c9565daf06040000"
  },
  "age": 15
}
```

Operator

Matching records

Match	Description	Sample
$gt	Greater than	db.foo.bar.find({age:{$gt:20}})
$gte	Greater than or equal to	db.foo.bar.find({age:{$gte:20}})
$lt	Lesser than	foo.bar.find({age:{$lt:20}})
$lte	Lesser than or equal to	db.foo.bar.find({age:{$lte:20}})
$ne	Unequal to	db.foo.bar.find({age:{$ne:20}})
$et	equal to	db.foo.bar.find({age:{$et:20}})
$mod	取模	db.foo.bar.find({age:{$mod:[6,5]}})
$in	Exist in the collection	db.foo.bar.find({age:{$in:[20,21]}})
$nin	Not exist in the collection	db.foo.bar.find({age:{$nin:[20,21]})

Match	Description	Sample
$all	All	db.foo.bar.find({age:{$all:[20,21]}})
$and	And	db.foo.bar.find({$and:{age:20,name:"Tom"}})
$not	Not	db.foo.bar.find({$not:{age:20,name:"Tom"}})
$or	Or	db.foo.bar.find({$or:{age:20,name:"Tom"}})
$type	Data type	db.foo.bar.find({age:{$type:16}})
$exists	Exists	db.foo.bar.find({age:{$exists:1}})
$elemMatch	Match elements	db.foo.bar.find({age:{$elemMatch:20}})
$size	Size	db.foo.bar.find({array:{$size:3}})
$regex	Regular	db.foo.bar.find({str:{$regex:'dh.*fj',$options:'i'}})
$options	Regex options	db.foo.bar.find({str:{$regex:'dh.*fj',$options:'i'}})

Update rule

Undate operator	Description	Information	Sample
$inc	Add	Add specified value to the value of specified field.	db.foo.bar.update({$inc:{age:25}})
$set	Set	Set the value of specified field as specified value.	db.foo.bar.update({$set:{age:10}})
$unset	Delete	Delete specified field in an object.	db.foo.bar.update({$unset:{age:""}})
$addtoset	Add to set	If the element to add doesn't exist in the array, the command add it, or it will be ignored. The target field should be array.	db.foo.bar.update({$addtoset: {array:[3,4,5]}})
$pop	Delete the last N value in an array	Delete the last N value in an array. The target field should be array.(If N is lesser than 0, it will delete the first N elements in the array.)	db.foo.bar.update({$pop:{array:2}})
$pull	Delete value	Delete the specified value in a target array. The target field should be array.	db.foo.bar.update({$pull:{array:2}})
$pull_all	Delete array	Delete all the values of a speficied array in a target array. The target field should be array.	db.foo.bar.update({$pull_all:{array: [2,3,4}})
$push	Push value	Insert a value into a target array. The target field should be array.	db.foo.bar.update({$push: {array:2}})
$push_all	Push array	Insert all the values in a specified array into a target array. The target field should be array.	db.foo.bar.update({$push_all: {array:[2,3,4}})

Note:

In sequoiaDB, the structure called JSON (field name: value) is used to express "=".Of course , you can use *$et* operator.

Sample:

db.collectionspace.collection.find({a:42})

This command find records that contains the value of "a" equal to 42.

Match Operator

operator	description	samples
$gt	greater than	db.foo.bar.find({age:{$gt:20}})
$gte	great than or equal	db.foo.bar.find({age:{$gte:20}})
$lt	less than	foo.bar.find({age:{$lt:20}})
$lte	less than or equal	db.foo.bar.find({age:{$lte:20}})
$ne	not equal	db.foo.bar.find({age:{$ne:20}})
$in	exist in collection	db.foo.bar.find({age:{$in:[20,21]}})
$nin	not exist in collection	db.foo.bar.find({age:{$nin:[20,21]})
$all	all	db.foo.bar.find({age:{$all:[20,21]}})
$and	and	db.foo.bar.find({$and:{age:20,name:"Tom"}})
$not	not	db.foo.bar.find({$not:{age:20,name:"Tom"}})
$or	or	db.foo.bar.find({$or:{age:20,name:"Tom"}})
$type	date type	db.foo.bar.find({age:{$type:16}})
$exists	exist	db.foo.bar.find({age:{$exists:1}})
$elemMatch	element match	db.foo.bar.find({age:{$elemMatch:20}})
$size	size	db.foo.bar.find({array:{$size:3}})
$regex	regex	db.foo.bar.find({str:{$regex:'dh.*fj',$options:'i'}})
$options	regex options	db.foo.bar.find({str:{$regex:'dh.*fj',$options:'i'}})

$gt

Grammar

```
{<field name>: {$gt:<value>}}
```

Description

"$gt" finds records that contain value greater than the specific value "value" on the field "field name".

Sample

- Find records that contains value greater than 20 on the field "age" in the collection "bar" of the collection space "foo".

  ```
  db.foo.bar.find({age: {$gt:20}})
  ```

- When "$gt" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value greater than 2 under the field "service".

  ```
  db.foo.bar.update({$set: {age:25}}, {"service.type": {$gt:2}})
  ```

$gte

Grammar

```
{<field name>: {$gte:<value>}}
```

Description

"$gte" finds records that contain value greater than or equivalent to the specific value "value" on the field "field name".

Sample

- Find records that contains value greater than or euqivalent to 20 on the field "age" in the collection "bar" of the collection space "foo".

```
db.foo.bar.find({age: {$gte:20}})
```

- When "$lte" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value greater than or euqivalent to 2 under the field "service".

```
db.foo.bar.update({$set: {age:25}}, {"service.type": {$gte:2}})
```

$lt

Grammar

```
{<field name>: {$lt:<value>}}
```

Description

"$gte" finds records that contain value lesser than the specific value "value" on the field "field name".

Sample

- Find records that contains value lesser than 20 on the field "age" in the collection "bar" of the collection space "foo".

```
db.foo.bar.find({age: {$lt:20}})
```

- When "$lte" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value lesser than 15 under the field "service".

```
db.foo.bar.update({$set: {age:25}}, {"service.type": {$lt:15}})
```

$lte

Grammar

```
{<field name>: {$lte:<value>}}
```

Description

"$lte" finds records that contain value lesser than or equivalent to the specific value "value" on the field "field name".

Sample

- Find records that contains value lesser than or euqivalent to 20 on the field "age" in the collection "bar" of the collection space "foo".

```
db.foo.bar.find({age: {$lte:20}})
```

- When "$lte" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value lesser than or euqivalent to 15 under the field "service".

```
db.foo.bar.update({$set: {age:25}}, {"service.type": {$lte:15}})
```

$ne

Grammar

```
{<field name>: {$ne:<value>}}
```

Description

"$ne" finds records that contain value unequal to the specific value "value" on the field "field name".

Sample

- Find records that contains value unequal to 20 on the field "age" in the collection "bar" of the collection space "foo".

```
db.foo.bar.find({age: {$ne: 20}})
```

- When "$lte" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value unequal to 2 under the field "service".

```
db.foo.bar.update({$set: {age: 25}}, {"service.type": {$ne: 15}})
```

$et

Grammar

```
{<field name>: {$et: <value>}}
```

Description

"$et" finds records that contain value equal to the specific value "value" on the field "field name".

Sample

- Find records that contains value equal to 20 on the field "age" in the collection "bar" of the collection space "foo".

```
db.foo.bar.find({age: {$et: 20}})
```

- When "$et" manipulates records with nested objects, the functionupdate() will update records that contains "age" value of 25 with records which conatins a "type" value equal to 15 under the field "service".

```
db.foo.bar.update({$set: {age: 25}}, {"service.type": {$et: 15}})
```

$mod

Grammar

```
{<field>: {$mod: [value1, value2]},...}
```

Description

$mod is a modulo operator,to return the records which the value of specified field count modulo value1 is equal to value2.

Note:

1.Param value1 is a integer except 0;if it is filled with float,then only the integer part is effective;in addition,it can't be other types.

2.Param value2 is a integer;if it is filled with float,then only the integer part is effective;other types are treated with 0.

Sample

- Return the records that the value of field age count modulo 5 is equal to 3 in collection bar

```
db.foo.bar.find({age: {$mod: [5, 3]}})
return
{
  "_id": {
    "$oid": "521d5446e2d3c4e31c000000"
  },
  "age": 3
}...
```

```
db.foo.bar.find({age: {$mod: [2.3, 1.5]}})
return
{
```

```
  "_id": {
    "$oid": "521d5446e2d3c4e31c000000"
  },
  "age": 3
}
{
  "_id": {
    "$oid": "521d544ee2d3c4e31c000002"
  },
  "age": 5
  }...
```

Only the integer part are valid for the two elments in the array[2.3,1.5] .

$in

Grammar

```
{<field name>:{$in:[<value 1>,<value 2>,...<value N>]}}
```

Description

"$in" finds records that match any value of the array ([<value 1>,<value 2>,...<value N>]) on the field ("field name") within a collection. If the field ("field name") is in the type of array, records that match any value of the array ([<value 1>,<value 2>,...<value N>]) on the field ("field name").

Sample

- Find records with "age" value of 20 or 25 in the collection "bar":

```
db.foo.bar.find({age:{$in:[20,25]}})
```

- When manipulating "$in" on array fields, the following expression finds records with "name" value of "Tom" or "Mike" in the collection "bar", then deletes the field of "age" in these records.

```
db.foo.bar.update({$unset:{age:""}},{name:{$in:["Tom","Mike"]}})
```

Note: Array containing one value like {<field name>:{$in:[<value>]}} equals {<field name>:<value>}.

```
db.foo.bar.find(age:{$in:[20]}) is equivalent to db.foo.bar.find({age:20})
```

$nin

Grammar

```
{<field name>:{$nin:[<value 1>,<value 2>,...<value N>]}}
```

Description

"$nin" finds records that match none of the values in the array ([<value 1>,<value 2>,...<value N>]) on the field ("field name") within a collection. If the field ("field name") is in the type of array, records containing elements that match any value of the array ([<value 1>,<value 2>,...<value N>]) on the field ("field name").

Sample

- Find records without the field "age" or the value of 20 or 25 on the field "age" in the collection "bar":

```
db.foo.bar.find({age:{$nin:[20,25]}})
```

- When manipulating "$nin" on array fields, the following expression finds records without the field "name" or the value of "Tom" or "Mike" on the field "name" in the collection "bar", then deletes the field of "age" in these records.

```
db.foo.bar.update({$unset:{age:""}},{name:{$nin:["Tom","Mike"]}})
```

Note:

Array containing one value like {<field name>:{$in:[<value>]}} equals {<field name>:<value>}.

```
db.foo.bar.find(age: {$nin: [20]}) equals to db.foo.bar.find({age: {$ne: 20}})
```

$all

Grammar

```
{<Field name>: {$all: [<value 1>,<value 2>,...<value N>]}}
```

Description

Manipulation "$all" is manipulated on field names in type of Array. It will find records that contains all the values in the specified array ([<value 1>,<value 2>,...<value N>]) on the specified field ("<field name>").

Sample

- Find records that contain "Tom" and "Mike" on the specified field "name" in the collection "bar":

  ```
  db.foo.bar.find({name: {$all: ["Tom","Mike"]}})
  ```

  As a result, the system will match records that contains the field "name" and "name" values as follow in the collection "bar":

  ```
  ["Tom","Mike",..]
  ["Tom","Jhon","Mike",...]
  ```

  But the record below won't be matched:

  ```
  ["Tom","Jhon"]
  ```

Note:

When "$all" is manipulated on a field not in type of array, for example: "db.foo.bar.find(age:{$all: [20]})" equals to "db.foo.bar.find({age:20})".

$and

Grammar

```
{$and: [{<expression 1>}, {<expression 2>},..., {<expression N>}]}
```

Description

$and is a logical manipulation. It is aimed at searching for records that satisfies all the expressions (<expression 1> , <expression 2>,...,<expression N>).

But if the result of the1st expression (<expression 1>) is false, SequoiaDB will ignore all the expressions afterward.

Sample

- Find records with "age" value of 20 and "price" value of less than 10 in collection "bar":

  ```
  db.foo.bar.find({$and: [{age: 20}, {price: {$lt: 10}}]})
  ```

  Note:

  Sequoiadb provides implicit "and" manipulation: separeate expressions with ",". For example,

  ```
  db.foo.bar.find({age: 20, price: {$lt: 10}})
  ```

- When "and" is manipulated on the same field like {age：{$lt:20}}and{age:{$exists:1}}, then we can use "$and" to manipulate the two separated expressions, or combine them: {age:{$lt:20,$exists:1}}.

  ```
  db.foo.bar.update({$inc: {salary: 200}}, {$and: [{age: {$lt: 20}}, {age: {$exists: 1}}]})
  db.foo.bar.update({$inc: {salary: 200}}, {age: {$lt: 20, $exists: 1}})
  ```

When the results of 2 manipulations are the same, it will find records containing the field "age" with the "age" value of less than 20 in collection bar, then add 200 to the "salary" value of these records.

$not

Grammar

```
{$not:[{<expression 1>},{<expression 2>},...,{<expression N>}]}
```

Description

The opration "$not" is a logcal "not". It select records that don't match "(<expression 1><expression 2>,...,<expression N>)".

Sample

- Select records that contains the value of "age" unequal to 20 or the value of "price" equal to or greater than 10.

```
db.foo.bar.find({$not:[{age:20},{price:{$lt:10}}]})
```

$or

Grammar

```
{$or:[{<expression 1>},{<expression 2>},...,{<expression N>}]}
```

Description

The operation "$or" is a logical "or" operation. It returns "true" when a record matches one expresson of the expression group (<expression 1>,<expression 2>,...,<expression N>).

Once the result is "true", the matched record will be returned.

Sample

- Select records that contain the value "Tom" on the field "name", and the value 20 on the field "age" or the value of "price" lesser than 10.

```
db.foo.bar.find({name:"Tom",$or:[{age:20},{price:{$lt:10}}]})
```

- Apply "$or" in nested fields. The following command will select records that contains the value of "age" lesser than 20 or the value "system" on the field "type" which is nested in the field "snapshot", and use $incto update the value of "salary" in these records.

```
db.foo.bar.update({$inc:{salary:200}},{$or:[{age:{$lt:20}}],{"snapshot.type":"system"}])
```

$type

Grammar

{<field name>:{$type$type:<BSON type>}}

Description

Match the records which the value of the "<field name>" equals to the specified value of "<BSON type>" in the collection.

BSON Type

Type	Decription	value
32-bit integer	Integer,range -2147483648 to 2147483647	16
64-bit integer	Long Integer,range -9223372036854775808 to	18

Type	Decription	value
	9223372036854775807。 if the users specify a value can't be applied to an Integer,then SequoiaDB will converted to a Long Integer automatically	
double	Float,range 1.7E-308 to 1.7E+308	1
string	String	2
ObjectID	12Byte object ID	7
boolean	Boolean(true|false)	8
date	Date(YYYY-MM-DD)	9
timestamp	Timestamp(YYYY-MM-DD-HH.mm.ss.ffffff)	17
Binary data	Base64 binary form	5
Regular expression	Regular Expression	11
Object	Nested JSON Document Object	3
Array	Nested Array Object	4
null	Null	10

Samples

- Select records which the type of *age* field is Integer .

```
db.foo.bar.find({age: {$type: 16}})
```

- Select records which the type of nested field *arr* is Array.

```
db.foo.bar.find({"content.arr": {$type: 4}})
```

$exists

Grammar

{<field name>:{$exists:<0|1>}}

Description

"$exists" finds records that contain or do not contain the field "field name". "0" means finding records that do not contain the field "field name". "1" means finding records that contain the field "field name".

Sample

- Find records that contain the field "age" in collection "bar".

```
db.foo.bar.find({age: {$exists: 1}})
```

- Find records that do not contain object "content" with the field "name" in collection "bar".

```
db.foo.bar.find({"content.name": {$exists: 0}})
```

$elemMatch

Grammar

{<filed>:{$elemMatch:<value>}}

Description

return all record in collection where the '<field name>' match the specified '<value>'.

Sample

- array object match

```
db.foo.bar.find({"add.0": {$elemMatch: "china"}, "add.1": {$elemMatch: "ABC"}})
```

  return all records in the collection *bar* where the value of first element is "china" and the value of second element is "ABC" in the array *add*.
- json object match

```
db.foo.bar.find({content: {$elemMatch: {name: "Tom", phone: 123}}})
```

  field *content* is a nested json object,this operation is to match all records in the collection *bar* where the field *name* has the value *"Tom"* and the field *phone* has the value *"123"* in the nested object *content*.

$size

Grammar

{"<field name>":{$size:"<value>"}}

Description

By applying the operator "$size", users can find records that contain an array field named "field name", the length of which should be equal to "value".

Sample

- Return records that contain the array field "arr"

  db.foo.bar.find({arr:{$size:2}})

$regex

description

$regex Operation provides regular expression pattern matching string search functions. SequoiaDB using PCRE regex.

$options

$regex Offers four select flay:

- i:Setting this modifier mode for case-insensetive match letters.
- m:By default, pcre that the target string is composed by a single character, "the line" metacharacter (^) matches only at the beginning of the string, and the "end of line" metacharacter ($) matches only at the end of the string, or the last newline. When this modifier is set, the "first line" and "end of line" will match the target string before or after any newline, In addition, most were matching the target string beginning and the very end position, if the target string no "₩ n", or pattern does not appear ^ or $, setting this modifier does not have any effect.
- x:Setting this modifier mode without escaped or is not blank character class data characters will always be ignored, and is located in an unescaped # character outside a character class and the next newline characters between lines have been ignored.
- s:Setting this modifier mode dot metacharacter matches all characters, including newlines, without this modifier, the point number does not match a newline.

example

- Back to collection bar next str field values match case-insensitive regular expressions dh.*fj records

  db.foo.bar.find({str:{$regex:'dh.*fj',$options:'i'}})

$options

Grammar

Description

Sample

Update Operator

update operator	description	information	samples
$inc	increase	add the given value to the value of specified field	db.foo.bar.update({$inc:{age:25}})
$set	set the specified field	set the given value to the specified field	db.foo.bar.update({$set:{age:10}})
$unset	delete the specified field	delete the specified field of object	db.foo.bar.update({$unset:{age:""}})
$addtoset	add to set	if the given element doesn't exist in the array,then added,otherwise skipped.the object field must be array type	db.foo.bar.update({$addtoset: {array:[3,4,5]}})
$pop	pull the last of N values in the array	pull the last of N values in the array,the object field must be array type(if N less than 0 means that delete the first of -N values from the beginning of array).	db.foo.bar.update({$pop:{array:2}})
$pull	pull values	pull the specified value from the object array，the object element must be array.	db.foo.bar.update({$pull:{array:2}})
$pull_all	pull array	pull each of the elements in the specified array from the object array.the object element must be array type.	db.foo.bar.update({$pull_all:{array: [2,3,4}})
$push	push values	push the value to the object array,the object element must be array type.	db.foo.bar.update({$push: {array:2}})
$push_all	push array	push each of values in the specified array to the object array,the object element must be array type.	db.foo.bar.update({$push_all: {array:[2,3,4}})

$inc

Grammar

{$inc:{<field name1>:<value1>,<field name2>:<value2>,...}}

Description

$inc operation is to increase the specified "<value>" for the specified "<field name>".If these is no specified field name,then add the field name and value to the record; If the specified field name exist in records, then the value of the specified field name will plus the specified value.

Sample

- select records in the collection *bar* where the field name *age* has the value greate than *15*, then update them, the value of filed name *age* will increase 5 ,and the value of field name ID will increase 1.

```
db.foo.bar.update({$inc: {age: 5, ID: 1}}, {age: {$gt: 15}})
```

- select records in the collection bar where the field name *arr* existed and is array type,and set the value of second element to increase 1.

```
db.foo.bar.update({$inc: {"arr.1": 1}}, {arr: {$exists: 1}})
```

$set

Grammar

{$set:{<field name1>:<value1>,<filed name2>:<value2>,...}}

Description

$set is used to update the value of specified field name(field name1,field name2) to the specified value(value1,value2).If there is no the specified field name in the records,then fill the field name and value to the record,else ,update the value to the specified value.

Samples

- Select records which the field *age* is not exist,and use '$set' update those records.

```
db.foo.bar.update({$set: {age: 5, ID: 10}}, {age: {$exists: 0}})
```

- Update all the records in the collection *bar*,set the value of field *str* to "abc"

```
db.foo.bar.update({$set: {str: "abd"}})
```

- Using '$set' update the elment of nested array object.the field *arr* is a nested array object in collection *bar*,for example,there have two records:{arr:[1,2,3],name:"Tom"},{name:"Mike",age:20},and the second record doesn't exist *arr* field.

```
db.foo.bar.update({$set: {"arr.1": 4}}, {name: {$exists: 1}})
```

  This operation is to select records which caontain *name* field,then use '$set' to update the array object *arr*.If there is no array object arr in original record,then $set will add the *arr* filed as a array object to the record.The above two records will update as:

```
{arr: [1, 4, 3], name: "Tom"}, {arr: {"1": 4}, name: "Mike", age: 20}
```

$unset

Grammar

{$unset:{<field name1>:"",<field name2>:"",...}}

Description

$unset operation is used to delete the specified field name.if the specified field doesnot exist in record,then skipped.

Samples

- Delete the fields *name* and *age* of records in the collection *bar*,if a record does not contain the field *name* or *age*,then skipped,do nothing.

```
db.foo.bar.update({$unset: {name: "", age: ""}})
```

- Use '$unset' to delete the element of array object.There has a record for example:{arr: [1,2,3],name:"Tom"}.The operation use $unset to delete the second element as follows:

```
db.foo.bar.update({$unset: {"arr.2": ""}})
```

after this operation,the record update as:{arr:[1,null,3],name:"Tom"}

- Use $unset to delete the element of nested object.There has a record for example:{content:{ID:1,type:"system",position:"manager"},name:"Tom"},

  content is a nested object,it has two elements:ID and type.The operation use $unset to delete element *type* as follows:

  ```
  db.foo.bar.update({$unset: {"content.type": ""}})
  ```

  after this operation,the record update as:{content:{ID:1,position:"manager"},name:"Tom"}

$rename

Grammar

Description

Sample

$addtoset

Grammar

{$addtoset:{<field name1>:[<value1>,<value2>,...,<valueN>] , <field name2>:[<value1>,<value2>,...,<valueN>],...}}

Description

$addtoset is to add element and value to the array object, the operation object must be array type field. $addtoset has the following rule:

- the records have the specified fields(<field name1>,<field name2>...).

  if the specified values([<vaule1>,<value2>,...,<valueN>])exist in the records, skipping with no operations, only add values that doesn't exist to the target array.
- the records doesn't exist the specified field name.

  If the records itself doesn't exist the specified field name(<field name1>,<field name2>...), then add the specified field names and values to the records.

Sample

- the records exist in the target array object. As the follwoing record:{arr:[1,2,4],age:10,name:"Tom"}

  ```
  db.foo.bar.update({$addtoset: {arr: [1, 3, 5]}}, {arr: {$exists: 1}})
  ```

  after this operation, the record update as：{arr:[1,2,4,3,5],age:10,name:"Tom"}.in the previous record , these is no 3 or 5 element in the array object *arr*,after using *$addtoset* operator,update them to *arr*.
- the records don't exist in the specified array object,as the following record:{name:"Mike",age:12}

  ```
  db.foo.bar.update({$addtoset: {arr: [1, 3, 5]}}, {arr: {$exists: 0}})
  ```

  after this operation,the record update as:{arr:[1,3,5],age:10,name:"Tom"}.in the previous record,these is no array object *arr*,after using *$addtoset* operator,update the array object *arr* and it's elements to the record.

$pop

Grammar

{$pop:{<field name1>:<N>,<field name2>:<N>,...}}

Description

$pop operation is to delete the last of N elements in the array object(<field name1>,<field name2>,...), the operation object must be array type.If the records don't exist the specified array object,skipping with no operations;if N is greate than the length of array object,then the length of array object update to zero,that is all of the element will drop;if N is less than zero,means to delete the first of -N elements from the begining of array object.

Sample

- delete the last two of elements from the array object *arr* in the collection *bar*.As the following record: {arr:[1,2,3,4],age:20,name:"Tom"}

```
db.foo.bar.update({$pop: {arr: 2}})
```

  after this operation,the record update as:{arr:[1,2],age:20,name:"Tom"}
- delete the last ten of elements from the array object *arr* in the collection *bar*.as the following record: {arr:[1,2,3,4],age:20,name:"Tom"}

```
db.foo.bar.update({$pop: {arr: 10}})
```

  after this operation,the record update as:{arr:[],age:20,name:"Tom"}
- delete the first two of elements from the array object *arr* in the collection *bar*,that is set N to -2.as the following record:{arr:[1,2,3,4],age:20,name:"Tom"}

```
db.foo.bar.update({$pop: {arr: -2}})
```

  after this operation,the record update as:{arr:[3,4],age:20,name:"Tom"}

$pull

Grammar

{$pull:{<field name1>:<value1>,<field name2>:<value2>,...}}

Description

$pull operation is to delete the specified values(<value1>,<field name2>:<value2>,...) from the given array object field(<field name1>,<field name2>), the operation object must be array type field. If the specified array object don't exist in record,skipping with no operations; If the specified values don't exist in array object, also skipping with no operations.

Sample

- delete the elements which have the value *2* from the array object *arr* and have the value *"Tom"* from array object *name* in the collection *bar*, as the following record:*{arr[1,2,4,5],age:10,name:["Tom","Mike"]}*

```
db.foo.bar.update({$pull: {arr: 2, name: "Tom"}})
```

  after this operation,the record update as:*{arr[1,4,5],age:10,name:["Mike"]}*
- delete the elements which have the value *2* from the array object *arr* and have the value *"Tom"* from array object *name* in the collection *bar*, as the following record:*{arr[1,3,4,5],age:10,name:["Tom","Mike"]}*

```
db.foo.bar.update({$pull: {arr: 2, name: "Tom"}})
```

after this operation,the record update as:*{arr[1,3,4,5],age:10,name:["Mike"]}*. As these is no element which has the value *2* in the array object *arr* ,so object *arr* have no change.

$pull_all

Grammer

{$pull_all:{<field name1>:[<value1>,<value2>,...,<valueN>],<field name2>:[<value1>,<value2>,...,<valueN>],...}}

Description

$pull_all is to pull the given values([<value1>,<value2>,...,<valueN>]) of the specified array object(<field name1>),the operation object must be array type field.

if the specified array object does not exist in the records,skipping with no operations; if the specified values don't exist in array object,nor do any operation.

Sample

- delete the elements which have the value *2* or *3* from the array object *arr* and have the value *"Tom"* from array object *name* in the collection *bar*, as the following record:*{arr[1,2,4,5],age:10,name:["Tom","Mike"]}*

  ```
  db.foo.bar.update({$pull_all: {arr: [2, 3], name: ["Tom"]}})
  ```

  after this operation,the record update as:*{arr[1,4,5],age:10,name:["Mike"]}*
- delete the elements which have the value *4* or *5* from array object *arr* in the collection *bar*.as the following record:*{arr[1,3,4,5],age:10,name:["Tom","Mike"]}*

  ```
  db.foo.bar.update({$pull_all: {arr: [4, 5]}})
  ```

  after this operation,the record update as:*{arr[1,3],age:10,name:["Tom","Mike"]}*.

$push

grammar

{$push:{<field name1>:<value1>,<field name2>:<value2>,...}}

description

$push A value of (<value1>) is inserted into the giver destination array (<value1>),Operand must be an array type field. If the record does not exist in the specified field name, field names will be specified in the form of an array object pushed into the records and populate its specified value; if the record exists in the specified field name and field names specified value exists, specify the value will be pushed into the record.

example

- The collection bar under arr array object push the value 1. Original records exist elements of an array object arr, if records:{arr[1,2,4],age:10,name:["Tom","Mike"]}

  ```
  db.foo.bar.update({$push: {arr: 1}})
  ```

  After this operation, record updates: {arr [1,2,4,1], age: 10, name: ["Tom", "Mike"]}. Although there are elements of the original arr 1, using $push operator, or adds an element a push to arr array object.
- Push bar to the collection does not exist in an array of objects and values. Record does not exist in the original array object name, if any, records:{arr[1,2],age:20}

  ```
  db.foo.bar.update({$push: {name: "Tom"}}, {name: {$exixts: 0}})
  ```

After this operation, record updates: {arr [1,2], age: 20, name: ["Tom"]}. Record does not exist in the original array object name, use $push operator will name an object in the form of an array pushed into the record.

$push_all

Grammar

{$push_all:{<field name1>:[<value1>,<value2>,...,<valueN>],<field2>:[<value1>,<value2>,...,<valueN>],...}}

Description

$push_all is used to push each of the spefied values(*[<value1>,<value2>,...,<valueN>]*) to the specified array object.the operation object must be array type field.if the specified array object does not exist in the records,then push the array object and all the values(*[<value1>,<value2>,...,<valueN>]*);if the specified values exist in array object,push them to the specified array object as the same.

Samples

- Push array *[1,2,8,9]* to the array object *arr* in the collection.Therae has a record like this: {arr[1,2,4,5],age:10,name:["Tom","Mike"]}

```
db.foo.bar.update({$push_all: {arr: [1, 2, 8, 9]}})
```

after this operation,the record update as:{arr[1,2,4,5,1,2,8,9],age:10,name:["Mike"]},althougth,the *arr* object have elements *1* and *2* in original record,using $push_all will push all the elements of array *[1,2,8,9]* to array object *arr* .

- Push array object *name* to collection *bar*,assuming the original record doesnot exist array object *name* ,there has a record like this:{arr[1,3,4,5],age:10}

```
db.foo.bar.update({$push_all: {name: ["Tom", "Jhon"]}}, {name: {$exixts: 0}})
```

after this operation,the record update as:{arr[1,3,4,5],age:10,name:["Tom","Mike"]}。

SQL Grammar

Sequoiadb is a document-oriented type of non-relational database, this section mainly descibes how to use SQL to access and process data in the Sequoiadb system.

Note: in the sequoiaDB,SQL statement is not case sensitive.

SQL grammar table

statement	description	samples
create collectionspace	create collectionspace	db.execUpdate("create collectionspace foo")
drop collectionspace	drop collectionspace	db.execUpdate("drop collectionspace foo")
create collection	create collection	db.execUpdate("create collection foo.bar")
drop collection	drop collection	db.execUpdate("drop collection foo.bar")
create index	create index	db.execUpdate("create index test_index on foo.bar (age)")
drop index	drop index	db.execUpdate("drop index test_index on foo.bar")
list collectionspaces	list collectionspace	db.execUpdate("list collectionspaces")
list collections	list collection	db.execUpdate("list collections")

statement	description	samples
insert into	insert	db.execUpdate("insert into foo.bar(age,name) values(20,₩"Tom₩")")
select from	query	db.exec("select * from foo.bar")
update set	update	db.execUpdate("update foo.bar set age=25")
delete from	delete	db.execUpdate("delete from foo.bar")
group by	group	db.exec("select dept_no，*count*(emp_no) as 员工总数 from foo.bar group by dept_no ")
order by	sort	db.exec("select * from foo.bar order by age desc")
limit	limit the number of return records	db.exec("select * from foo.bar limit(5)")
offset	set the number of records skipped	db.exec("select * from foo.bar offset(5)")
as	aliases	
join on	join	
left outer join on	left join	
right outer join on	right join	
count	count	
sum	sum	
sum	average	
sum	max	
sum	min	

sql create collectionspace

create collectionspace

Be used to create collectionspace in the database .

Grammar

```
create collectionspace <cs_name>
```

<cs_name>：collectionspace name. the length of collectionspace can't over 128Byte,and collectionspace name can't be enpty.

Sample

- create collectionspace *foo*

```
db.execUpdate("create collectionspace foo") //Equivalent db.createCS("foo")
```

sql drop collectionspace

drop collectionspace

Be used to drop the collectionspace in the database.

Grammar

```
drop collectionspace <cs_name>
```

<cs_name>collectionspace name,the collectionspace name must be exist in the database.

Sample

- This sample will drop the collectionspace *foo*.

```
db.execUpdate("drop collectionspace foo") //Equivalent db.dropCS("foo")
```

sql create collection

create collection

Be used to create collectio, must specify the collection in which collectionspace.

Grammer

```
create collection <cs_name>.<cl_name>
```

<cs_name>：collectionspace in database

<cl_name>：collection name.the length of it can't exceed 127Byte,and collection name can't be empty ,and can't exist same collection name in a collectionspace.

Sample

- create collection *bar* in the collectionspace *foo*

```
db.execUpdate("create collection foo.bar") //Equivalent db.foo.createCL("bar")
```

sql drop collection

drop collection

Be used to drop the collection in the collectionspace.

Grammar

```
drop collection <cs_name>.<cl_name>
```

<cs_name>collectionspace name in the database.the collectionsapce name must be in the database.

<cl_name>collection name. collection name also must be in the collectionsapce.

Sample

- The following sample will drop the collection *bar* in the collectionspace *foo*.

```
db.execUpdate("drop collection foo.bar") //Equivalent db.foo.dropCL("bar")
```

sql create index

create index

Be used to create index in the collections.in the case of without reading collection,indexes enable database application to find data faster.

Grammer

```
create [unique] index <index_name> on <cs_name>.<cl_name> (field1_name [asc|desc],...)
```

[unique]：TO logo if the created index is unique.unique index means that the same field name can't have the same index name.

<index_name>：index name

<cs_name>：collectionspace name

<cl_name>：collection name

field1_name：field name where create index , with an index name can be created on multiple fields.

[asc|desc]：sort, asc stand for ascending index the value of a field.desc stand for descending index the value of a field.the default is asc.

Sample

- This sample will create a simple index, the name is "test_index" on the field name *age* in the collectionspace *foo*,collection *bar*.

```
db.execUpdate("create index test_name on foo.bar (age)")
```

  if want to descending index the value of a field,can add the reserved word *desc* after the field name.

```
db.execUpdate("create index test_name on foo.bar (age desc)")
```

  if want to the index in more than one field ,can list all the field name in the brackets,and separated by commas.

```
db.execUpdate("create index test_name on foo.bar (age desc,name asc)")
```

- the following sample will create a unique index on the field *age*.

```
db.execUpdate("create unique index test_name on foo.bar (age)")
```

sql drop index

drop index

Be used to drop the index in the collection.

Grammar

```
drop index <index_name> on <cs_name>.<cl_name>
```

<index_name>：index name

<cs_name>：collectionspace name

<cl_name>：collection name

Sample

- The following sample will drop the index name "test_index" in the collectionspace *foo*,collection *bar*.Assume the index name is already exist.

```
db.execUpdate("drop index test_index on foo.bar") //Equivalent db.foo.bar.dropIndex("test_index")
```

sql list collectionspaces

list collectionspaces

Be used to list all the collectionspaces in the database.

Grammar

```
list collectionspaces
```

Sample

- This sample will return all the collectionspaces in the database.

```
db.exec("list collectionspaces")
result:
{
  "Name": "testfoo"
  "Name":"big"
```

```
  ...
}
```

sql list collections

list collections

Be used to list all the collections in the collectionspace.

Grammar

```
list collections
```

Sample

- This sample will return all the collections.

```
db.exec("list collectionspaces")
result:
{
  "Name": "testfoo.testbar"
  "Name":"big.small"
  ...
}
```

sql insert into

insert into

Be used to insert new records to the collection.

Grammar

```
insert into <cs_name>.<cl_name>(<field1_name,field2_name,...>) values(<value1,value2,...>)
or
insert into <cs_name>.<cl_name> <select_set>
```

<cs_name>：collectionspace name

<cl_name>：collection name

<field_name>：field name

<value>：values the field name corresponding

<select_set>：query result set

Sample

- This sample will insert one record to the collection *bar*,field names are *age* and *name*,corresponding values are *25* and *"Tom"*.

```
db.execUpdate("insert into foo.bar(age,name) vaules(25,\"Tom\")")
```

- This sample will insert batch records to the collection *bar*,these records is the result set select from collection *small*.

```
db.execUpdate("insert into foo.bar select * from big.small")
```

sql select

select

Be used to select data from collection,the result will be storage as a result set.

Grammar

```
select * from <cs_name>.<cl_name>
or
select <field1_name,field2_name,...> from <cs_name>.<cl_name>
```

<cs_name> : collectionspace name

<cl_name> : collection name

<field_name> : field name

Sample

- This sample will return the select field name.if a match record doesnot contain the specified field name,then return *null* value .

```
db.exec("select age,name from foo.bar")
result:
{
  "age": 10,
  "name": null
}
{
  "age": 10,
  "name": "Tom"
}
}...
```

- This sample will return all records of all the field name .

```
db.exec("select * from foo.bar")
result:
{
  "_id": {
    "$oid": "51c909b0c5b855e029000000"
  },
  "age": 10
}
{
  "_id": {
    "$oid": "51c909b9c5b855e029000001"
  },
  "age": 10,
  "name": "Tom"
}
{
  "_id": {
    "$oid": "51c909c2c5b855e029000002"
  },
  "age": 10,
  "name": "Tom",
  "phone": 123456
}...
```

Note:

1.you can select likewhere、group by、order by、limit、offsetkeywords to select those records you want.

2.if the query source is not collection,then all the field name in this layer query need to use alias (* except)

example:select T.a , T.b from (select * from foo.bar) as T where T.a<10

3.subquery must use alias.the alias appear in subquery only act on upper layer.

sql update

update

Be used to modify records in the collection.

Grammar

```
update <cs_name>.<cl_name> set (<field1_name>=<value1>,...) [where <condition>]
```

<cs_name> : collectionspace name

<cl_name> : collection name

<condition> : condition,only update the records match the condition.

Sample

- This sample will modify all the records in the collection,the value of field name *age* will be update *20*,if a records doesnot contain feild *age*,then 'age:20' will be add to the record.

```
db.execUpdate("update foo.bar set age=20")
```

- This sample will be modify the match records,only update the records that satisfy the condition *age<10*.

```
db.execUpdate("update foo.bar set age=20 where age<10")
```

sql delete

delete

Be used to delete records in the collection.

Grammar

```
delete from <cs_name>.<cl_name> [where <condition>]
```

<cs_name> : collectionspace name

<cl_name> : collection name

<condition> : condition,only delete the records which match the condition.

Sample

- This sample will delete all the records in the collection.

```
db.execUpdate("delete from foo.bar")
```

- This sample will delete the records which match the condition *age<10*.

```
db.execUpdate("delete from foo.bar where age<10")
```

sql group by

group by

Be used to group result set According to one or more filed name combined with aggragate function.

Grammar

```
group by <field1_name [ASC|DESC ], ...>
```

<field_name> : field name

[asc|desc] : sort, asc stand for ascending,desc stand for descending. default asc.

Sample

- Calculating the number of employees in each department and group by the field name *dept_no*.

```
db.exec("select dept_no, count(emp_no) as 员工总数 from foo.bar group by dept_no")
```

Note:

likesum，count，min，max，avgthese counting function must use alias.

sql order by

order by

Be used to sort the result set according to the specified field name,default asc.

Grammar

```
order by <field1_name [ASC|DESC ], ...>
```

<field_name>：field name

[asc|desc]：sort, asc stand for ascending,desc stand for descending. default asc

Sample

- To count the number employees in each department ,and group by the feild name dept_no,and sort in desc order by the field name.

```
db.exec("select dept_no, count(emp_no) as 员工总数 from foo.bar group by dept_no order by dept_no desc")
```

Note:

likesum，count，min，max，avg these counting function must use alias.

sql limit

limit

Be used to limit the number of returned records.

Grammar

```
limit<limit_num>
```

<limit_num>：limit number

Sample

- This sample will return the first 10 records from the result set.

```
db.exec("select * from foo.bar limit 10")
```

sql offset

offset

Be used to set the number of skipped records.

Grammar

```
offset<offset_num>
```

<offset_num>：the number of skipped records

Sample

- This sample will skip the first five records,then return from the sixth record.

```
db.exec("select * from foo.bar offset 5")
```

sql as

As

Be used to specify alias for the collections or field names.

Grammar

```
<cs_name.cl_name | (select_set) | field_name> AS <alias_name>
```

<cs_name>：collectionspace name

<cl_name>：collection name

select_set：result set

field_name：field name

<alias_name>:alias name

Sample

- specify alias for collection

  ```
  db.exec("select T1.age,T1.name from foo.bar as T1 where T1.age>10")
  ```

- specify alias for field

  ```
  db.exec("select age as 年龄 from foo.bar where age>10")
  ```

- specify alias for result set

  ```
  db.exec("select T.age,T.name from (select age,name from foo.bar) as T where T.age>10")
  ```

sql inner join

inner join

According to two or more collections in the relationship between the field name,inner join is used to query data from those collections.

Grammar

```
<collection1_name | (select_set1) as <alias1_name>
inner join
<collection2_name | (select_set2)> as <alias2_name>
[ON condition]
```

Sample

- There have employee information table *foo.emp* and depatmenrt information table *foo.dept*, query the employee number filed *emp_no* in which department name field *dept_name*.

  ```
  db.exec("select E.emp_no,D.dept_name from foo.emp as E inner join foo.dept as D on E.dept_no=D.dept_no")
  ```

Note:

1.can't contain non-union conditions,the following is a wrong way.

```
select T1.a,T2.b from foo.bar1 as T1 inner join foo.bar2 as T2 on T1.a<10
```

2.can't use 'select *' statement in join layer.

sql left outer join

left outer join

left outer join will return all the records from the left collection name(collection1_name),even though there is no matched records in the right collection name(collection2_name).

Grammar

```
<collection1_name | (select_set1) as <alias1_name>
left outer join
<collection2_name | (select_set2)> as <alias2_name>
[ON condition]
```

Sample

- There have employee information table *foo.emp* and depatmenrt information table *foo.dept*, query the employee number filed *emp_no* in which department name field *dept_name*.

```
db.exec("select E.emp_no,D.dept_name from foo.emp as E left outer join foo.dept as D on E.dept_no=D.dept_no where
 D.dept_no<4")
```

sql right outer join

right outer join

right outer join will return all the records from the right collection name(collection2_name),even though there is no matched records in the left collection name(collection1_name).

Grammar

```
<collection1_name | (select_set1) as <alias1_name>
right outer join
<collection2_name | (select_set2)> as <alias2_name>
[ON condition]
```

Sample

- There have employee information table *foo.emp* and depatmenrt information table *foo.dept*, query the employee number filed *emp_no<10* in which department name field *dept_name*.

```
db.exec("select E.emp_no,D.dept_name from foo.emp as E right outer join foo.dept as D on E.dept_no=D.dept_no where
 E.emp_no<10")
```

sql sum()

sum()

Be used to sum for the specified field name.

Grammar

```
sum(field_name) as <alisa_name>
```

Note: 1.when use sum() function,you must use aliases.

2.automatically skipped for the non-numeric field.

Sample

- This sample will sum for the field name *age* in the collection *bar*.

```
db.exec("select sum(age) as 年龄总和 from foo.bar")
```

sql count()

count() function

Be used to count, return the number of records that match the specified field.

Grammar

```
count(field_name) as <alisa_name>
```

Note: 1.When use count function to count , you must use alisa name .

Sample

- counting for the field name *age* in the collection *bar*.

```
db.exec("select count(age) as amount from foo.bar")
```

sql avg()

avg() function

Be userd to calculate average for all values of the specified field.

Grammar

```
avg(field_name) as <alisa_name>
```

Note: 1.When use avg function to calculate average for the field , you must use alisa name .

2.It will auto skip for non-numeric .

Sample

- calculating average for the field name *age* in the collection *bar*.

```
db.exec("select avg(age) as 平均年龄 from foo.bar")
```

sql max()

max() function

Be used to return the maximum value of the specified field name.

Grammar

```
max(field_name) as <alisa_name>
```

Note: 1.when use max() to return the max value of the specified field name,you must use aliases.

Sample

- Return the max value of the field name*age*.

```
db.exec("select max(age) as 最大年龄 from foo.bar")
```

sql min()

min() function

Be used to return the min value of the specified field name.

Grammar

```
min(field_name) as <alisa_name>
```

Note: 1.when use min() to return the min value of the specified field name,you must use aliases.

Sample

- Return the min value of the field name *age*.

```
db.exec("select min(age) as 最小年龄 from foo.bar")
```

Mapping Table from SQL to sequoiadb

Concepts and terms

SQL	SequoiaDB
database	database
table	collection
row	document / BSON document
column	field
index	index
table joins	embedded documents
primary key (Users can take any unique column as primary key.)	primary key In Sequoiadb, primary key is automatically as the field called "_id".

Create and Alter

The following table shows create and alter statements on table level in SQL and the corresponding ones in Sequoiadb.

SQL Statements	Sequoiadb Statements	Relative Link
create table student (id not null, stu_id varchar(50), age number primary key(id))	The system will automatically generate a collection when data is firstly inserted. If the value on "_id" is not specified, the system will automatically generate a "_id" value. db.collectionspace.student({stu_id:"01",age:20}) Of course, you can also create a collection manually db.collectionspace.createCL("student")	insert() , createCL()
alter table student add name varchar(50)	In a collection, the structure is not changable, because there is no relative manipulation to describe or change the structure. But the method update() can add new field to document record with "$set".	update() , $set

SQL Statements	Sequoiadb Statements	Relative Link
	db.collectionspace.student.update({},{$set:{name:"Tom"}})	
alter table student drop column name	In a collection, the structure is not changable, because there is no relative manipulation to describe or change the structure. But the method update() can delete existing field from document record with "$unset". db.collectionspace.student.update({},{$unset:{name:"Tom"}})	update() , $unset
create index index_stu_id on student (stu_id)	db.collectionspace.student.createIndex("index_stu_id",{stu_id:-1})	createIndex() , index
drop table student	db.collectionspace.dropCL("student")	dropCL()

Insert

The following table shows insert statement on table level in SQL and the corresponding one in Sequoiadb.

SQL Statements	Sequoiadb Statements	Relative Link
insert into student(stu_id,age) values("01",20)	db.collectionspace.student.insert({stu_id:"01",age:20})	insert()

Select

The following table shows read statement on table level in SQL and the corresponding one in Sequoiadb.

SQL Statements	Sequoiadb Statements	Relative Link
select * from student	db.collectionspace.student.find()	find()
select stu_id,age from student	db.collectionspace.student.find({},{stu_id:"01",age:20})	find()
select * from student where age>25	db.collectionspace.student.find({age:{$gt:25}})	find() $gt
select age from student where age=25 and stu_id="01"	db.collectionspace.student.find({age:25,stu_id="01"},{age:25})	find()
select count(*) from student	db.collectionspace.student.count()	count()
select count(stu_id) from student	db.collectionspace.student.count({stu_id:{$exists:1}})	count() , $exists

Update

The following table shows update statement on table level in SQL and the corresponding one in Sequoiadb.

SQL Statements	Sequoiadb Statements	Relative Link
update student set age=25 where stu_id="01"	db.collectionspace.student.update({stu_id:"01"},{$set:{age:25}})	update() , $set
update student set age=age+2 where stu_id="01"	db.collectionspace.student.update({stu_id:"01"},{$inc:{age:2}})	update() , $inc

delete

The following table shows delete statement on table level in SQL and the corresponding one in Sequoiadb.

SQL Statements	Sequoiadb Statements	Relative Link
delete from student where age=20	db.collectionspace.student.remove({age:20})	remove()
delete from student	db.collectionspace.student.remove()	remove()

Limits

Document

Description	Limits
Minimum length of document	At least contains one field
Maximum length of document	When document is transformed into BSON, the length should be lesser than 16777168 bytes.
Field name	It should not be started with "$". It should not contain ".".

Collection

Description	Limits
Maximum length of collection name	127 bytes
Collection name	It should not be started with "$" or "SYS". It should not contain ".".
Maximum capacity of collection consisted of single node	It is the maximum capacity of collection space
Maximum amount of collections in a collection space consisted of single node	4096

Collection Space

Description	Limits
Maximum length of collection space name	127 bytes
Collection space name	It should not be started with "$" or "SYS". It should not contain ".".
Size of data page	4096, 8192, 16384, 32768, 65536
Maximum capacity of collection space consisted of single node	It can be 64GB, 128GB, 256GB, 512GB, 1TB.
Maximum amount of standalone-node collection space	4096

Index

Description	Limits
Maximum length of each data index key	1024 bytes
Total length of index (It includes index name, index key, etc.) .	When an index is transformed into BSON, the length should be equal to or lesser than the size of data page minus 48 bytes.
Multiple index	It can contain all kinds of legal fields. But it contains at most one field which contains array.
Order value of index key	1 or -1
Maximum amount of index in one collection	64

Database

Description	Limits
Minimum size of log file	32MB

Description	Limits
Maximum size of log file	2GB

Node

Description	Limits
Maximum amount of nodes in each replset	7
Create node	Nodes should be created through hostname rather than IP address.
Network	All the systems in a cluster should be able to visit each other through hostname.
Vote condition of master node	Vote will not be carried out unless more than half of nodes in replset take part in it.

Shard

Description	Limits
Data split	Each moment, only a range of data can be split in each collection.
Shard key	The value of shard key is unchangable after data is inserted.
_id	"_id" in shard collection is unique in replset, but maybe not globally unique.
Unique index	It should contain all fields in sharding key.

Driver

Description	Limits
Thread-safe	Each connection object and its subobject are not thread-safe to each other. Different connections are thread-safe to each other.

Error Code List

Description	Error Code
IO Exception	-1
Out of Memory	-2
Permission Error	-3
File Not Exist	-4
File Exist	-5
Invalid Argument	-6
Invalid size	-7
Interrupt	-8
hit end of file	-9
system error	-10
has no space	-11
EDU status is not valid	-12
Timeout error	-13
Database is quiesced	-14
Network error	-15
Network is closed from remote	-16
Database is in shutdown status	-17
Application is forced	-18
Given path is not valid	-19
Unexpected file type specified	-20
There's no space for DMS	-21
collection already exist	-22
collection does not exist	-23

Description	Error Code
user record is too big	-24
record does not exist	-25
remote overflow record exist	-26
invalid record	-27
storage unit need reorg	-28
end of collection	-29
context is already opened	-30
context is closed	-31
option is not supported yet	-32
collection space already exist	-33
collection space not exist	-34
storage unit file is invalid	-35
context not exist	-36
more than one field has array	-37
duplicate key exist	-38
key is too large	-39
index extent has no space	-40
index key not exist	-41
hit max number of index	-42
failed to initialize index	-43
collection is dropped	-44
two records get same key and rid	-45
duplicate index name	-46
index name doesn't exist	-47
index flag is unexpected	-48
hit end of index	-49
hit max of dedup buffer	-50
invalid predicates	-51
index is no longer exist	-52
index hint is not valid	-53
no more temp tables avaliable	-54
exceed max number of SU	-55
$id index can't be dropped	-56
log was not found in log buf	-57
log was not found in log file	-58
replication group not exist	-59
replication group exist	-60
invalid request id is received	-61
session ID does not exist	-62
system edu can't be forced	-63
database is not connected	-64
unexpected result received	-65
corrupted record	-66
backup has already started	-67
backup not completed	-68
backup in progress mode	-69
backup file is damaged	-70
there's no primary found	-71
the requested node not exist	-72
engine help argument is specified	-73
connection state is not valid	-74
invalid handle	-75
client object is freed	-76
transfer has already listened	-77
can not listen specified addr	-78
cannot connect to specified addr	-79
connection does not exist	-80
failed to send	-81

Description	Error Code
timer id not found	-82
route info not found	-83
broken msg	-84
invalid net handle	-85
reorg file is not valid	-86
reorg file is in read only mode	-87
collection status is not valid	-88
collection is not in reorg state	-89
replication group is not activated	-90
the member is not in this group	-91
collection flag not compatible	-92
stroage unit version not compatible	-93
local group version is expired	-94
page size is not valid	-95
remote group version is expired	-96
failed to create a poll	-97
log record is corrupted	-98
given lsn greater than latest	-99
received unknown mesage	-100
updated information is same as old one	-101
unknow message	-102
empty heap	-103
not primary	-104
data node not enough	-105
data node have no catalog info	-106
data node catlog version old	-107
coord node catalog version old	-108
exceds the max group size	-109
failed to sync log	-110
failed to replay log	-111
error http struct	-112
failed to negotiate	-113
failed to change dps metadata	-114
SME is corrupted	-115
application is interrupted	-116
application is disconnected	-117
character encoding errors	-118
get failed msg from the node	-119
buffer array is full	-120
sub context is conflict	-121
coord received EOC message	-122
DPS file size are not the same	-123
dps file not recognise	-124
no resource	-125
invalid lsn	-126
data to pipe is too big	-127
catalog auth failed	-128
node is full sync	-129
catnode failed assign data-node	-130
php driver internal error	-131
failed to send the message	-132
there is no node-group register in catalogue-node	-133
remote-node disconnected	-134
couldn't find the match catalogue-info	-135
update catalog failed	-136
received invalid request which opcode is unknowned	-137
coord couldn't find the group-info in local	-138
dms extent is corrupted	-139

Description	Error Code
remote cluster manage failed	-140
remote engines have been stopped partially	-141
sdb service is starting	-142
sdb service has already been started	-143
sdb service is restarting	-144
sdb node is already existed	-145
sdb node is not existed	-146
unable to lock	-147
dms state is not compatible with current command	-148
rebuild has already started	-149
rebuild in progress mode	-150
there is no data in the coord-node's cache	-151
errors occur during the process of evalution	-152
group already exist	-153
group does not exist	-154
node does not exist	-155
failed to start the node	-156
node'configure conflicts	-157
group is empty	-158
The operation is for coord node only	-159
failed to operate on node only	-160
The mutex job already exist	-161
The specified job does not exist	-162
The catalog information is corrupted	-163
$shard index can't be dropped	-164
The command can't be run in the node	-165
The command can't be run in the serice plane	-166
The group info not exist	-167
Group name is conflict	-168
The collection is not sharded	-169
The record does not contains valid sharding key	-170
A task that already exists does not compatible with the new task	-171
The collection does not exists on the specified group	-172
The specified task does not exist	-173
The record contains more than one sharding key	-174
The mutex task already exist	-175
The split key is not valid or not in the source group	-176
The unique index must include all fields in sharding key	-177
Sharding key cannot be updated	-178
authority is forbidden	-179
There is no catalog address specified by user	-180
current record has been deleted	-181
search condition cannot match any records	-182
index page is reorged and the pos got different lchild	-183
There are duplicate field name exists in the record	-184
Too many records to be inserted at once	-185
Sort-Merge Join only supports equal predicates	-186
Trace is already started	-187
Trace buffer does not exist	-188
Trace file is not valid	-189
transaction-lock is incompatible	-190
system is doing rollback	-191
Bad record when doing sdb import	-192
repeat var name was found	-193
column field is ambiguous	-194
have an error in sql syntax	-195
invalid transactional operation	-196
append to lock-wait-queue	-197

Description	Error Code
record is deleting	-198
index is dropped or invalid	-199
repeat to create catalog-group	-200
parse json error	-201
parse CSV error	-202
log file is out of size	-203
can not remove the only node in the group	-204
need to manually complete the cleanup	-205
can not remove node or group of catalog	-206
group is not exist	-207
can not remove the group with data in it	-208
end of queue	-209
collection has not sharding index, can not split by percent	-210
the param field not exist	-211
too many trace break points are specified	-212
prefetchers are all busy	-213
domain not exist	-214
domain already exist	-215
group is not in domain	-216
sharding type is not hash	-217
split percent is lower	-218
task already finished, can not be caceled	-219
collection is loading	-220
load is error, and rollback	-221
routeID is different from the local	-222
service already exists	-223
not find field	-224
csv field line end	-225
unknow file type	-226
not all nodes are successful to export configuration	-227
this node is not primary and has no data	-228
secret value not same with data unit	-229
engine version argument is specified	-230
sdb help argument is specified	-231
sdb version argument is specified	-232
store procedure is not exist	-233